

尼恩大架构 最强环境 系列文章

一键打造 本地elk 实操环境:

[ELK日志平台 \(elasticsearch+logstash+kibana\) 原理和实操 \(史上最全\)](#)

高级开发必备, 架构师必备

一键打造 本地Prometheus+Grafana 实操环境:

[Prometheus+Grafana \(史上最全\)](#)

高级开发必备, 架构师必备

一键打造 本地centos 虚拟机环境:

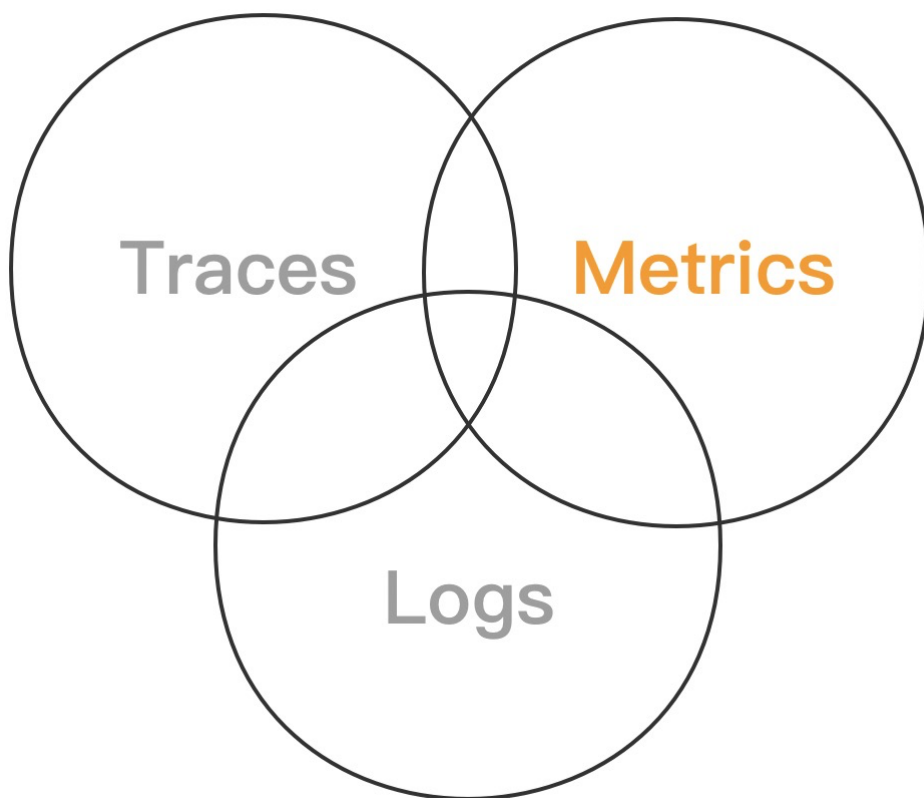
地表最强 开发环境: [vagrant+java+springcloud+redis+zookeeper镜像下载\(&制作详解\)](#)

高级开发必备, 架构师必备

尼恩大架构的 精彩文章大汇总:

[疯狂创客圈 JAVA 高并发 总目录](#)

什么是性能可观测



可观测性包括 Metrics、Traces、Logs 3 个维度。可观测能力帮助我们在复杂的分布式系统中快速排查、定位问题, 是分布式系统中必不可少的运维工具。

在性能压测领域中，可观测能力更为重要，除了有助于定位性能问题，其中Metrics性能指标更直接决定了压测是否通过，对系统上线有决定性左右，具体如下：

• Metrics，监控指标

系统性能指标，包括请求成功率、系统吞吐量、响应时长

资源性能指标，衡量系统软硬件资源使用情况，配合系统性能指标，观察系统资源水位

• Logs，日志

施压引擎日志，观察施压引擎是否健康，压测脚本执行是否有报错

采样日志，采样记录 API 的请求和响应详情，辅助排查压测过程中的一些出错请求的参数是否正常，并通过响应详情，查看完整的错误信息

• Traces，分布式链路追踪用于性能问题诊断阶段，通过追踪请求在系统中的调用链路，

定位报错 API 的报错系统和报错堆栈，快速定位性能问题点

本篇阐述如何使用 Prometheus 实现性能压测 Metrics 的可观测性。

系统监控的核心指标

系统性能指标

压测监控最重要的 3 个指标：请求成功率、服务吞吐量(TPS)、请求响应时长(RT)，这 3 个指标任意一个出现拐点，都可以认为系统已达到性能瓶颈。

这里特别说明下响应时长，对于这个指标，用平均值来判断很有误导性，因为一个系统的响应时长并不是平均分布的，往往会出现长尾现象，表现为一部分用户请求的响应时间特别长，但整体平均响应时间符合预期，这样其实是影响了一部分用户的体验，不应该判断为测试通过。因此对于响应时长，常用 99、95、90 分位值来判断系统响应时长是否达标。

另外，如果需要观察请求响应时长的分布细节，可以补充请求建联时长(Connect Time)、等待响应时长(Idle Time)等指标。

资源性能指标

压测过程中，对系统硬件、中间件、数据库资源的监控也很重要，包括但不限于：

- CPU 使用率
- 内存使用率
- 磁盘吞吐量
- 网络吞吐量
- 数据库连接数
- 缓存命中率
-

压测施压机性能指标

压测链路中，施压机性能是容易被忽略的一环，为了保证施压机不是整个压测链路的性能瓶颈，需要关注如下施压机性能指标：

- 压测进程的内存使用量
- 施压机 CPU 使用率，Load1、Load5 负载指标
- 基于 JVM 的压测引擎，需要关注垃圾回收次数、垃圾回收时长

什么是 Load1、Load5 负载指标？

在Linux机器上，有多个命令都可以查看机器的负载信息。其中包括uptime、top、w等。

uptime命令

uptime命令能够打印系统总共运行了多长时间和系统的平均负载。

uptime命令可以显示的信息显示依次为：现在时间、系统已经运行了多长时间、目前有多少登陆用户、系统在过去的1分钟、5分钟和15分钟内的平均负载。

```
[root@cdh1 ~]# uptime
17:35:43 up 0 min,  2 users,  load average: 4.10, 0.92, 0.30

[root@cdh1 ~]# uptime
17:35:43 up 23:41, 3 users, load averages: 1.74 1.87 1.97
```

这行信息的后半部分，显示“load average”，它的意思是“系统的平均负荷”，

里面有三个数字，我们可以从中判断系统负荷是大还是小。1.74 1.87 1.97 这三个数字的意思分别是1分钟、5分钟、15分钟内系统的平均负荷。**我们一般表示为load1、load5、load15。**

一般情况下，大家最好根据自己机器的实际情况，建立一个指标的基线（如近一个月的平均值），只要日常的load在基线上下范围内不太大都可以接收，如果差距太多可能就要人为介入检查了。

大部分情况下的建议的值：

当系统负荷持续大于0.75，你必须开始调查了，问题出在哪里，防止情况恶化。

当系统负荷持续大于1.0，你必须动手寻找解决办法，把这个值降下来。

当系统负荷达到5.0，就表明你的系统有很严重的问题，长时间没有响应，或者接近死机了。你不应该让系统达到这个值。

以上指标都是基于单CPU的，但是现在很多电脑都是多核的。所以，对一般的系统来说，是根据cpu数量去判断系统是否已经过载（Over Load）的。如果我们认为0.75算是单核机器负载的安全线的话，那么四核机器的负载最好保持在 $3(4 \times 0.75 = 3)$ 以下。

还有一点需要提一下，在Load Avg的指标中，有三个值，1分钟系统负荷、5分钟系统负荷，15分钟系统负荷。我们在排查问题的时候也是可以参考这三个值的。

一般情况下，1分钟系统负荷表示最近的暂时现象。15分钟系统负荷表示是持续现象，并非暂时问题。如果load15较高，而load1较低，可以认为情况有所好转。反之，情况可能在恶化。

立体式监控系统选型

一套全新的立体式监控系统，能够集业务服务、架构中间件、运维资源于一体，简单易用，同时便于维护。

选型主要针对 Cat、夜莺、Prometheus 做调研选型。

	Cat	夜莺	Prometheus
Contributors	77	56	643
Star	16.3k	4k	40.2k
偏向性	链路监控、日志监控	资源监控	任何业务指标与中间件监控
功能	一般	简单，但不够强大、不够灵活	接入Grafana后功能丰富，十分灵活
存储	HDFS	官方强烈推荐M3DB	任何实现Prometheus远端存储协议的存储
开源时间	2011	2020	2012
数据上报方式	埋点构建消息树	调用Http接口推送本地Agent	Prometheus server主动pull
社区活跃度	一般	一般	活跃

总体来说，

- Cat 更适合链路监控，
- 夜莺更像一个简化版的 Prometheus+Grafana，
- 而 Prometheus 拥有非常灵活的 PromQL、完善的 Exporter 生态

最终选择了 Prometheus。

什么是prometheus

prometheus:当前一套非常流行的开源监控和报警系统。

prometheus的运行原理

通过HTTP协议周期性抓取被监控组件的状态。输出被监控组件信息的HTTP接口称为exporter。

常用组件大部分都有exporter可以直接使用，比如haproxy,nginx，Mysql,Linux系统信息（包括磁盘、内存、CPU、网络等待）。

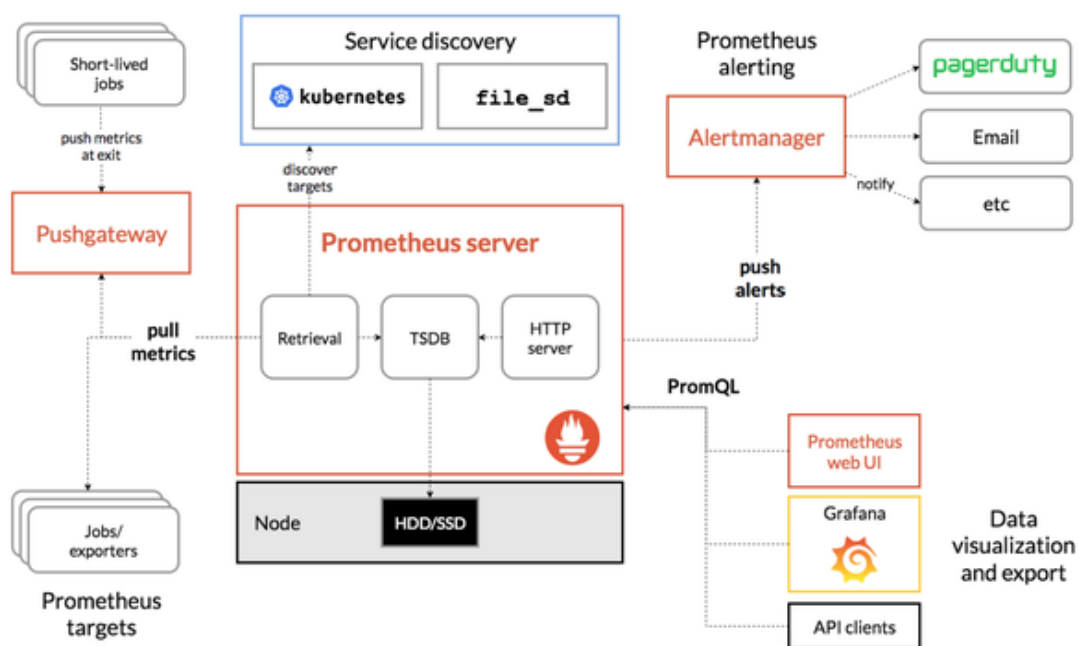
prometheus主要特点

- 一个多维数据模型（时间序列由metrics指标名字和设置key/value键/值的labels构成）。
- 非常高效的存储，平均一个采样数据占~3.5字节左右，320万的时间序列，每30秒采样，保持60天，消耗磁盘大概228G。
- 一种灵活的查询语言（PromQL）。
- 无依赖存储，支持local和remote不同模型。
- 采用http协议，使用pull模式，拉取数据。

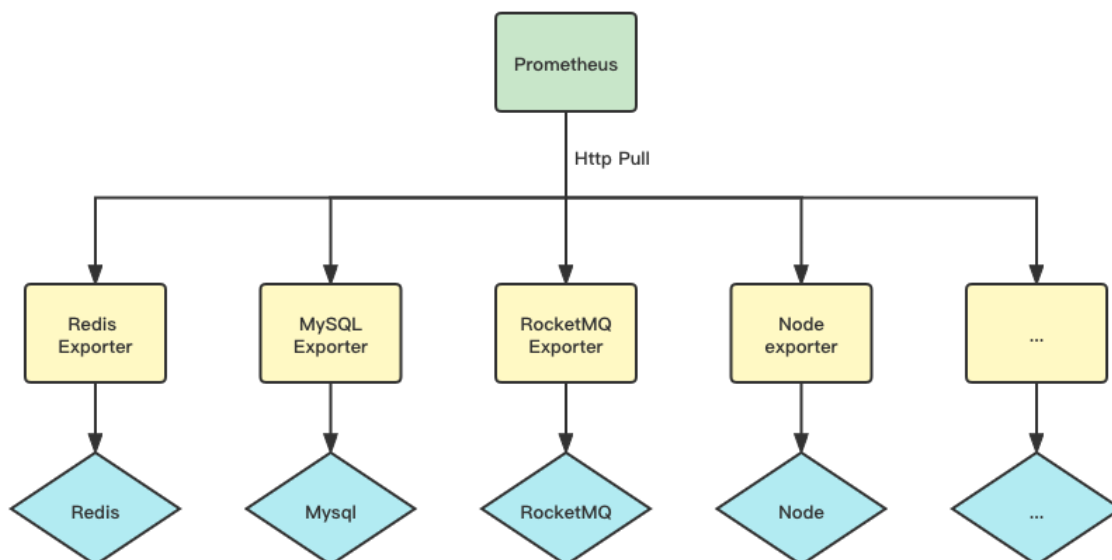
- 监控目标，可以采用服务器发现或静态配置的方式。
- 多种模式的图像和仪表板支持，图形化友好。
- 通过中间网关支持推送时间。

Prometheus 生态体系

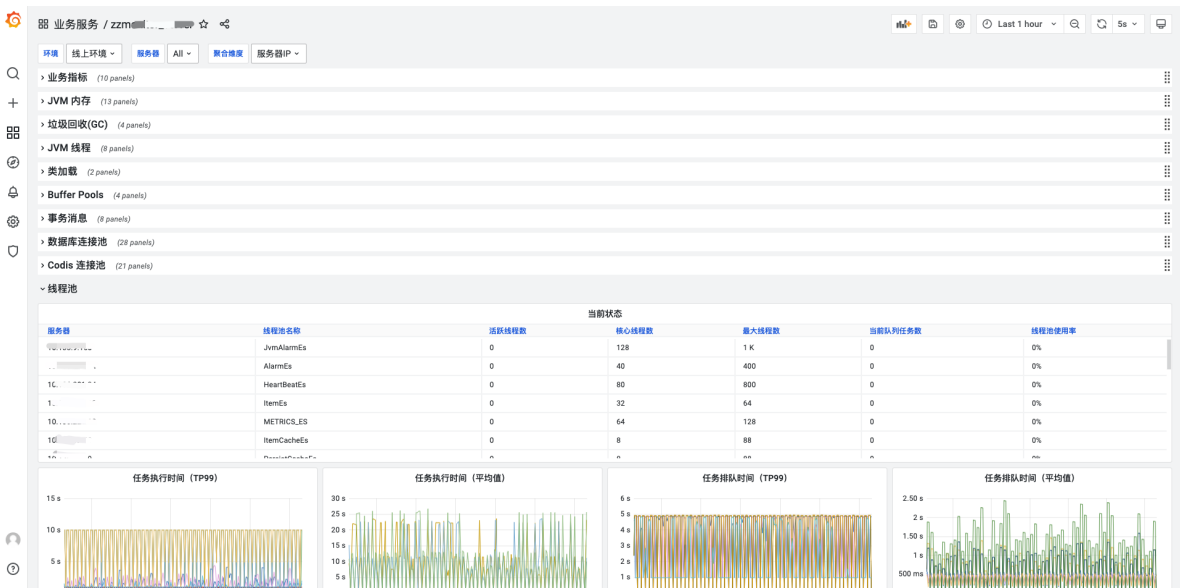
Prometheus 自带一个单机的 TSDB，他以 pull 模式抓取指标，被抓取的目标需要以 Http 的方式暴露指标数据。对于业务服务，服务需要将地址注册到注册中心，Prometheus 做服务发现，然后再做指标抓取。



Prometheus 拥有非常完善的 Exporter 生态，大部分我们使用的中间件都有成熟的 Exporter，利用这些 Exporter，我们可以快速的搭建起监控体系。



Prometheus 的黄金搭档 Grafana 在可视化方面表现也十分不错，Grafana 拥有非常丰富的面版、灵活易用。以下是我们落地的实际效果图。



Prometheus 能力

Prometheus 从客户端角度区分出了几种指标类型：Counter、Gauge、Histogram、Summary（转转内部不建议使用）。

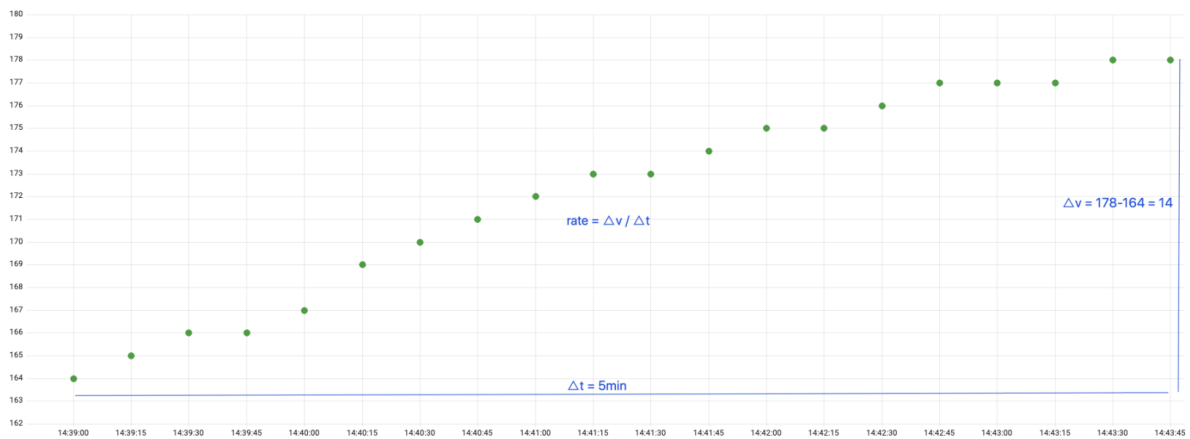
1 Counter

Counter 是一个只增不减的计数器，Prometheus 抓取的是 Counter 当前累计的总量。常见的如 GC 次数、Http 请求次数都是 Counter 类型的监控指标。

```
Counter counter = Counter.build().name("upload_picture_total").help("上传图片数").register();

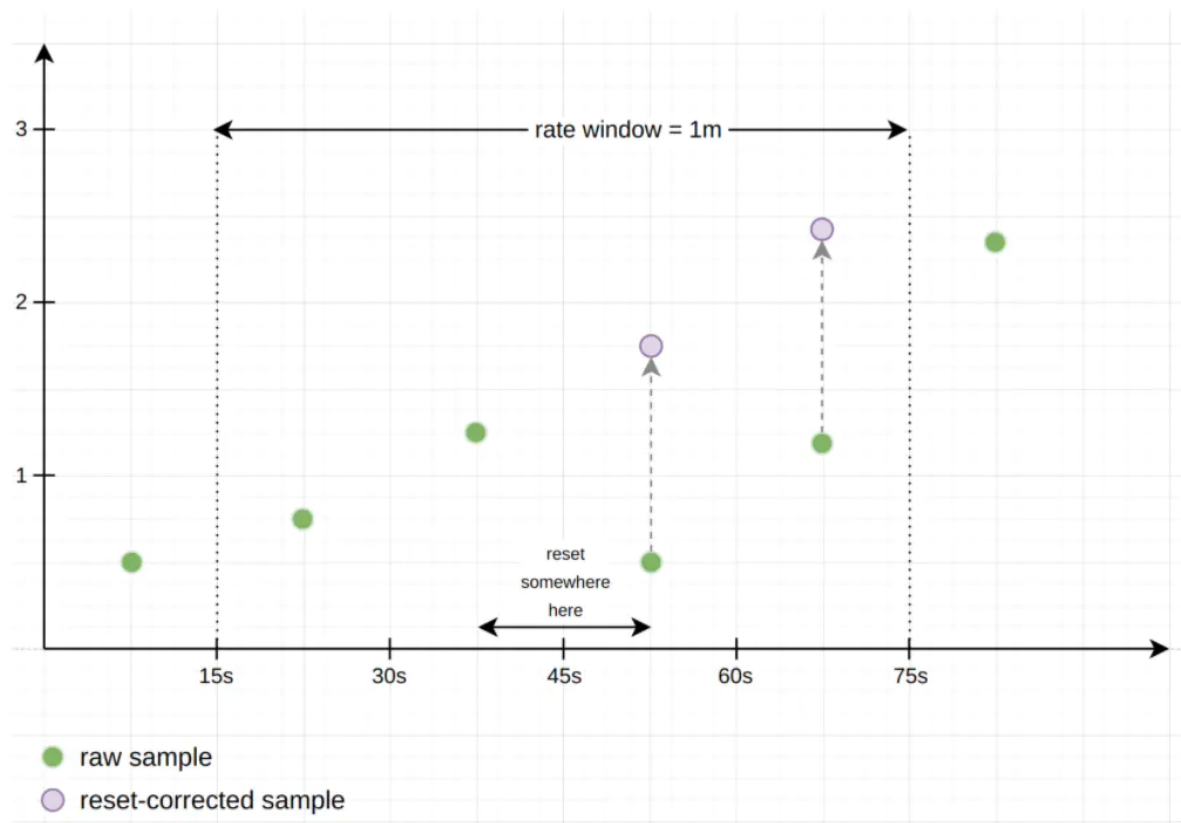
counter.inc();
```

以下是 Counter 的样本数据特点，我们可以在查询时计算出任意一段时间的增量，也可以计算任意一段时间的增量/时间，即 QPS。



如果服务器重启，Counter 累计计数归零，Prometheus 还能计算出准确的增量或 QPS 吗？

Prometheus 称这种情况为 Counter 重置，Counter 在重置后总是从 0 开始，那么根据这个假设，在给定的时间窗口计算增量时，只需将重置后的样本值叠加上重置前的值，以补偿重置，就像重置从未发生过一样。



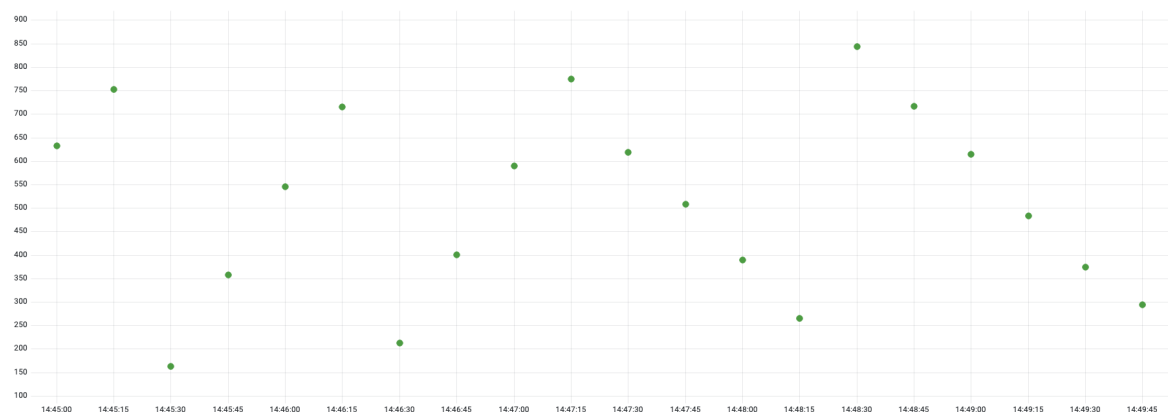
2 Gauge

Gauge 是个可增可减的仪表盘，Prometheus 抓取的是 Gauge 当前时刻设置的值。

常见的如内存使用量、活跃线程数等都是 Gauge 类型的监控指标。

```
Gauge gauge = Gauge.build().name("active_thread_num").help("活跃线程数").register();
gauge.set(20);
```

以下是 Gauge 类型的样本数据特点，我们一般不做二次计算，直接展示 Prometheus 抓取的原始值。



3 Histogram

Histogram 通常用于数据分布统计，Histogram 需要定义桶区间分布，根据用户上报的数据，来决定具体落到哪个桶内。

```
Histogram histogram = Histogram.build().name("http_request_cost").help("Http请求耗时").buckets(10, 20, 30, 40).register();
histogram.observe(20);
```

以下是部分源码，各个桶内存储上报的总次数，Prometheus 抓取的是各个桶的当前状态。

```
public void observe(double value) {
    for (int i = 0; i < bucket.length; ++i) {
        //遍历所有桶，如果数据小于桶上限，对应桶+1
        if (value <= bucket[i].le) {
            bucket[i].add(1);
            break;
        }
    }
    //增加sum总值
    sum.add(value);
}
```

基于各个时刻的桶内数据，我们可以计算出任意一段时间的数据分布情况。

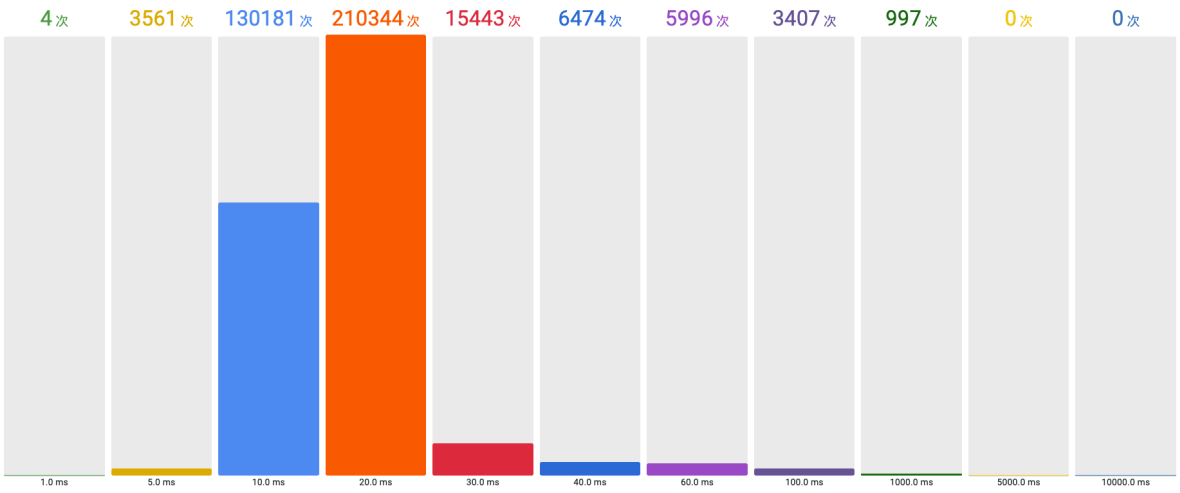
如下为例，我们可以利用桶内数据计算出 10:01~10:02 的数据分布情况。

	10ms	20ms	30ms	40ms
2021-10-01 10:01		5	10	20
2021-10-01 10:02	1	10	10	20

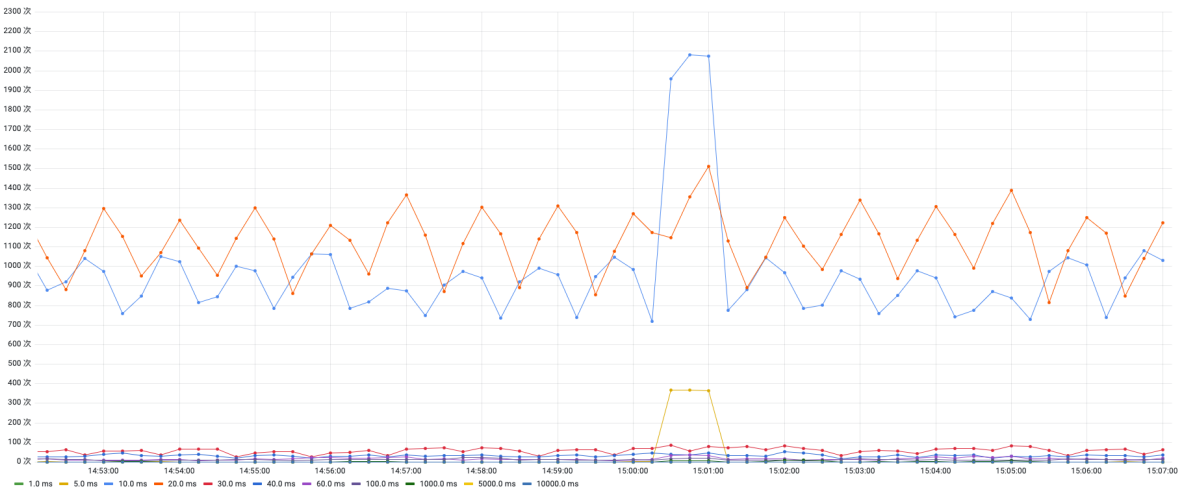
increase[1m]	10ms	20ms	30ms	40ms
--------------	------	------	------	------

这个计算结果可视化后可能如下图所见，实际意义可能是一天的 RPC 调用耗时分布。

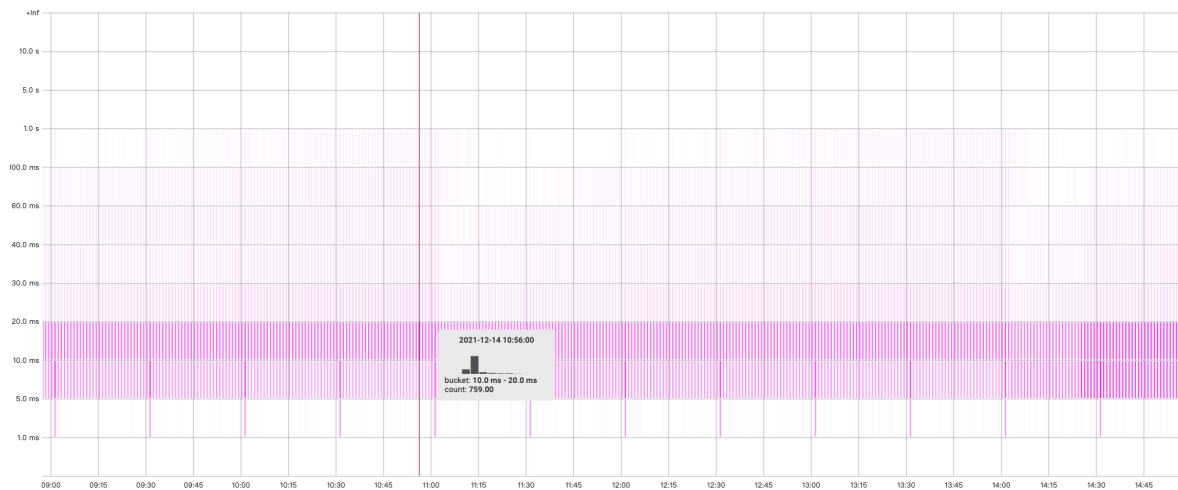
基于这个计算结果，还可以计算出 TP 值，即 n%的值都不高于 x。



如果我们每个时间点都计算一次分布，我们就得到了这样的面板：[分布折线图](#)。



利用这个数据，我们还可以换一种表现形式：[热力图](#)。热力图每个时刻都是一个直方图，热力图通过颜色深浅区分出数据分布的占比。



Histogram 除了 buckets，还记录了 sum（上报数据的总值）、count（上报数据的总次数）。我们通过一段时间内 sum 的增量/count 的增量，即可算出上报数据的平均值。

4 多维标签

Prometheus 的指标可以定义多个标签，比如对于以下指标：

```
Counter counter =
Counter.build().name("http_request").labelNames("method","uri").help("Http请求数").register();

counter.labels("POST", "/addGoods").incr();
counter.labels("POST", "/updateGoods").incr(2);
counter.labels("GET", "/getGoods").incr();
```

数据上报后，类比 MySQL 表，表名为 http_request，某一时刻的表数据如下：

method	uri	value
POST	/addGoods	1
POST	/updateGoods	2
GET	/getGoods	1

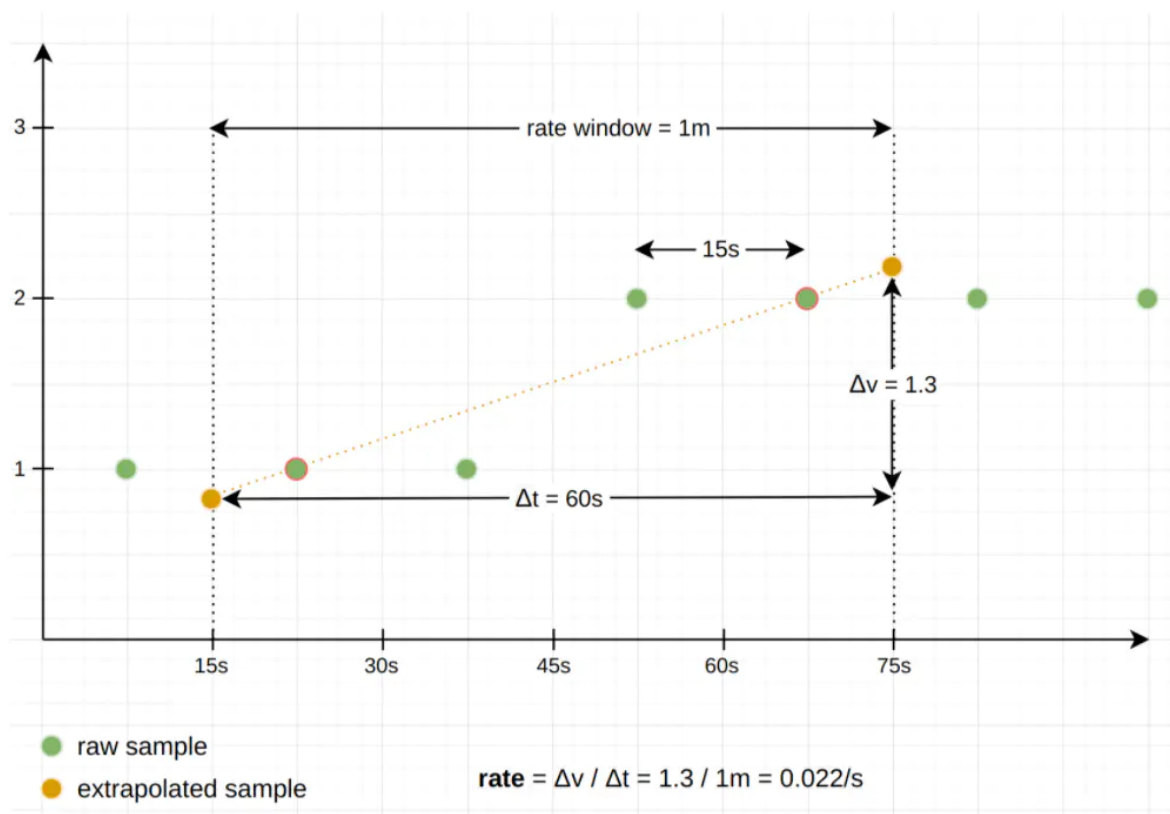
有了这些结构化的数据，就可以在任意维度进行查询和聚合。

比如，查询 uri=/addGoods 的数据；再比如，按照 method 维度做 SUM 聚合，查看总数。

5 误差

Prometheus 在设计上就放弃了一部分数据准确性，得到的是更高的可靠性，架构简单、数据简单、运维简单、节约机器成本与人力成本。通常对于监控系统，数据拥有少量的误差是可以接受的。

如计算增量时，给定的时间窗口中的第一个和最后一个样本，永远不会与真实的采样点 100%重合，Prometheus 会做数据线性外推，估算出对应时间的样本值。



什么是 Grafana

虽然 Prometheus 提供的 Web UI 也可以很好的查看不同指标的视图，但是这个功能非常简单，只适合用来调试。

要实现一个强大的监控系统，还需要一个能定制展示不同指标的面板，能支持不同类型的展现方式（曲线图、饼状图、热点图、TopN 等），这就是仪表盘（Dashboard）功能。

因此 Prometheus 开发了一套仪表盘系统 [PromDash](#)，不过很快这套系统就被废弃了，官方开始推荐使用 Grafana 来对 Prometheus 的指标数据进行可视化，这不仅是因为 Grafana 的功能非常强大，而且它和 Prometheus 可以完美的无缝融合。

[Grafana](#) 是一个用于可视化大型测量数据的开源系统，它的功能非常强大，界面也非常漂亮，

使用它可以创建自定义的控制面板，你可以在面板中配置要显示的数据和显示方式，

它 [支持很多不同的数据源](#)，比如：Graphite、InfluxDB、OpenTSDB、Elasticsearch、Prometheus 等，而且它也 [支持众多的插件](#)。

下面我们就体验下使用 Grafana 来展示 Prometheus 的指标数据。

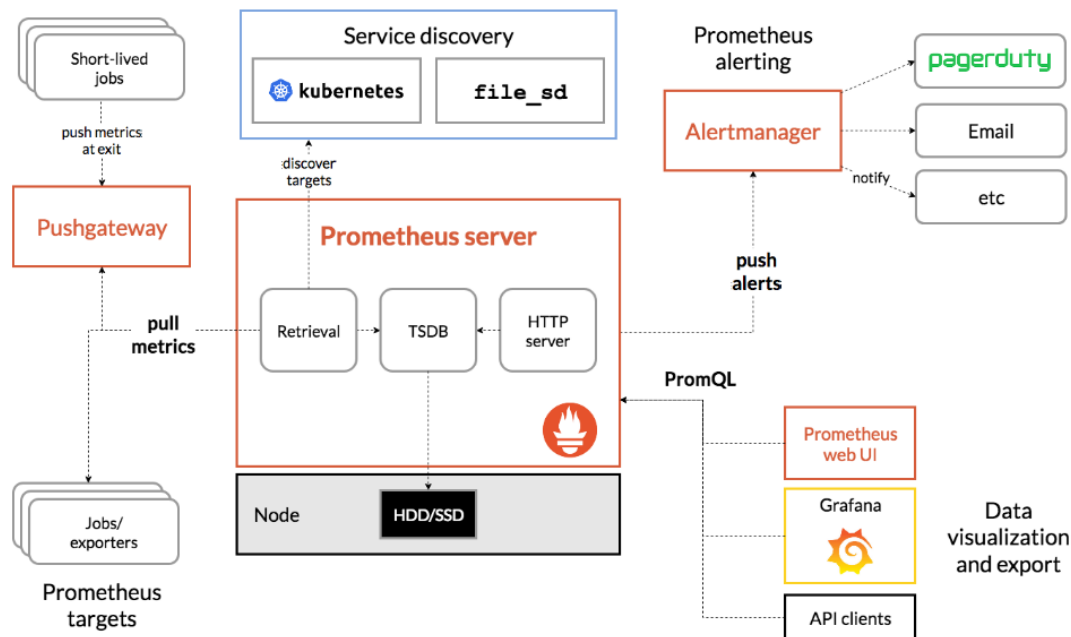
首先我们来安装 Grafana，我们使用最简单的 [Docker 安装方式](#)：

```
$ docker run -d -p 3000:3000 grafana/grafana
```

运行上面的 docker 命令，Grafana 就安装好了！你也可以采用其他的安装方式，参考 [官方的安装文档](#)。安装完成之后，我们访问 <http://localhost:3000/> 进入 Grafana 的登陆页面，输入默认的用户名和密码（admin/admin）即可。

Prometheus的体系结构

下图说明了Prometheus的体系结构及其某些生态系统组件：

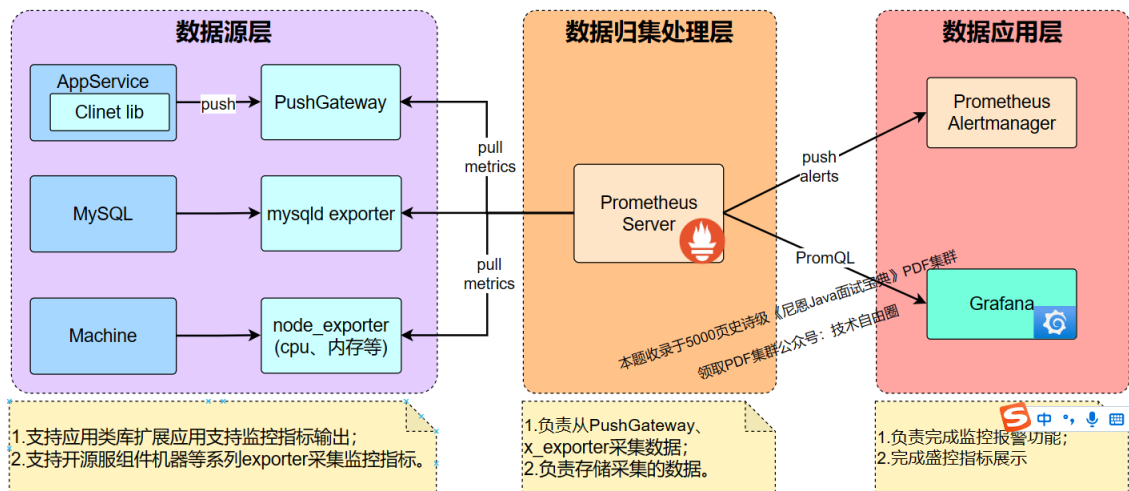


Prometheus直接或通过中间推送网关从已检测的作业中删除指标，以处理短暂的作业。

它在本地存储所有报废的样本，并对这些数据运行规则，以汇总和记录现有数据中的新时间序列，或生成警报。Grafana或其他API使用者可以用来可视化收集的数据。

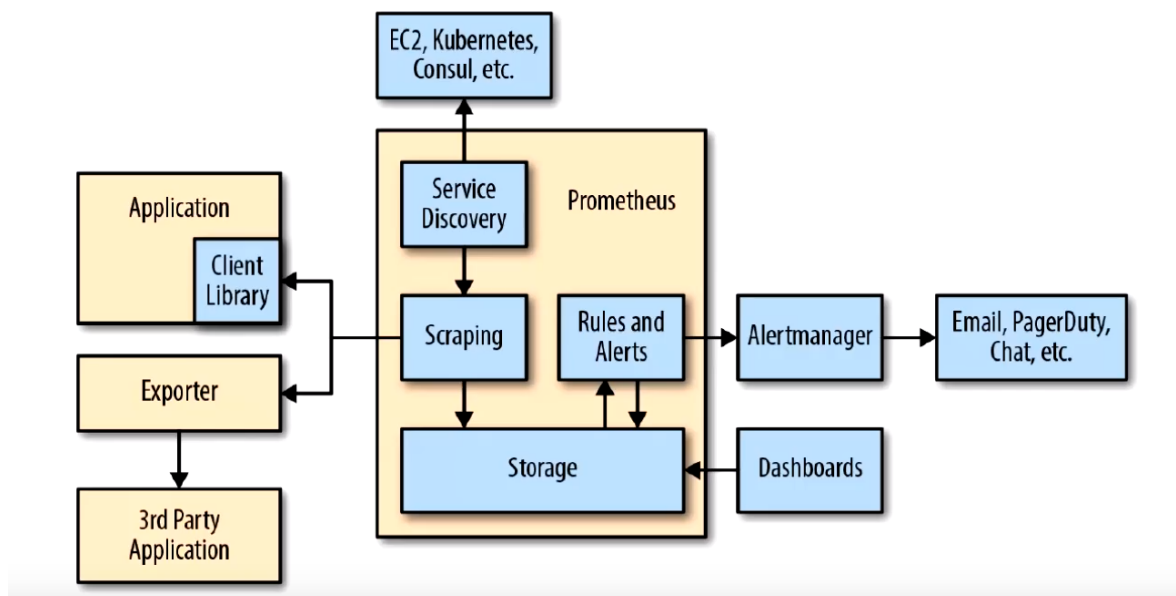
Prometheus+Grafana分层架构

从分层的角度来说，Prometheus+Grafana分层架构图如下：



Prometheus体系涉及的组件

Prometheus负责时序指标数据的采集及存储，但数据的分析、聚合及直观展示以及告警等功能并非由Prometheus Server所负责。

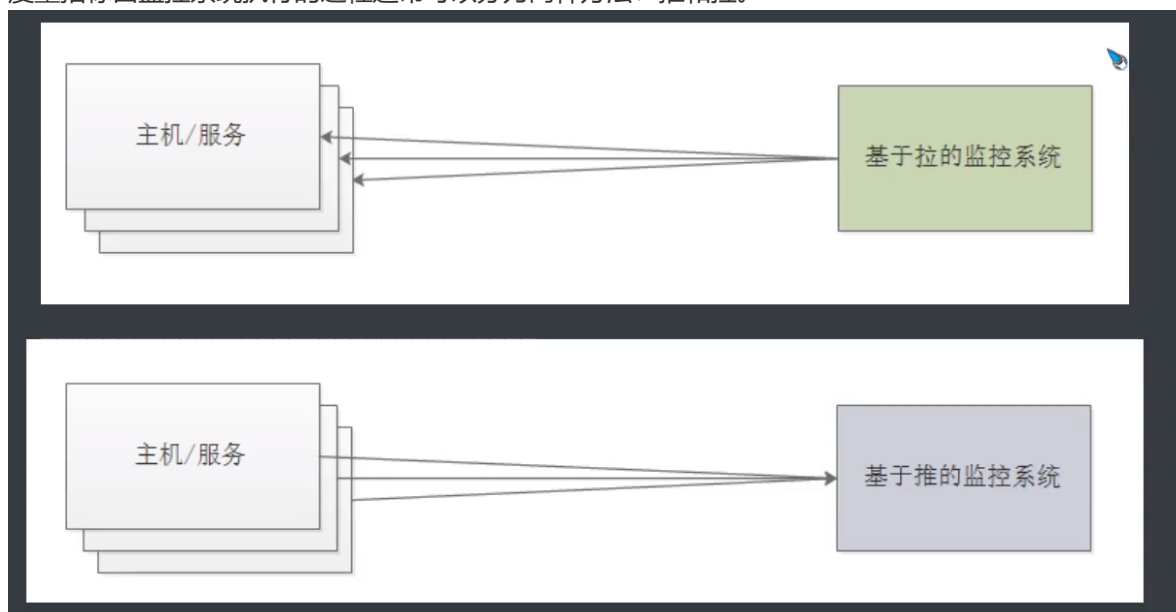


Prometheus生态系统包含多个组件，其中许多是可选的：

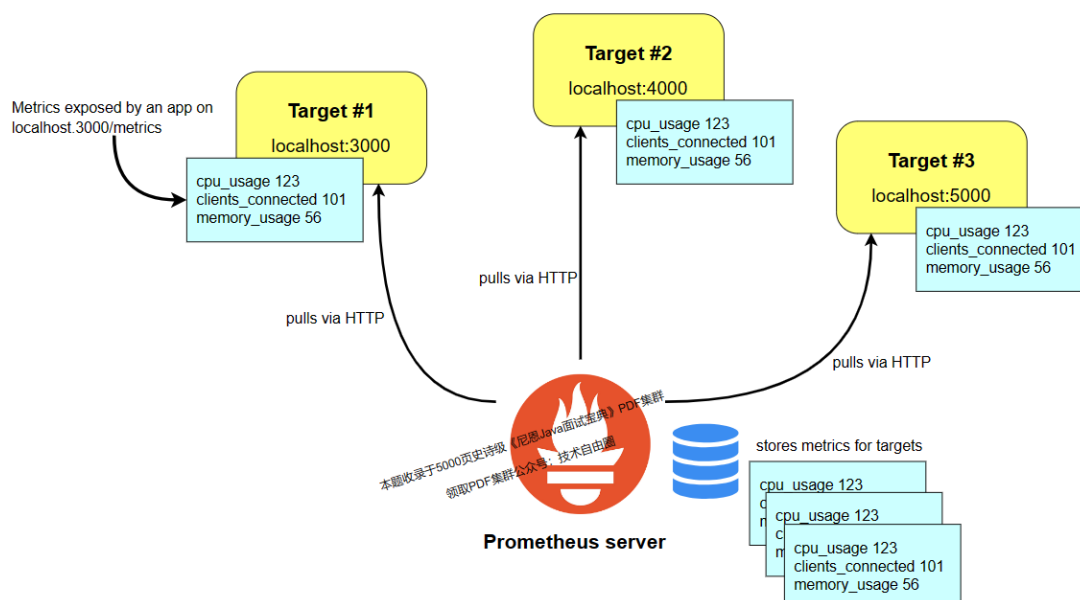
- Prometheus server - 收集和存储时间序列数据
- Client Library: 客户端库，为需要监控的服务生成相应的
- metrics 并暴露给 - Prometheus server。当 Prometheus server 来 pull 时，直接返回实时状态的 metrics。
- pushgateway - 对于短暂运行的任务，负责接收和缓存时间序列数据，同时也是一个数据源
- exporter - 各种专用exporter，面向硬件、存储、数据库、HTTP服务等
- alertmanager - 处理报警
- webUI等，其他各种支持的工具，本身的界面值适合用来语句查询，数据可视化，需要第三方组件，比如Grafana。

如何收集度量值

度量指标由监控系统执行的过程通常可以分为两种方法：推和拉。

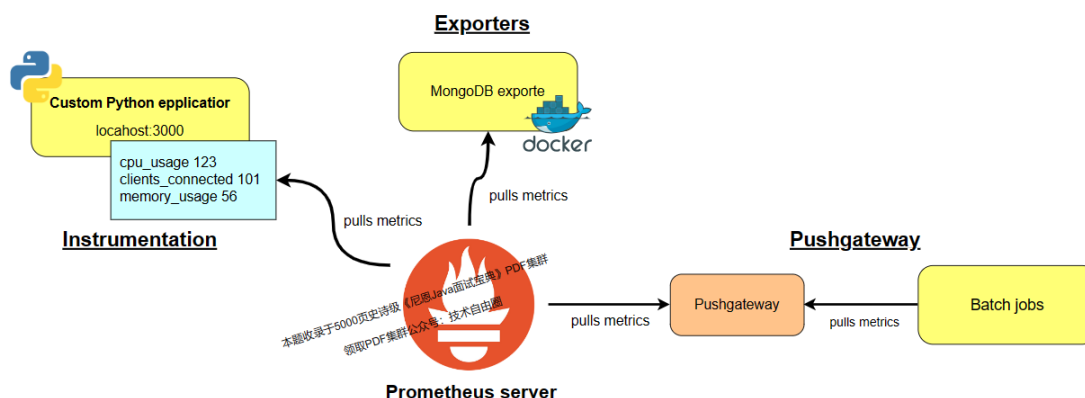


Prometheus基于HTTP call，从配置文件中指定的网络端点(endpoint) 上周期性获取指标数据。



Prometheus支持通过三种类型的途径从目标上“抓取(Serape)”指标数据：

- Exporters：被监控的目标不支持pro的数据格式，通过exporters抽取指标数据，进行格式化处理后成pro兼容的数据格式，再响应给pro server。
- Instrumentation：应用系统内建了pro兼容的指标数据格式，pro server可以直接采集。
- Push gateway：pro采用 pull 模式，可能由于不在一个子网或者防火墙原因，导致 Prometheus 无法直接拉取各个 target 数据。在监控业务数据的时候，需要将不同数据汇总，由 Prometheus 统一收集。暂存在pushgateway，等待Prometheus server拉取。



指标类型

Prometheus使用4种方法来描述监视的指标。

计数器

Counter:

计数器，用于保存单调递增型的数据，例如站点访问次数等；不能为负值，也不支持减少，但可以重置回0；

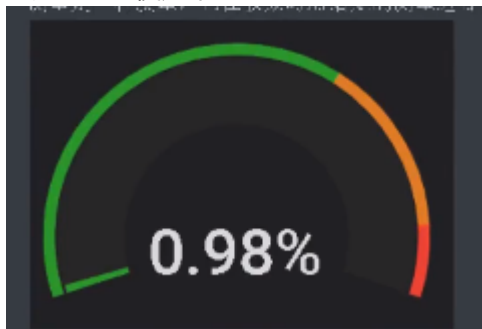
比如网络总访问数。

仪表盘

Gauge:仪表盘，用于存储有着起伏特征的指标数据，例如内存空闲大小等。

Gauge是Counter的超集;但存在指标数据丢失的可能性时，Counter能让用户确切了解指标随时间的变化状态，而Gauge则可能随时间流逝而精准度越来越低。

比如CPU的使用率：



直方图

在大多数情况下人们都倾向于使用某些量化指标的平均值，例如 CPU的平均使用率、页面的平均响应时间。这种方式的问题很明显，以系统API调用的平均响应时间为例:如果大多数API请求都维持在100ms的响应时间范围内，而个别请求的响应时间需要5s，那么就会导致某些WEB页面的响应时间落到中位数的情况，而这种现象被称为长尾问题。

Histogram常用于观察，一个Histogram包含下列值的合并:

- Buckets:桶是观察的计数器。它应该有个最大值的边界和最小值的边界。它的格式为`basename>_bucket`
- 观察结果的和，这是所有观察的和。针对它的格式是《`basename>_sum`》
- 观察结果统计，这是在本次观察的和。它的格式为《`basename>_count`》

Summary

Summary与Histogram类型类似，用于表示段时间内的数据采集结果(通常是请求持续时间或响应大小)，但它直接存储了分位数通过客户端计算，然后展示出来，而不是通过区间来计算。

指标摘要及聚合

指标摘要

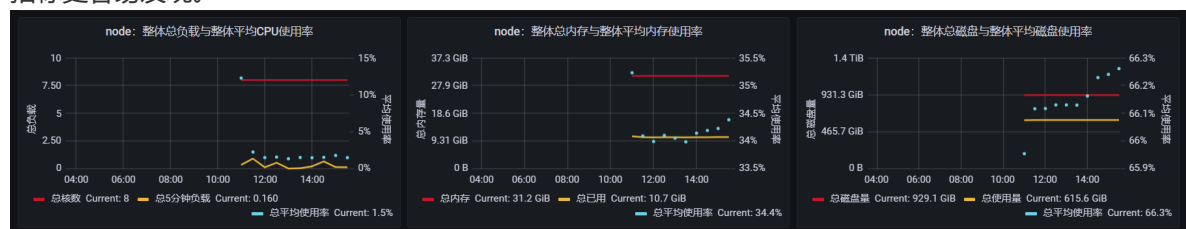
通常来说，单个指标对我们价值很小，往往需要联合并可视化多个指标，这其中需要一些数学变换，例如，我们可能会统计函数应用于指标或指标组，一些可能应用常见函数包括:

- 计数:计算特定时间间隔内的观察点数，
- 求和:将特定时间间隔内所有观察点的值累计相加。
- 平均值，提供特定时间间隔内所有值的平均值，
- 中间数:数值的几何中点，正好50%的数值位于它前面，而另外50%则位于它后面，
- 百分位数:度量占总数特定百分比的观察点的值。
- 标准差:显示指标分布中与平均值的标准差，这可以测量出数据集的差异程度。标准差为0表示数据都等于平均值较高的标准差意味着数据分布的范围很广
- 变化率:显示时间序列中数据之间的变化程度。

指标聚合

除了上述的指标摘要外，你可能经常希望能看到来自单个源的指标的聚合图，例如所有应用程序服务器的磁盘空间使用情况。指标聚合最典型的样式就是在一张图上显示多个指标，这有助于你识别环境的发展趋势。

例如，负载均衡器中的间歇性故障可能导致多个服务器的Web流量下降，这通常比通过查看每个单独的指标更容易发现。



监控实操1：一键安装 prometheus

bridge网络管理

不指定网络驱动时默认创建的bridge网络

```
docker network create monitor-network
```

查看网络内部信息

```
docker network inspect monitor-network
```

列所有列表的网络

```
docker network ls
```

移除指定的网络

```
docker network rm default_network
```

创建库

注意，数据库提前建好grafana 库：

```
/usr/bin/mysql -uroot -p123456 --connect-expired-password <<EOF

create database grafana default character set utf8mb4 collate
utf8mb4_unicode_ci;

grant all privileges on grafana.* to root@'%' identified by '123456' WITH GRANT
OPTION;

flush privileges;

EOF
```

由于尼恩新版的 mysql，使用容器启动，首先进行入容器

```
#进入容器
docker exec -it mysql /bin/bash

# 再建立 库
/usr/bin/mysql -uroot -p123456 --connect-expired-password <<EOF

create database grafana default character set utf8mb4 collate
utf8mb4_unicode_ci;

grant all privileges on grafana.* to root@'%' identified by '123456' WITH GRANT
OPTION;

flush privileges;

EOF
```

只要建立库就行，表自动创建，这个不用管

docker编排文件

```
version: "3.5"
services:
  prometheus:
    image: prom/prometheus:v2.30.0
    container_name: prometheus
    volumes:
      - /tmp:/tmp
      - ./prometheus.yml:/etc/prometheus/prometheus.yml
    command: "--config.file=/etc/prometheus/prometheus.yml --
storage.tsdb.path=/prometheus --storage.tsdb.wal-compression --
storage.tsdb.retention.time=7d"
    hostname: prometheus
    ports:
      - "9090:9090"
    depends_on:
      - exporter
    networks:
      base-env-network:
        aliases:
          - prometheus

  exporter:
    image: prom/node-exporter:v1.2.2
    container_name: exporter
    hostname: exporter
    ports:
      - "9100:9100"
    networks:
```

```

    base-env-network:
      aliases:
        - exporter

# 用于UI展示
# https://grafana.com/docs/grafana/latest/installation/docker
grafana:
  image: grafana/grafana:8.1.5
  container_name: grafana
  hostname: grafana
  restart: unless-stopped
  ports:
    - "3000:3000"
#  volumes:
#    - "./grafana/grafana-storage:/var/lib/grafana"
#    - "./grafana/public:/usr/share/grafana/public" # 这里面可处理汉化包 可参考
# https://github.com/wangHL0927/grafana-chinese
#    - "./grafana/conf:/usr/share/grafana/conf"
#    - "./grafana/log:/var/log/grafana"
#    - "/etc/localtime:/etc/localtime"
  environment:
    GF_EXPLORE_ENABLED: "true"
    GF_SECURITY_ADMIN_PASSWORD: "admin"
    GF_INSTALL_PLUGINS: "grafana-clock-panel,grafana-simple-json-
datasource,alexanderzobnin-zabbix-app"
    # 持久化到mysql数据库
    GF_DATABASE_URL: "mysql://root:123456@192.168.56.121:3306/grafana" # TODO
修改
  depends_on:
    - prometheus
  networks:
    base-env-network:
      aliases:
        - grafana

# mysql数据库 => 用于grafana持久化数据
# mysql:
#   image: nien/mysql:5.7
#   container_name: prometheus_mysql
#   restart: unless-stopped
#   volumes:
#     - "./mysql5.7/my.cnf:/etc/mysql/my.cnf"
#     - "./mysql5.7/data:/var/lib/mysql"
#     - "./mysql5.7/log/mysql/error.log:/var/log/mysql/error.log"
#   environment:
#     TZ: Asia/Shanghai
#     LANG: en_US.UTF-8
#     MYSQL_ROOT_PASSWORD: root          # 设置root用户密码
#     MYSQL_DATABASE: grafana           # 初始化数据库grafana
#   ports:
#     - "3306:3306"

# docker network create base-env-network
networks:
  base-env-network:
    external:
      name: "base-env-network"

```

一键安装 prometheus的shell 脚本

```
mkdir -p cd /home/docker-compose/prometheus

cd /home/docker-compose/prometheus

docker-compose down

rm -rf /home/docker-compose/prometheus

cd /
cp -rf /vagrant/chapter33/prometheus /home/docker-compose/

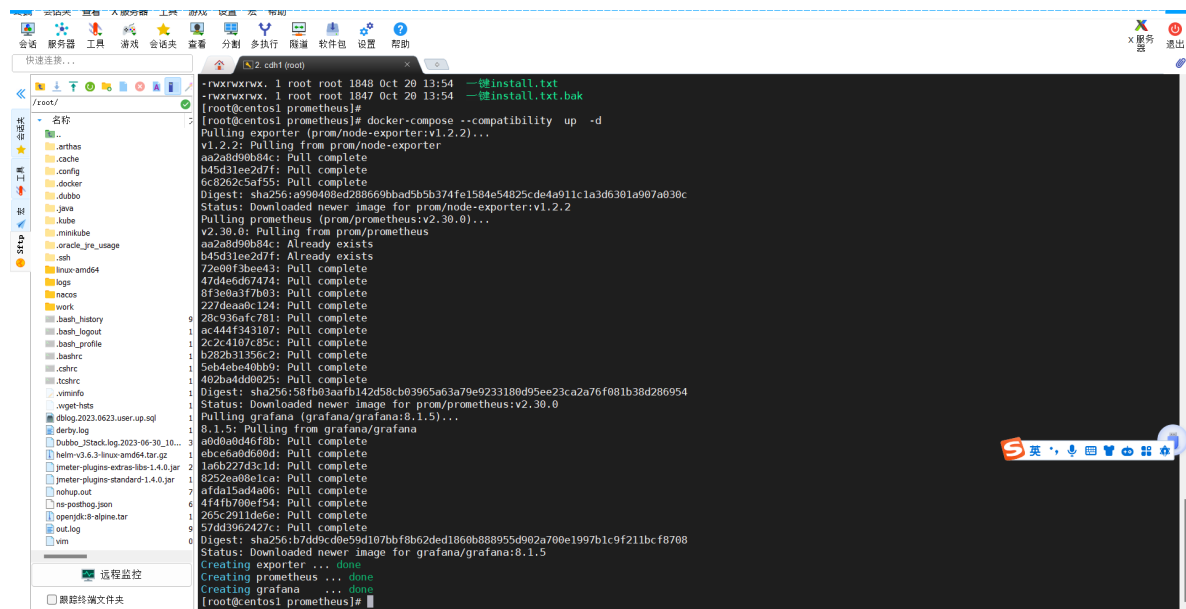
chmod 777 -R /home/docker-compose/prometheus

cd /home/docker-compose/prometheus

ls -l

docker-compose --compatibility up -d
```

执行过程如下，很溜的



进入 prometheus

<http://promethus>所在的服务器地址:9090

<http://cdh1:9090/>

←↻⚠ 不安全 | cdh1:9090/graph?g0.expr=&g0.tab=1&g0.stacked=0&g0.show_exemplars=0&g0.range_input=1h

PrometheusAlertsGraphStatus▼HelpClassic UI

☐ Use local time☐ Enable query history☒ Enable autocomplete☒ Use exp

Q

Expression (press Shift+Enter for newlines)

Table

Graph

<

Evaluation time

>

No data queried yet

Add Panel

CSDN @架构师-尼恩

点击service

PrometheusAlertsGraphStatus▼HelpClassic UI

Service Discovery

- exporter (1 / 1 active targets)
- prometheus (1 / 1 active targets)

exportershow less

Discovered Labels

__address__="exporter:9100"

__metrics_path__="/metrics"

__scheme__="http"

__scrape_interval__="10s"

__scrape_timeout__="10s"

instance="exporter"

job="exporter"

Target Labels

instance="exporter"

job="exporter"

prometheusshow less

Discovered Labels

__address__="prometheus:9090"

__metrics_path__="/metrics"

__scheme__="http"

__scrape_interval__="10s"

__scrape_timeout__="10s"

instance="prometheus"

job="prometheus"

Target Labels

instance="prometheus"

job="prometheus"

点击targets

page:20/94 of 疯狂创客圈Java架构班配套讲义：跟着40岁老架构师-尼恩，狠卷3高架构，穿透3高架构，走向技术自由！

← 不安全 | cdh1-9090/classic/targets

Prometheus Alerts Graph Status ▾ Help New UI

Targets

All Unhealthy Collapse All

exporter (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://exporter:9100/metrics	UP	instance="localhost" job="exporter"	4.279s ago	6.417ms	

prometheus (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://prometheus:9090/metrics	UP	instance="prometheus" job="prometheus"	1.826s ago	4.227ms	

spring-boot (0/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://192.168.0.88:20040/application/prometheus	DOWN	instance="192.168.0.88:20040" job="spring-boot"	9.69s ago	5.001s	Get "http://192.168.0.88:20040/application/prometheus": context deadline exceeded

CSDN @架构师-尼恩

进入 grafana

grafana 默认密码 admin admin

<http://cdh1:3000/explore?left=%5B%22now-1h%22,%22now%22,%22--%20Grafana%20--%22,%27B%7D%5D&orgId=1>

Explore Grafana

Split

Last 1 hour

Clear all

Run query

Query type Random Walk

+ Add query Query history Inspector

CSDN @架构师-尼恩

用久了之后，如果 grafana 默认密码 admin admin 不正确，可以手动在mysql修改密码

```
#重置admin用户的密码为默认admin
update user set password =
'59acf18b94d7eb0694c61e60ce44c110c7a683ac6a8f09580d626f90f4a242000746579358d77dd
9e570e83fa24faa88a8a6', salt = 'F3FAXvm33R' where login = 'admin';
```

啥也没有

监控实操2：Prometheus+Grafana监控SpringBoot项目JVM信息

- [1. SpringBoot项目配置JVM采集](#)
- [2. Prometheus配置](#)
- [3. 配置grafana](#)
- [4. 扩展-通过JMX Exporter监控JVM信息](#)

SpringBoot项目配置JVM采集

(1) maven依赖

```
<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

(2) application.properties

```
spring.application.name=springboot_jvm
server.port=6001
management.endpoints.web.exposure.include=*
management.metrics.tags.application=${spring.application.name}
```

(3) SpringBoot添加PrometheusMeterRegistry 注册器

```
@Configuration
public class CustomPrometheusConfig
{
    @Bean
    @ConditionalOnMissingBean
    public PrometheusMeterRegistry prometheusMeterRegistry(PrometheusConfig
prometheusConfig,
                                                            CollectorRegistry
collectorRegistry, Clock clock) {
        return new PrometheusMeterRegistry(prometheusConfig, collectorRegistry,
clock);
    }

    @Bean
    @ConditionalOnMissingBean
    public CollectorRegistry collectorRegistry() {
        return new CollectorRegistry(true);
    }
}
```

```
}  
}
```

(4) 启动springboot项目，访问 可以看到一些统计指标

<http://localhost:18081/consumer/actuator/prometheus>

<http://192.168.56.1:28088/seata-seckill/actuator/prometheus>

```
← ↻ ⚠ 不安全 | 192.168.56.1:28088/seata-seckill/actuator/prometheus  
  
# HELP jvm_gc_pause_seconds Time spent in GC pause  
# TYPE jvm_gc_pause_seconds summary  
jvm_gc_pause_seconds_count{action="end of minor GC",application="seata-seckill",cause="Metadatas GC Threshold",} 1.0  
jvm_gc_pause_seconds_sum{action="end of minor GC",application="seata-seckill",cause="Metadatas GC Threshold",} 0.011  
jvm_gc_pause_seconds_count{action="end of major GC",application="seata-seckill",cause="Metadatas GC Threshold",} 1.0  
jvm_gc_pause_seconds_sum{action="end of major GC",application="seata-seckill",cause="Metadatas GC Threshold",} 0.089  
jvm_gc_pause_seconds_count{action="end of minor GC",application="seata-seckill",cause="Allocation Failure",} 2.0  
jvm_gc_pause_seconds_sum{action="end of minor GC",application="seata-seckill",cause="Allocation Failure",} 0.019  
# HELP jvm_gc_pause_seconds_max Time spent in GC pause  
# TYPE jvm_gc_pause_seconds_max gauge  
jvm_gc_pause_seconds_max{action="end of minor GC",application="seata-seckill",cause="Metadatas GC Threshold",} 0.011  
jvm_gc_pause_seconds_max{action="end of major GC",application="seata-seckill",cause="Metadatas GC Threshold",} 0.089  
jvm_gc_pause_seconds_max{action="end of minor GC",application="seata-seckill",cause="Allocation Failure",} 0.011  
# HELP tomcat_global_received_bytes_total  
# TYPE tomcat_global_received_bytes_total counter  
tomcat_global_received_bytes_total{application="seata-seckill",name="http-nio-28088",} 0.0  
# HELP tomcat_global_error_total  
# TYPE tomcat_global_error_total counter  
tomcat_global_error_total{application="seata-seckill",name="http-nio-28088",} 0.0  
# HELP tomcat_global_request_seconds  
# TYPE tomcat_global_request_seconds summary  
tomcat_global_request_seconds_count{application="seata-seckill",name="http-nio-28088",} 1.0  
tomcat_global_request_seconds_sum{application="seata-seckill",name="http-nio-28088",} 0.152  
# HELP tomcat_global_request_max_seconds  
# TYPE tomcat_global_request_max_seconds gauge  
tomcat_global_request_max_seconds{application="seata-seckill",name="http-nio-28088",} 0.152  
# HELP jvm_memory_committed_bytes The amount of memory in bytes that is committed for the Java virtual machine to use  
# TYPE jvm_memory_committed_bytes gauge  
jvm_memory_committed_bytes{application="seata-seckill",area="nonheap",id="Code Cache",} 3.0343168E7  
jvm_memory_committed_bytes{application="seata-seckill",area="nonheap",id="Metaspace",} 9.1316224E7  
jvm_memory_committed_bytes{application="seata-seckill",area="heap",id="PS Old Gen",} 2.57949696E8  
jvm_memory_committed_bytes{application="seata-seckill",area="heap",id="PS Eden Space",} 2.90455552E8  
jvm_memory_committed_bytes{application="seata-seckill",area="heap",id="PS Survivor Space",} 1.835008E7  
jvm_memory_committed_bytes{application="seata-seckill",area="nonheap",id="Compressed Class Space",} 1.1927552E7  
# HELP jvm_threads_daemon_threads The current number of live daemon threads  
# TYPE jvm_threads_daemon_threads gauge  
jvm_threads_daemon_threads{application="seata-seckill",} 55.0  
# HELP jvm_threads_states_threads The current number of threads having NEW state  
# TYPE jvm_threads_states_threads gauge  
jvm_threads_states_threads{application="seata-seckill",state="runnable",} 11.0  
jvm_threads_states_threads{application="seata-seckill",state="new",} 0.0  
jvm_threads_states_threads{application="seata-seckill",state="blocked",} 0.0  
jvm_threads_states_threads{application="seata-seckill",state="waiting",} 31.0  
jvm_threads_states_threads{application="seata-seckill",state="timed-waiting",} 19.0  
jvm_threads_states_threads{application="seata-seckill",state="terminated",} 0.0  
# HELP tomcat_sessions_active_current_sessions  
# TYPE tomcat_sessions_active_current_sessions gauge  
tomcat_sessions_active_current_sessions{application="seata-seckill",} 0.0
```

CSDN @架构师-尼恩

Prometheus配置

参考末尾的 基于nacos的服务发现

访问地址：

<http://cdh1:9090/targets#pool-spring-boot>

← ↻ ⚠ 不安全 | cdh1:9090/targets#pool-spring-boot

Prometheus Alerts Graph Status ▾ Help Classic UI

exporter (1/1 up) [show less](#)

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://exporter:9100/metrics	UP	instance="exporter" job="exporter"	25.481s ago	5.894ms	

nacos-public-discovery (2/2 up) [show less](#)

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://192.168.56.1:28088/seata-seckill/actuator/prometheus	UP	instance="192.168.56.1:28088" job="seata-seckill"	26.266s ago	45.018ms	
http://192.168.56.1:18081/consumer/actuator/prometheus	UP	instance="192.168.56.1:18081" job="service-consumer-demo"	34.428s ago	24.821ms	

prometheus (1/1 up) [show less](#)

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://prometheus:9090/metrics	UP	instance="prometheus" job="prometheus"	33.447s ago	3.179ms	

spring-boot (0/1 up) [show less](#)

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://192.168.0.88:20040/application/prometheus	DOWN	instance="192.168.0.88:20040" job="spring-boot"	31.307s ago	5.0s	Get "http://192.168.0.88:20040/application/prometheus": dial tcp 192.168.0.88:20040: connect: connection refused

监控实操3：配置grafana监控Linux系统

使用 `node_exporter` 可以暴露Linux系统的指标信息，
然后Prometheus就可以通过定时扫描的方式获取并存储指标信息了。

使用 Exporter 收集指标

目前为止，我们看到的都还只是一些没有实际用途的指标，如果我们要在我们的生产环境真正使用Prometheus，往往需要关注各种各样的指标，譬如服务器的CPU负载、内存占用量、IO开销、入网和出网流量等等。

正如上面所说，Prometheus 是使用 Pull 的方式来获取指标数据的，要让 Prometheus 从目标处获得数据，首先必须在目标上安装指标收集的程序，并暴露出 HTTP 接口供 Prometheus 查询，这个指标收集程序被称为 **Exporter**，不同的指标需要不同的 Exporter 来收集，目前已经有大量的 Exporter 可供使用，几乎囊括了我们常用的各种系统和软件，官网列出了一份 [常用 Exporter 的清单](#)，各个 Exporter 都遵循一份端口约定，避免端口冲突，即从 9100 开始依次递增，这里是 [完整的 Exporter 端口列表](#)。

另外值得注意的是，有些软件和系统无需安装 Exporter，这是因为他们本身就提供了暴露 Prometheus 格式的指标数据的功能，比如 Kubernetes、Grafana、Etc、Ceph 等。

这一节就让我们来收集一些有用的数据。

linux直接安装node_exporter

首先我们来收集服务器的指标，这需要安装 [node_exporter](#)，这个 exporter 用于收集 *NIX 内核的系统，

如果你的服务器是 Windows，可以使用 [WMI exporter](#)。

node_exporter

Exporter for machine metrics [prometheus/node_exporter](#)

1.1.2 / 2021-03-05 Release notes				
File name	OS	Arch	Size	SHA256 Checksum
node_exporter-1.1.2.darwin-amd64.tar.gz	darwin	amd64	4.72 MiB	3919266f1dbad5f7e5ce7b4207057fc253a8322f570607ec0d3e73f4a53338e3
node_exporter-1.1.2.linux-amd64.tar.gz	linux	amd64	8.82 MiB	8c1f6a317457a668e0aee68ad710f6b4098db2cad10204649b51e3c043aa3e70d

这次我们直接把安装到Linux服务器上，将下载的安装包解压到指定目录，并修改文件夹名称：

```
cd /mydata
tar -zxvf node_exporter-1.1.2.linux-amd64.tar.gz
mv node_exporter-1.1.2.linux-amd64 node_exporter
```

进入解压目录，使用如下命令运行，服务将运行在 9100 端口上；

```
cd node_exporter
./node_exporter >log.file 2>&1 &
```

- 使用 `curl` 命令访问获取指标信息接口，获取到信息表示运行成功；

```
curl http://localhost:9100/metrics
# HELP promhttp_metric_handler_requests_in_flight Current number of scrapes
being served.# TYPE promhttp_metric_handler_requests_in_flight
gaugepromhttp_metric_handler_requests_in_flight 1# HELP
promhttp_metric_handler_requests_total Total number of scrapes by HTTP status
code.# TYPE promhttp_metric_handler_requests_total
counterpromhttp_metric_handler_requests_total{code="200"}
2175promhttp_metric_handler_requests_total{code="500"}
0promhttp_metric_handler_requests_total{code="503"} 0
```

使用Docker容器安装node_exporter

如果使用Docker容器安装，监控的会是Docker容器的指标信息

```
exporter:
  image: prom/node-exporter:v1.2.2
  container_name: exporter
  hostname: exporter
  ports:
    - "9100:9100"
  networks:
    base-env-network:
      aliases:
        - exporter
```

注意事项

一般情况下，node_exporter 都是直接运行在要收集指标的服务器上的，官方不推荐用 Docker 来运行 node_exporter。如果逼不得已一定要运行在 Docker 里，要特别注意，这是因为 Docker 的文件系统和网络都有自己的 namespace，收集的数据并不是宿主机真实的指标。可以使用一些变通的方法，比如运行 Docker 时加上下面这样的参数：

```
docker run -d \
  --net="host" \
  --pid="host" \
  -v "/:/host:ro,rslave" \
  quay.io/prometheus/node-exporter \
  --path.rootfs /host
```

关于 node_exporter 的更多信息，可以参考 [node_exporter 的文档](#) 和 Prometheus 的官方指南 [Monitoring Linux host metrics with the Node Exporter](#),

另外，Julius Volz 的这篇文章 [How To Install Prometheus using Docker on Ubuntu 14.04](#) 也是很好的入门材料。

创建一个任务定时扫描暴露的指标信息

接下来修改Prometheus的配置文件，创建一个任务定时扫描暴露的指标信息；

```
# 采集node exporter监控数据，即linux
- job_name: exporter
  static_configs:
    - targets: ['exporter:9100']
      labels:
        instance: exporter
```

<http://exporter:9100/metrics>

The screenshot shows the Prometheus web interface. At the top, there's a navigation bar with 'Prometheus', 'Alerts', 'Graph', 'Status', 'Help', and 'Classic UI'. Below this is the 'Targets' section. It has three tabs: 'All', 'Unhealthy', and 'Collapse All'. The 'All' tab is selected, showing a list of target groups. The first group is 'exporter (1/1 up)' with a 'show less' button. It contains one target with the endpoint 'http://exporter:9100/metrics', state 'UP', labels 'instance="exporter"' and 'job="exporter"', last scrape '6.513s ago', and scrape duration '6.906ms'. The second group is 'nacos-public-discovery (2/2 up)' with a 'show less' button. It contains two targets: one with endpoint 'http://192.168.56.1:28088/seata-seckill/actuator/prometheus', state 'UP', labels 'instance="192.168.56.1:28088"' and 'job="seata-seckill"', last scrape '7.300s ago', and scrape duration '26.599ms'; the other with endpoint 'http://192.168.56.1:18081/consumer/actuator/prometheus', state 'UP', labels 'instance="192.168.56.1:18081"' and 'job="service-consumer-demo"', last scrape '5.462s ago', and scrape duration '28.876ms'. The third group is 'prometheus (1/1 up)' with a 'show less' button. It contains one target with the endpoint 'http://prometheus:9090/metrics', state 'UP', labels 'instance="prometheus"' and 'job="prometheus"', last scrape '6.513s ago', and scrape duration '6.906ms'. At the bottom right, there's a 'Scrape' button and a user profile 'CSDN @架构师-尼恩'.

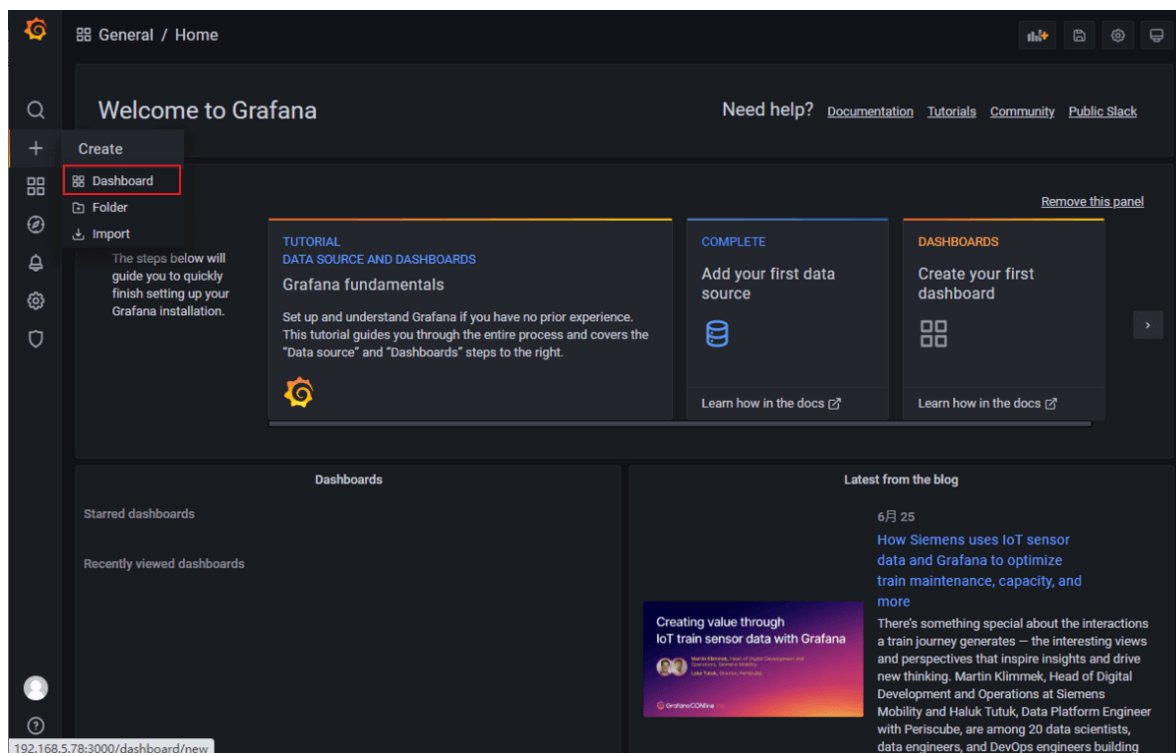
Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://exporter:9100/metrics	UP	instance="exporter" job="exporter"	6.513s ago	6.906ms	

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://192.168.56.1:28088/seata-seckill/actuator/prometheus	UP	instance="192.168.56.1:28088" job="seata-seckill"	7.300s ago	26.599ms	
http://192.168.56.1:18081/consumer/actuator/prometheus	UP	instance="192.168.56.1:18081" job="service-consumer-demo"	5.462s ago	28.876ms	

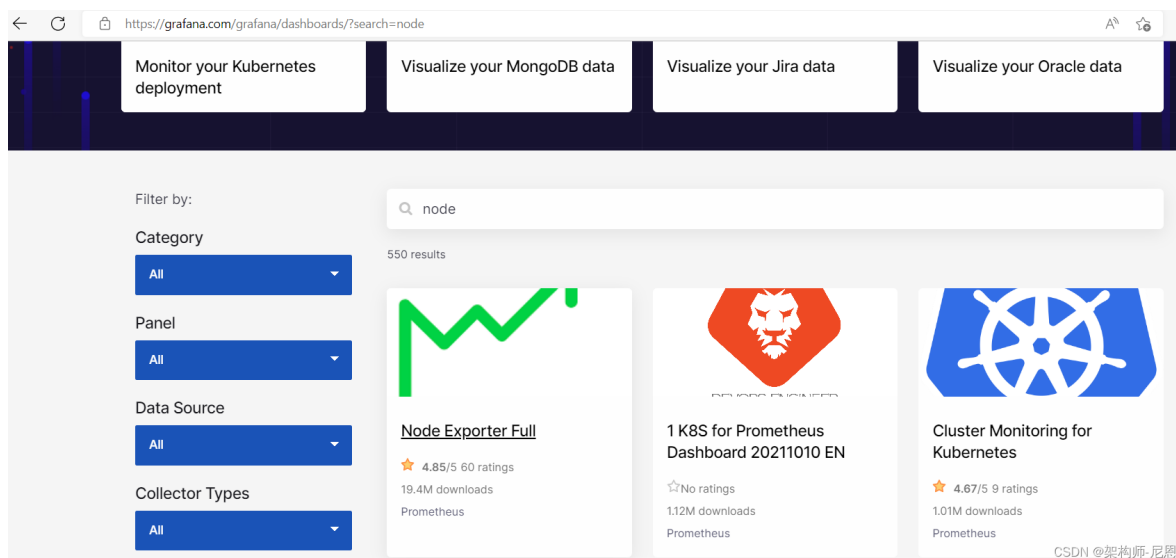
Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://prometheus:9090/metrics	UP	instance="prometheus" job="prometheus"	6.513s ago	6.906ms	

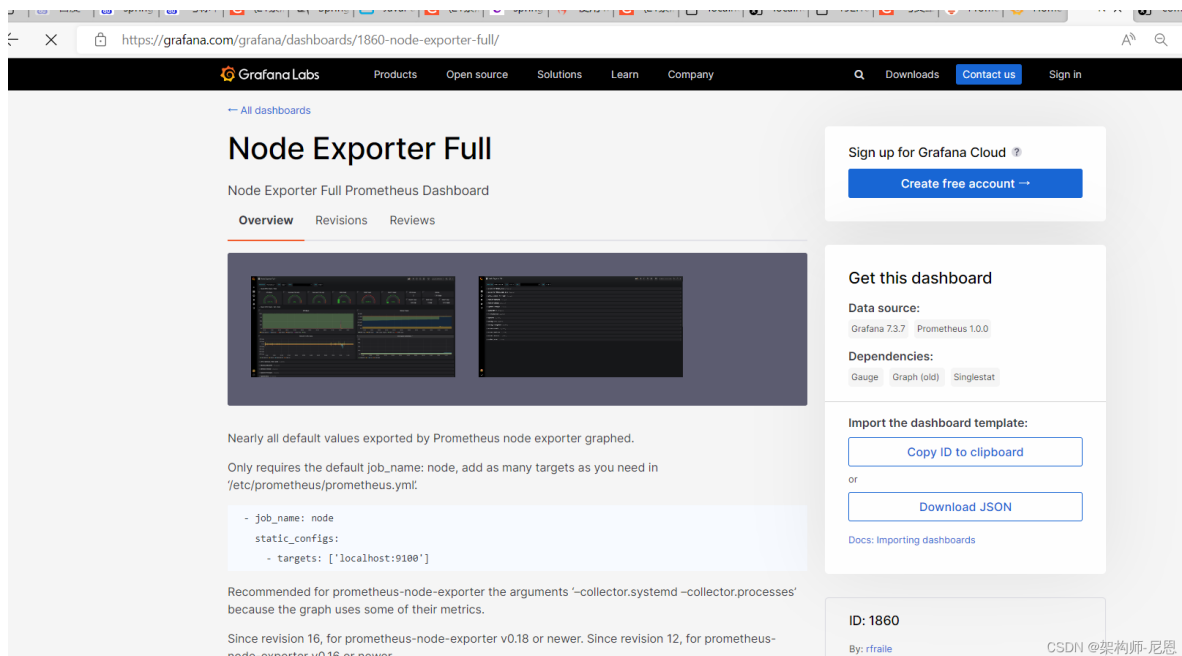
创建仪表盘grafana

重启Prometheus容器，可以通过 加号->Dashboard 来创建仪表盘；



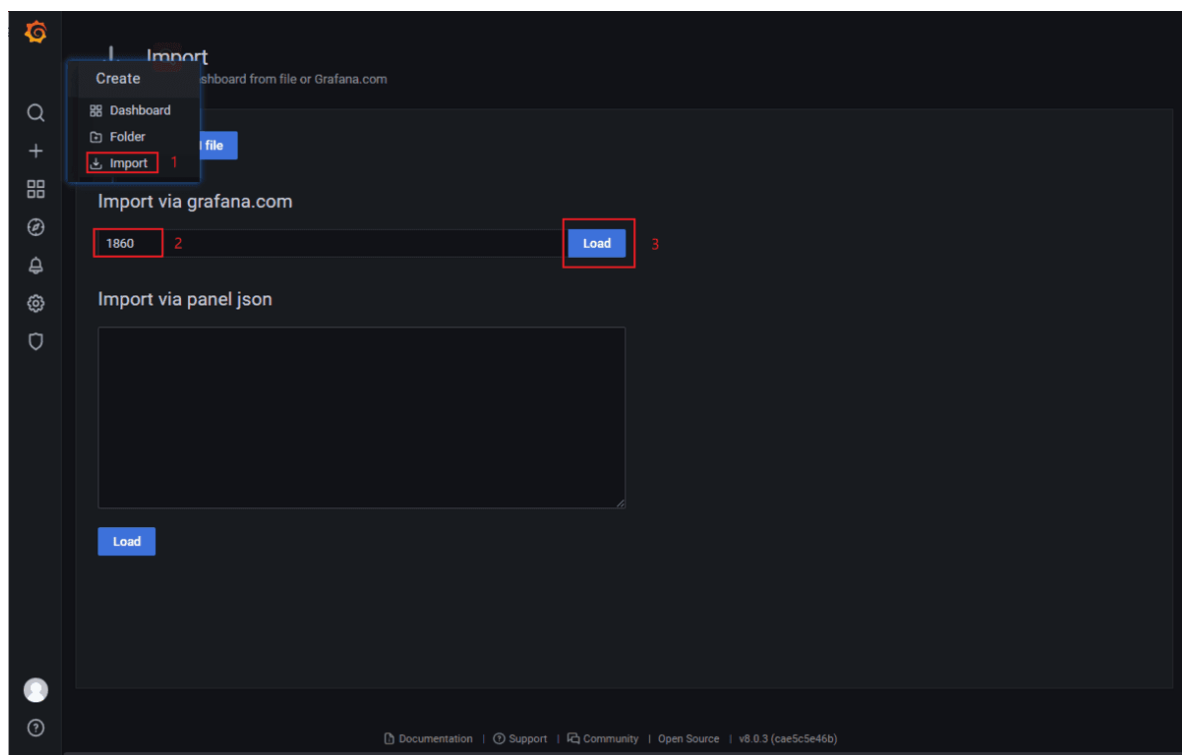
<https://grafana.com/grafana/dashboards/>



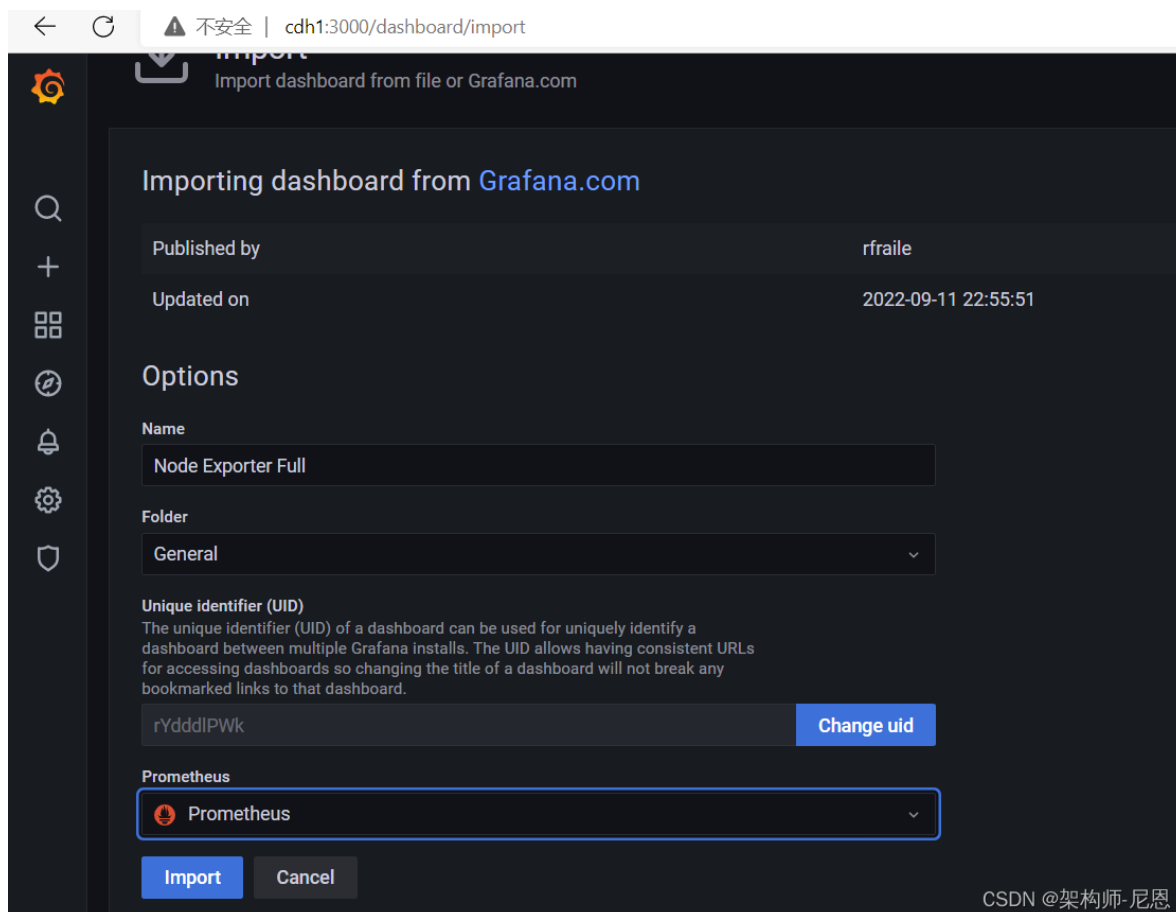


导入Dashboard

选择导入Dashboard并输入ID，最后点击 Load 即可；

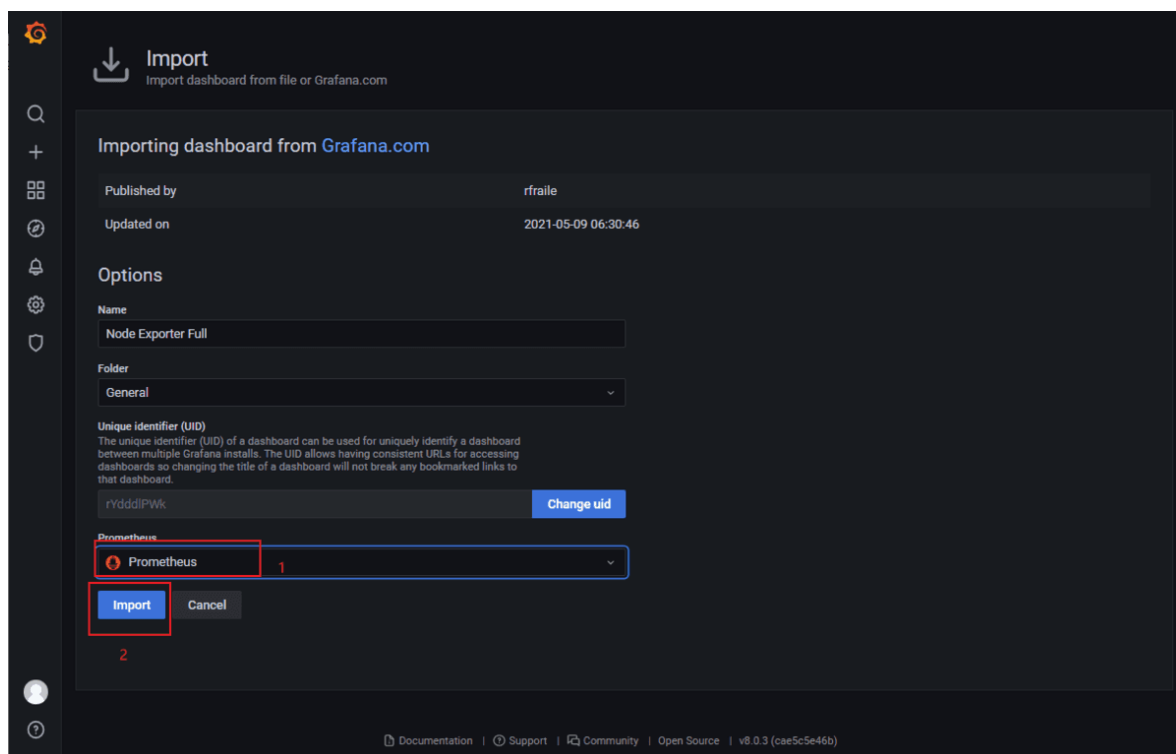


导入之后

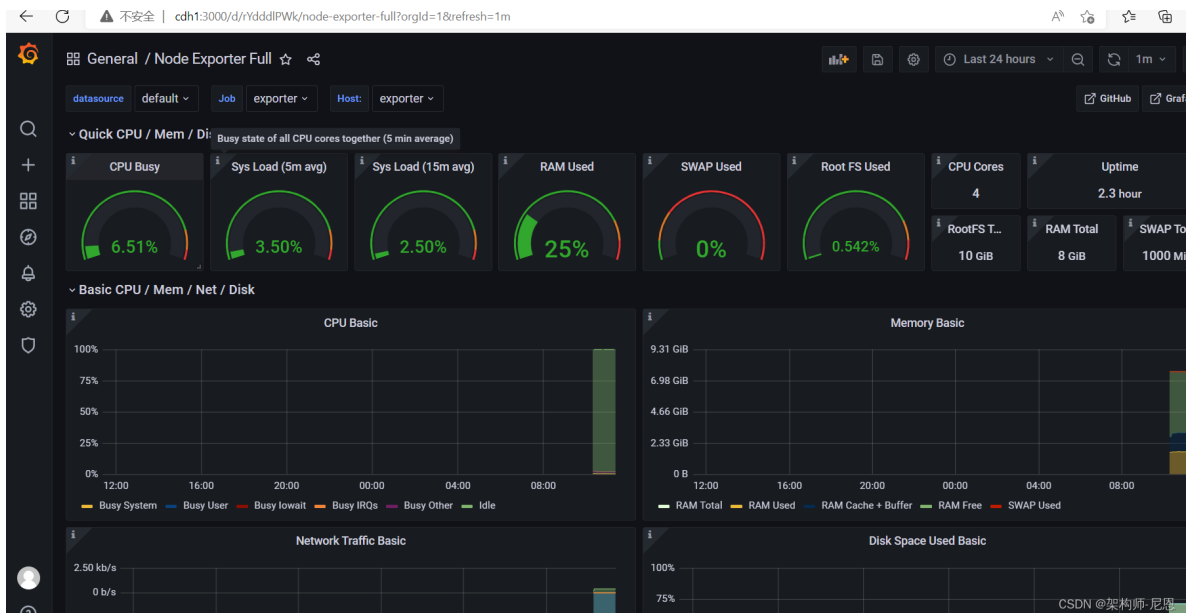


选择数据源为Prometheus

选择数据源为Prometheus，最后点击 `Import`；



导入成功后就可以在Grafana中看到实时监控信息了，是不是够炫酷！



监控实操4：配置grafana监控SpringBoot应用

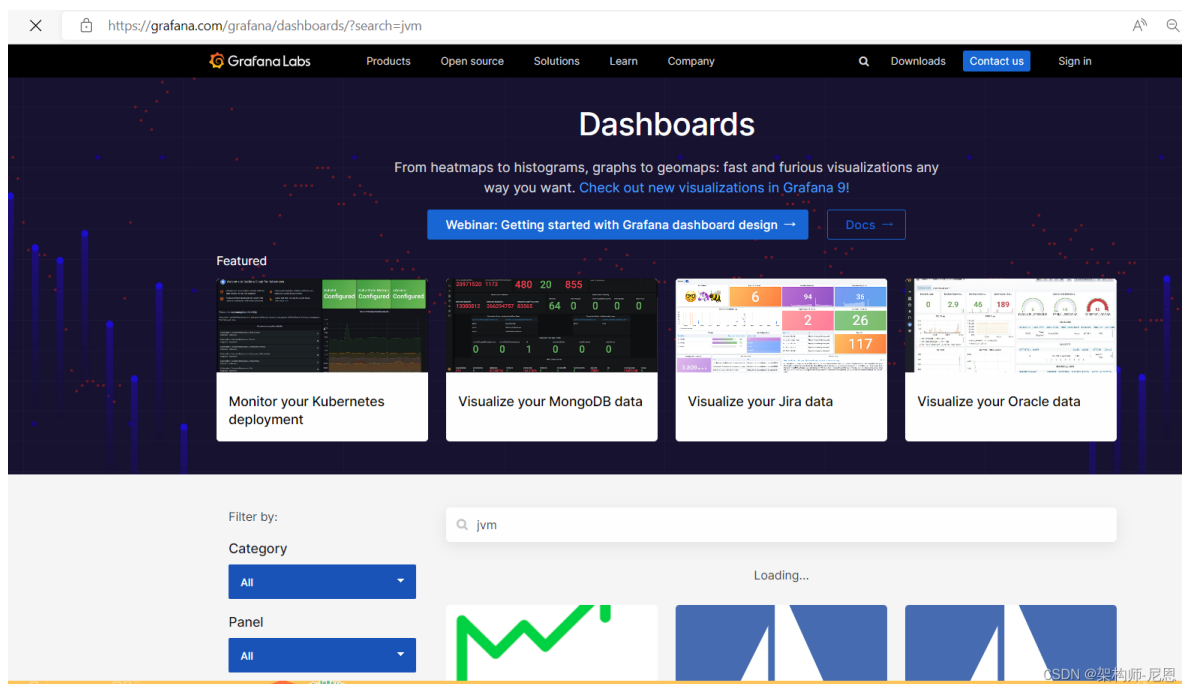
主要步骤

- 通过Prometheus提供的Java client包，在spring boot工程中生成我们关心的业务指标，
- 将spring boot工程打成docker 镜像
- 部署docker容器
- 修改Prometheus对应的 `file_sd_configs` 文件，将部署的服务追加进去
- 通过Grafana观察业务指标

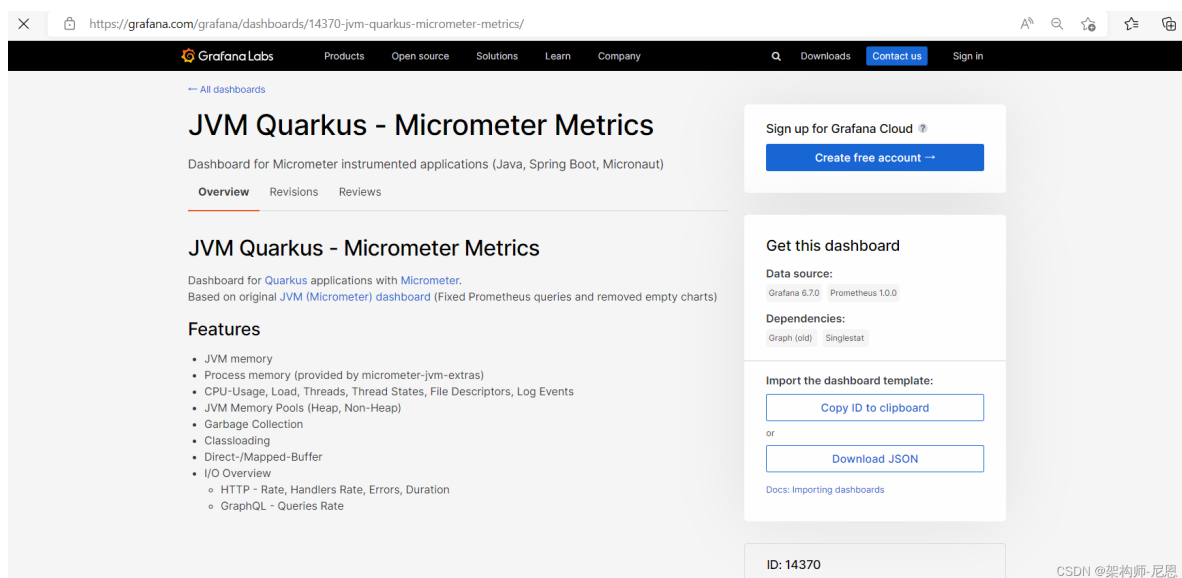
本文主要阐述的是对容器中业务指标的监控，对容器的监控以及环境的搭建参照 [一键部署 脚本](#)。

具体步骤，请参见视频

找jvm的 dashboard

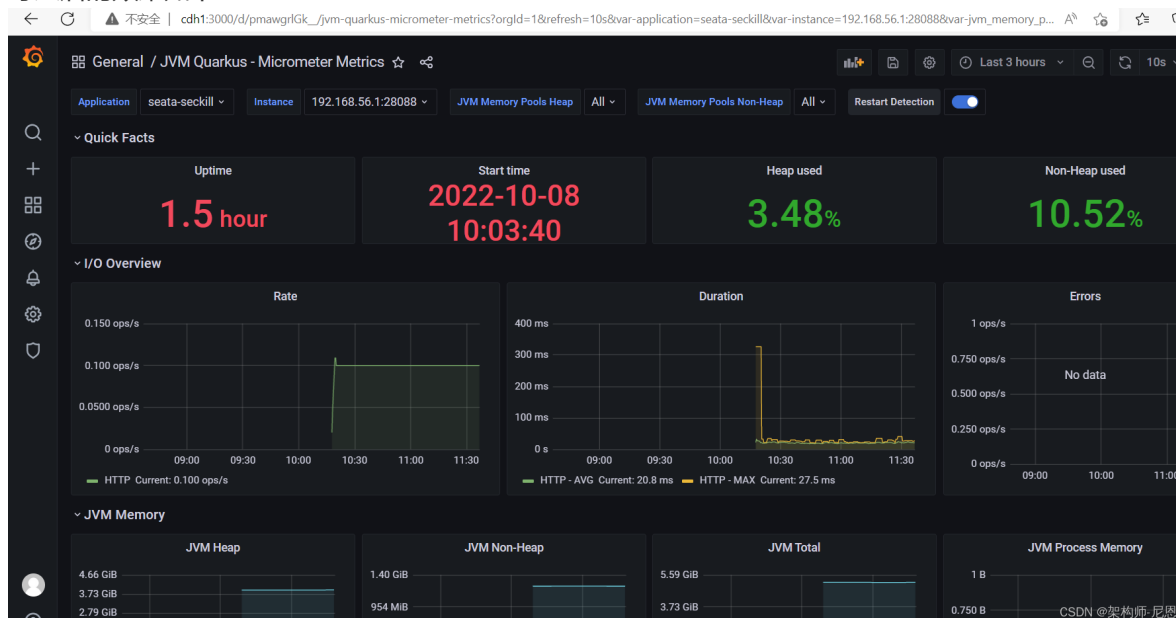


JVM Quarkus 面板



14370

导入后的效果如下：



基础知识: Prometheus数据模型

Prometheus会将所有采集到的样本数据以时间序列(time-series)的方式保存在内存数据库中，并且定时保存到硬盘上。

从根本上，说Prometheus存储的所有数据都是**时间序列**。

所以说：**Prometheus中的数据，都是时间序列值。**

time-series 时间序列值

一条 具有时间戳的数据流，只属于单个度量指标和该度量指标下的多个标签维度。

一条 Prometheus 数据，就是一条数据流，由一个指标名称 (metric) 和 N 个标签 (label, $N \geq 0$) 组成的，

例如：

`http_requests_total` 表示系统**收到请求总数**的时间序列，
同一指标中的每个key/value(称为label)都称作一个维度，如
(`http_requests_total{code=200}`) 代表请求成功的时间序列。

又例如：

```
promhttp_metric_handler_requests_total{code="200",instance="192.168.0.107:9090",job="prometheus"} 106
```

这条数据的指标名称为 `promhttp_metric_handler_requests_total`，并且包含三个标签 `code`、`instance` 和 `job`，这条记录的值为 106。

在springboot应用中，可以通过Prometheus 插件，看到大量的时间序列值，如下图所示

```
# HELP tomcat_threads_busy_threads
# TYPE tomcat_threads_busy_threads gauge
tomcat_threads_busy_threads{name="http-nio-18081",} 1.0
# HELP jvm_buffer_total_capacity_bytes An estimate of the total capacity of the buffers in this pool
# TYPE jvm_buffer_total_capacity_bytes gauge
jvm_buffer_total_capacity_bytes{id="direct",} 90112.0
jvm_buffer_total_capacity_bytes{id="mapped",} 0.0
# HELP jvm_threads_states_threads The current number of threads having NEW state
# TYPE jvm_threads_states_threads gauge
jvm_threads_states_threads{state="runnable",} 15.0
jvm_threads_states_threads{state="blocked",} 0.0
jvm_threads_states_threads{state="waiting",} 54.0
jvm_threads_states_threads{state="timed-waiting",} 21.0
jvm_threads_states_threads{state="new",} 0.0
jvm_threads_states_threads{state="terminated",} 0.0
# HELP jvm_threads_live_threads The current number of live threads including both daemon and non-daemon threads
# TYPE jvm_threads_live_threads gauge
jvm_threads_live_threads 90.0
# HELP tomcat_global_error_total
# TYPE tomcat_global_error_total counter
tomcat_global_error_total{name="http-nio-18081",} 5.0
# HELP jvm_threads_peak_threads The peak live thread count since the Java virtual machine started or peak was reset
# TYPE jvm_threads_peak_threads gauge
jvm_threads_peak_threads 90.0
# HELP process_uptime_seconds The uptime of the Java virtual machine
# TYPE process_uptime_seconds gauge
process_uptime_seconds 734.702
# HELP tomcat_sessions_alive_max_seconds
# TYPE tomcat_sessions_alive_max_seconds gauge
tomcat_sessions_alive_max_seconds 0.0
# HELP process_start_time_seconds Start time of the process since unix epoch.
# TYPE process_start_time_seconds gauge
process_start_time_seconds 1.6651911794E9
# HELP jvm_gc_live_data_size_bytes Size of old generation memory pool after a full GC
# TYPE jvm_gc_live_data_size_bytes gauge
jvm_gc_live_data_size_bytes 3.7069664E7
# HELP jvm_gc_memory_allocated_bytes_total Incremented for an increase in the size of the young generation memory pool after one GC to before
# TYPE jvm_gc_memory_allocated_bytes_total counter
jvm_gc_memory_allocated_bytes_total 9.58583704E8
# HELP jvm_gc_max_data_size_bytes Max size of old generation memory pool
# TYPE jvm_gc_max_data_size_bytes gauge
jvm_gc_max_data_size_bytes 2.822766592E9
# HELP tomcat_sessions_rejected_sessions_total
```

CSDN @架构师-尼恩

上面说过，Prometheus 是一个时序数据库，相同指标、相同标签的数据，构成一条时间序列。

外部应用通过PromQL来对时间序列进行相应的查找，

在查询时还可以利用Prometheus提供的功能丰富的函数对时间序列中的值进行计算。

每条time-series通过指标名称(metrics name)和一组标签集(labelset)命名。

换句话说：

每个序列通过 指标名称(metrics name)以及一组标签集(labelset)命名唯一确定，

总之，time-series是按照时间戳和值的序列顺序存放的，我们称之为向量(vector)。

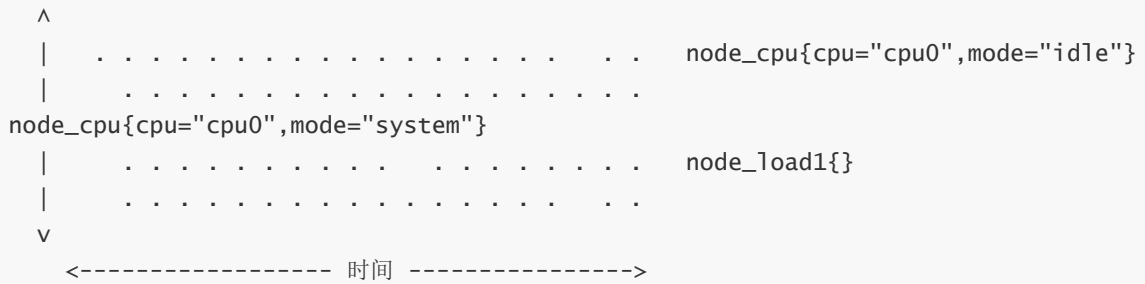
Sample样本值

Prometheus会将所有采集到的样本数据以时间序列（time-series）的方式保存在内存数据库中，并且定时保存到硬盘上。

time-series是按照时间戳和值的序列顺序存放的，我们称之为向量(vector)。

每条time-series通过指标名称(metrics name)和一组标签集(labelset)命名。

如下所示，可以将time-series理解为一个以时间为Y轴的数字矩阵：



在time-series 中的每一个点称为一个样本（sample），样本由以下三部分组成：

- 指标(metric): metric name和描述当前样本特征的labelsets;
- 时间戳(timestamp): 一个精确到毫秒的时间戳;
- 样本值(value): 一个float64的浮点型数据表示当前样本的值。

```
<----- metric -----> <-timestamp -> <-value->
http_request_total{status="200", method="GET"}@1434417560938 => 94355
http_request_total{status="200", method="GET"}@1434417561287 => 94334

http_request_total{status="404", method="GET"}@1434417560938 => 38473
http_request_total{status="404", method="GET"}@1434417561287 => 38544

http_request_total{status="200", method="POST"}@1434417560938 => 4748
http_request_total{status="200", method="POST"}@1434417561287 => 4785
```

指标metric相同，意味着(metrics name)相同，label相同的记录，称为一个样本值，样本形成了实际的时间序列数据。

每个采样值包括：

- 一个64位的浮点值
- 一个精确到毫秒级的时间戳

Prometheus一个时间序列，存的就是各个样本值，在不同时间点的数据

除了存储时间序列数据外，Prometheus还可以生成临时派生的时间序列作为查询的结果。

metrics name指标名称

在形式上，所有的指标(Metric)都通过如下格式标示：

```
<metric name>{<label name>=<label value>, ...}
```

metric度量指标名称：

指定监控目标系统的测量特征

如：http_requests_total - 接收http请求的总计数。

指标的名称(metric name)可以反映被监控样本的含义（比如，http_request_total 表示当前系统接收到的HTTP请求总量）。

metric度量指标可能包含ASCII字母、数字、下划线和冒号，

指标名称 必须配正则表达式 `[a-zA-Z_:[a-zA-Z0-9_:]*` 。

指标名称只能由ASCII字符、数字、下划线以及冒号组成并必须符合正则表达式 `[a-zA-Z_:[a-zA-Z0-9_:]*`。

标签(label)反映了当前样本的特征维度，

通过这些维度Prometheus可以对样本数据进行过滤，聚合等。标签的名称只能由ASCII字符、数字以及下划线组成并满足正则表达式 `[a-zA-Z_:[a-zA-Z0-9_:]*`。（可以根据标签label 来筛选数据）

其中以 `__` 作为前缀的标签，是系统保留的关键字，只能在系统内部使用。

标签的值则可以包含任何Unicode编码的字符。

在Prometheus的底层实现中，指标名称实际上是以 `__name__=<metric name>` 的形式保存在数据库中的，

因此以下两种方式均表示的同一条time-series：

```
api_http_requests_total{method="POST", handler="/messages"}
```

等同于：

```
{__name__="api_http_requests_total", method="POST", handler="/messages"}
```

在Prometheus源码中也可以找到指标(Metric)对应的数据结构，如下所示：

```
type Metric LabelSet

type LabelSet map[LabelName]LabelValue

type LabelName string

type LabelValue string
```

再来举个例子，一条 Prometheus 数据由一个指标名称 (metric) 和 N 个标签 (label, N >= 0) 组成的，

比如下面这个例子：

```
promhttp_metric_handler_requests_total{code="200",instance="192.168.0.107:9090",
job="prometheus"} 106
```

这条数据的指标名称为 `promhttp_metric_handler_requests_total`，

并且包含三个标签 `code`、`instance` 和 `job`，这条记录的值为 106。

上面说过，Prometheus 是一个时序数据库，相同指标相同标签的数据构成一条时间序列。

如果以传统数据库的概念来理解时序数据库，可以把指标名称当作表名，标签是字段，timestamp 是主键，还有一个 float64 类型的字段表示值（Prometheus 里面所有值都是按 float64 存储）。

这种数据模型和 OpenTSDB 的数据模型是比较类似的，详细的信息可以参考官网文档 Data model。

另外，关于指标和标签的命名，官网有一些指导性的建议，可以参考 Metric and label naming。

注意：指标名称中的冒号，是保留用于用户定义的录制规则。

它们不应被 exporter 或直接仪表使用。

label 标签

标签开启了 Prometheus 的多维数据模型：

对于相同的度量名称，通过不同标签列表的结合，会形成特定的度量维度实例。

例如：

如 (`http_requests_total{code=200}`) 代表请求成功的时间序列。

如 (`/api/tracks{method=POST}`) 所有包含度量名称为 `/api/tracks` 的 http 请求，打上 `method=POST` 的标签，则形成了具体的 http 请求。

这个查询语言在这些度量和标签列表的基础上进行过滤和聚合。

改变任何度量上的任何标签值，则会形成新的时间序列图。

标签 label 名称的命名规则：

标签 label 名称可以包含 ASCII 字母、数字和下划线。

它们必须匹配正则表达式 `[a-zA-Z_][a-zA-Z0-9_]*`。

带有 `_` 下划线的标签名称被保留内部使用。

标签 labels 值包含任意的 Unicode 码。

Notation(符号)

Notation(符号)表示一个度量指标和一组键值对标签，需要使用以下符号：

`[metric name]{[label name]=[label value], ...}`

例如，度量指标名称是 `api_http_requests_total`，标签为 `method="POST"`，`handler="/messages"` 的示例如下所示：

`api_http_requests_total{method="POST", handler="/messages"}`

这些命名和 OpenTSDB 使用方法是一样的。

TSDB 时序数据库

什么是 TSDB？

TSDB(Time Series Database) 时序数据库，我们可以简单的理解为一个优化后用来处理时间序列数据的软件，并且数据中的数组是由时间进行索引的。

OpenTSDB is a distributed, scalable Time Series Database (TSDB) written on top of HBase;

基于Hbase的分布式的，可伸缩的时间序列数据库。主要用途，就是做监控系统；譬如收集大规模集群（包括网络设备、操作系统、应用程序）的监控数据并进行存储，查询。

时间序列数据库的特点

- 1、大部分时间都是写入操作
- 2、写入操作几乎时顺序添加的，大多数示函数到达后都以时间排序。
- 3、写操作很少写入很久之前的数据，也很少更新数据。

大多数情况在数据被采集到数秒或者数分钟后就会被写入数据库

- 4、删除操作一般为区块删除，选定开始的历史时间并指定后续的区块。

很少单独删除某个时间或者分开的随机时间的数据。

- 5、基本数据大，一般超过内存大小。

一般选取的只是某一小部分且规律，缓存几乎不起任何作用。

- 6、读操作是十分典型的升序或者降序的顺序读

- 7、高并发的读操作十分常见

指标名称要表明观察的目的（不是强制的）如（`http_requests_total` 表示系统**收到请求总数**的时间序列），

同一指标中的每个key/value(称为label)都称作一个维度，如（`http_requests_total{code=200}`）代表请求成功的时间序列。

外部应用通过PromQL来对时间序列进行相应的查找，

在查询时还可以利用Prometheus提供的功能丰富的函数对时间序列中的值进行计算。

度量指标类型

Prometheus提供的客户端jar包中，把指标类型分为如下四种

- Counter
- Gauge
- Histogram
- Summary

注意：

这个分类只针对客户端使用者有效，在Prometheus server端是不进行区分的。

对于Prometheus server端而言，客户端返回的，都是时间序列对应的一个Sample。

如 `http_requests_total{code=200} 290`，

表示 Prometheus server拉取指标的这个时间点，请求成功的总数是290次。

一个Sample，是一个纯文本数据，

另外：

即便不用Prometheus提供的客户端，只要返回的数据满足这种格式，Prometheus server就能正常存储。

注意：Prometheus服务还没有充分利用这些类型，将来可能会发生改变。

Counter(计数器)类型

counter 是表示单个单调递增计数器的 **累积度量值**，其值只能在重启时增加或重置为零。

例如，您可以使用计数器来表示所服务的请求数，已完成的任务或错误。

注意：

不要使用计数器来暴露可能减少的值。

另外：

不要使用计数器来处理当前正在运行的进程数，而是使用gauge。

使用场景

Counter 用于计数，例如：请求次数、任务完成数、错误发生次数，这个值会一直增加，不会减少。

Counter对应的指标值只能是一个单独的数值，并且除了能在服务启动时重置外，只能对指标值做累加操作，不能做减法操作，

可以用来统计请求次数、任务执行次数、关键业务对象操作次数等。

Gauge(计量器、测量器)

gauge是一个度量指标，它表示一个既可以递增,又可以递减的值。

测量器主要测量类似于温度、当前内存使用量等，

也可以统计当前服务运行随时增加或者减少的线程数量

使用场景

Gauge 就是一般的数值，可大可小，例如：温度变化、内存使用变化。

- Gauge对应的指标值只能是一个单独的数值，
- 与Counter不同的是，可以对Gauge代表的指标值做加减操作，一般用来表示温度、正在执行的job等指标

Histogram(柱状图、直方图)

Histogram 柱状图，不再是简单对指标的sample值进行加减等操作，对于每一个sample值执行下面的三个操作：

- 根据Histogram定义时指定的bucket区间，将sample分到各个bucket中，每个bucket中存放的是落入这个区间的个数
- 对每个采样点值累计和(sum)
- 对采样点的次数累计和(count)

Histogram 是直方图，或称为柱状图，常用于跟踪事件发生的规模，例如：请求耗时、响应大小。

例子

例如我们通过Prometheus提供的客户端通过

```
Histogram.build().name("job_execute_time")
.help("job执行时间时间分布（分）")
.buckets(1,5,10)
.register();`
```

定义了一个histogram，用来统计job执行时间的分布。

对应的buckets是 (1, 5, 10) ，代表四个区间

- <=1分钟
- <=5分钟
- <=10分钟
- <无穷大

Histogram会生成如下6个维度的指标值

```
job_execute_time_bucket{le="1.0"},}
job_execute_time_bucket{le="5.0"},}
job_execute_time_bucket{le="10.0"},}
job_execute_time_bucket{le="+Inf"},}
job_execute_time_count
job_execute_time_sum
```

当我们有一个job执行时间为5.6分钟，则对应的各个维度的值变成

```
job_execute_time_bucket{le="1.0"},} 0.0
job_execute_time_bucket{le="5.0"},} 0.0
job_execute_time_bucket{le="10.0"},} 1.0
job_execute_time_bucket{le="+Inf"},} 1.0
job_execute_time_count 1.0
job_execute_time_sum 5.6
```

无穷大的肯定是和 `job_execute_time_count` 一致的

可以看到Histogram类型的指标不会保留各个sample的具体数值，每个bucket中也只是记录样本数的counter。

原理

直方图对观察结果进行采样（通常是请求持续时间或响应大小等），并将其计入可配置存储桶中。

它还提供所有观察值的总和。

基本度量标准名称为的直方图在scrape区间显示多个时间序列：

- 暴露的观察桶的累积计数器： `<basename>_bucket{le="<upper inclusive bound>"}`
- 所有观测值的总和： `<basename>_sum`
- 已观察到的事件数： `<basename>_count`，和 `<basename>_bucket{le="+Inf"}` 相同

使用`histogram_quantile`函数, 计算直方图或者是直方图聚合计算的分位数阈值。

一个直方图计算Apdex值也是合适的, 当在buckets上操作时, 记住直方图是累计的。

Summary

Summary 采样点分位图统计,类似于histgram, 但是采用分位数来将sample分到不同的bucket中, 具体的区别查看[HISTOGRAMS AND SUMMARIES](#),

虽然它还提供观察的总数和所有观测值的总和, 但它在滑动时间窗口上计算可配置的分位数。

基本度量标准名称 `<basename>` 的 `summary` 在scrape期间公开了多个时间序列：

- 流 ϕ -quantiles ($0 \leq \phi \leq 1$), 显示为 `<basename>{quantiles="[\phi]"}`
- `<basename>_sum`, 是指所有观察值的总和
- `<basename>_count`, 是指已观察到的事件计数值

有关 ϕ -分位数, Summary用法和histogram图差异的详细说明, 详见[histogram和summaries](#)

有关 `summaries` 的客户端使用文档：

- [Go](#)
- [Java](#)
- [Python](#)
- [Ruby](#)

Summary 和 Histogram 的区分

Summary 和 Histogram 的一个区分要点是 `summary` 在客户端侧计算分位数, 然后直接暴露它们,

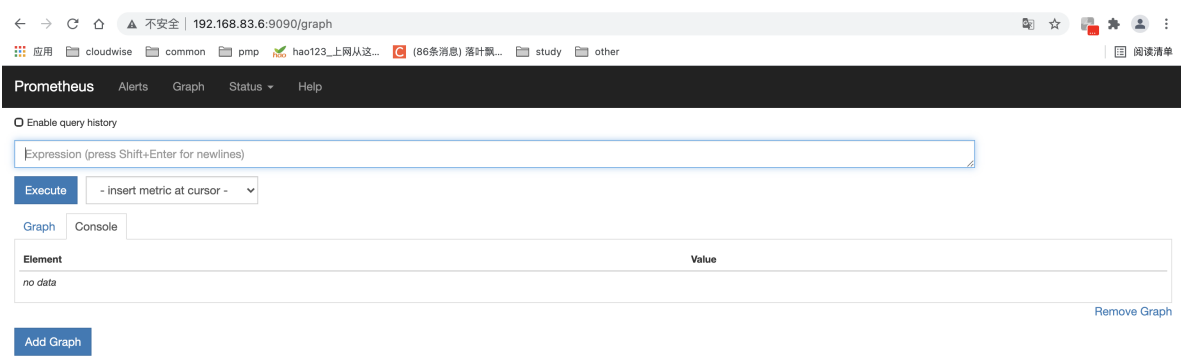
然而 `histogram` 在客户端侧暴露桶的技术, 然后在服务端侧使用 `histogram_quantile()` 函数来计算分位数。

	Histogram	Summary
所需的配置	合理定义直方图的区间	确定好分位数和滑动窗口。 其他的分位数和滑动窗口后面是不能计算的
客户端性能	监控容易，只需要累加Counter即可	监控比较昂贵，因为要计算分位数
服务端性能	服务器必须计算分位数。如果临时计算的时间过长（例如在大型仪表板中），则可以使用[记录规则]	服务端消耗低
时间序列的个数 (除了 <code>_sum</code> 和 <code>_count</code>)	每个桶一个时间序列	每个分位数一个时间序列
分位数误差	相关bucket的宽度定义决定了误差	分位数是预先定义好的
时间滑动窗口的指定分位数	使用普罗表达式的实时查询	由client预定义好
聚合	使用普罗表达式的实时查询	通常来说不可聚合

学习 PromQL

通过上面的步骤安装好 Prometheus 之后，我们现在可以开始体验 Prometheus 了。

Prometheus 提供了可视化的 Web UI 方便我们操作，直接访问 `http://localhost:9090/` 即可，它默认会跳转到 Graph 页面：



第一次访问这个页面可能会不知所措，我们可以先看看其他菜单下的内容，

比如：

Alerts 展示了定义的所有告警规则，

Status 可以查看各种 Prometheus 的状态信息，有 Runtime & Build Information、Command-Line Flags、Configuration、Rules、Targets、Service Discovery 等等。

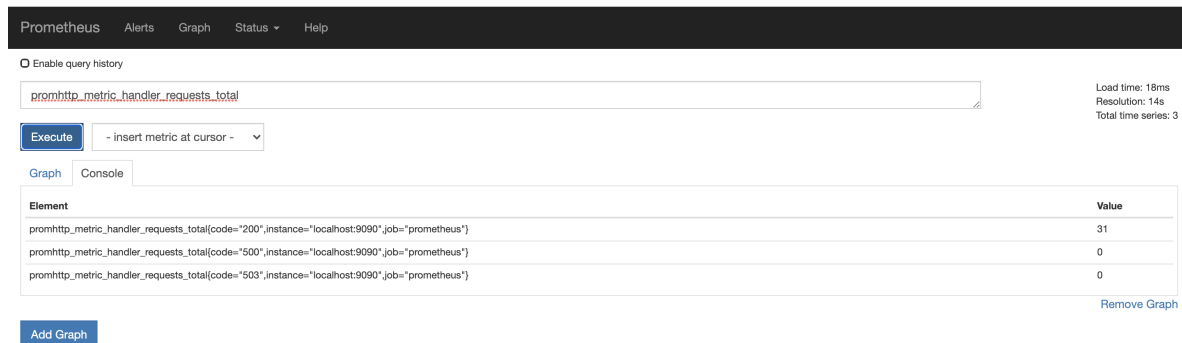
实际上 Graph 页面才是 Prometheus 最强大的功能，在这里我们可以使用 Prometheus 提供的一种特殊表达式来查询监控数据，这个表达式被称为 **PromQL (Prometheus Query Language)**。

通过 PromQL 不仅可以在 Graph 页面查询数据，而且还可以通过 Prometheus 提供的 HTTP API 来查询。

查询的监控数据有列表和曲线图两种展现形式（对应上图中 Console 和 Graph 这两个标签）。

我们上面说过，Prometheus 自身也暴露了很多的监控指标，也可以在 Graph 页面查询，展开 Execute 按钮旁边的下拉框，可以看到很多指标名称，

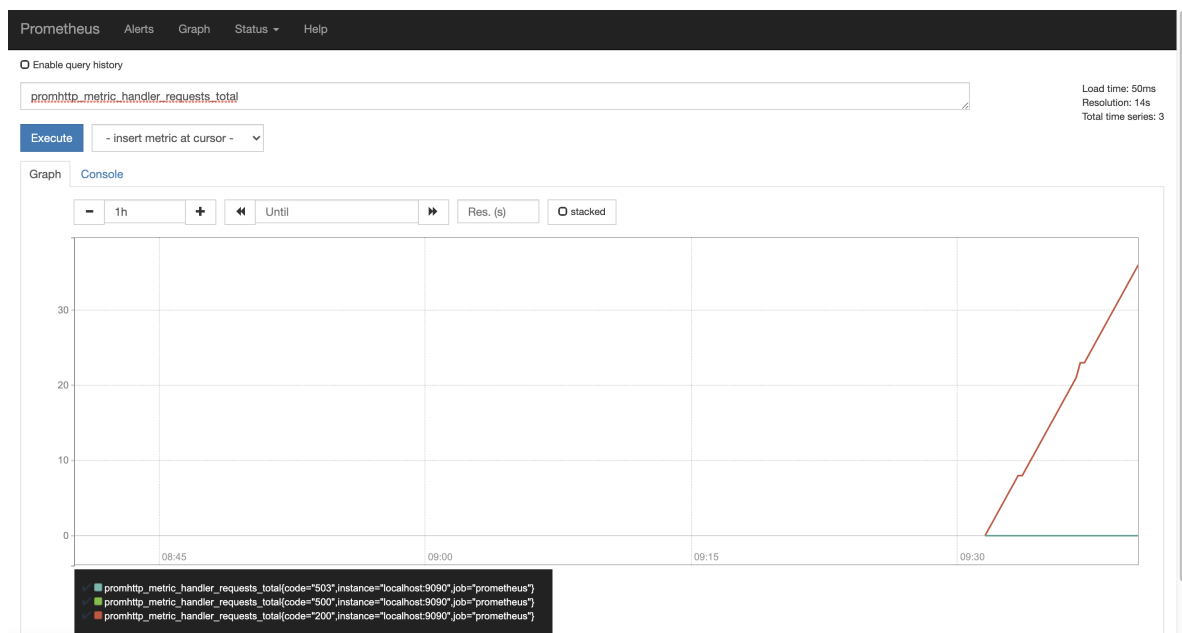
我们随便选一个，譬如：`promhttp_metric_handler_requests_total`，这个指标表示 `/metrics` 页面的访问次数，Prometheus 就是通过这个页面来抓取自身的监控数据的。在 Console 标签中查询结果如下：



上面在介绍 Prometheus 的配置文件时，可以看到 `scrape_interval` 参数是 15s，

也就是说 Prometheus 每 15s 访问一次 `/metrics` 页面，所以我们过 15s 刷新下页面，可以看到指标值会自增。

在 Graph 标签中可以看得更明显：



要学习 PromQL，首先我们需要了解下 Prometheus 的数据模型，

一条 Prometheus 数据由一个指标名称（metric）和 N 个标签（label， $N \geq 0$ ）组成的，

比如下面这个例子：

```
promhttp_metric_handler_requests_total{code="200",instance="192.168.0.107:9090",job="prometheus"} 106
```

这条数据的指标名称为 `promhttp_metric_handler_requests_total`，并且包含三个标签 `code`、`instance` 和 `job`，这条记录的值为 106。

上面说过，Prometheus 是一个时序数据库，相同指标相同标签的数据构成一条时间序列。

如果以传统数据库的概念来理解时序数据库，可以把指标名当作表名，标签是字段，timestamp 是主键，还有一个 float64 类型的字段表示值（Prometheus 里面所有值都是按 float64 存储）。

这种数据模型和 OpenTSDB 的数据模型是比较类似的，详细的信息可以参考官网文档 [Data model](#)。

另外，关于指标和标签的命名，官网有一些指导性的建议，可以参考 [Metric and label naming](#)。

虽然 Prometheus 里存储的数据都是 float64 的一个数值，但如果我们按类型来分，可以把 Prometheus 的数据分成四大类：

- Counter
- Gauge
- Histogram
- Summary

Counter 用于计数，例如：请求次数、任务完成数、错误发生次数，这个值会一直增加，不会减少。

Gauge 就是一般的数值，可大可小，例如：温度变化、内存使用变化。

Histogram 是直方图，或称为柱状图，常用于跟踪事件发生的规模，例如：请求耗时、响应大小。

它特别之处是可以对记录的内容进行分组，提供 count 和 sum 的功能。

Summary 和 Histogram 十分相似，也用于跟踪事件发生的规模，不同之处是，它提供了一个 quantiles 的功能，可以按百分比划分跟踪的结果。

例如：quantile 取值 0.95，表示取采样值里面的 95% 数据。

更多信息可以参考官网文档 [Metric types](#)，Summary 和 Histogram 的概念比较容易混淆，属于比较高阶的指标类型，可以参考 [Histograms and summaries](#) 这里的说明。

这四种类型的数据只在指标的提供方区分，也就是上面说的 Exporter，如果你需要编写自己的 Exporter 或者在现有系统中暴露供 Prometheus 抓取的指标，你可以使用 [Prometheus client libraries](#)，这个时候你就需要考虑不同指标的数据类型了。如果你不用自己实现，而是直接使用一些现成的 Exporter，然后在 Prometheus 里查查相关的指标数据，那么可以不用太关注这块，不过理解 Prometheus 的数据类型，对写出正确合理的 PromQL 也是有帮助的。

PromQL 入门

我们从一些例子开始学习 PromQL，

最简单的 PromQL

最简单的 PromQL 就是直接输入指标名称，比如：

```
# 表示 Prometheus 能否抓取 target 的指标，用于 target 的健康检查
up
```

这条语句会查出 Prometheus 抓取的所有 target 当前运行情况，譬如下面这样：

```
up{instance="192.168.0.107:9090",job="prometheus"}    1
up{instance="192.168.0.108:9090",job="prometheus"}    1
up{instance="192.168.0.107:9100",job="server"}    1
up{instance="192.168.0.108:9104",job="mysql"}    0
```

也可以指定某个 label 来查询：

```
up{job="prometheus"}
```

这种写法被称为 [Instant vector selectors](#),

这里不仅可以使用 `=` 号, 还可以使用 `!=`、`=~`、`!~`, 比如下面这样:

```
up{job!="prometheus"}
up{job=~"server|mysql"}
up{job=~"192\.168\.0\.107\."}
```

`=~` 是根据正则表达式来匹配, 必须符合 [RE2 的语法](#)。

和 Instant vector selectors 相应的, 还有一种选择器, 叫做 [Range vector selectors](#),

它可以查出一段时间内的所有数据:

```
http_requests_total[5m]
```

这条语句查出 5 分钟内所有抓取的 HTTP 请求数, 注意它返回的数据类型是 `Range vector`, 没办法在 Graph 上显示成曲线图,

一般情况下, 会用在 Counter 类型的指标上, 并和 `rate()` 或 `irate()` 函数一起使用 (注意 `rate` 和 `irate` 的区别)。

```
# 计算的是每秒的平均值, 适用于变化很慢的 counter
# per-second average rate of increase, for slow-moving counters
rate(http_requests_total[5m])

# 计算的是每秒瞬时增加速率, 适用于变化很快的 counter
# per-second instant rate of increase, for volatile and fast-moving counters

irate(http_requests_total[5m])

sum(irate(gateway_request_seconds_count{application=\"api-gateway\"}[15s]))
```

此外, PromQL 还支持 `count`、`sum`、`min`、`max`、`topk` 等 [聚合操作](#),

还支持 `rate`、`abs`、`ceil`、`floor` 等一堆的 [内置函数](#), [更多的例子](#), 还是上官网学习吧。

如果感兴趣, 我们还可以把 [PromQL 和 SQL 做一个对比](#), 会发现 PromQL 语法更简洁, 查询性能也更高。

除了使用 `m` 表示分钟以外, PromQL 的时间范围选择器支持其它时间单位:

- `s` - 秒

- m - 分钟
- h - 小时
- d - 天
- w - 周
- y - 年

时间位移操作

在瞬时向量表达式或者区间向量表达式中，都是以当前时间为基准：

```
http_request_total{} # 瞬时向量表达式，选择当前最新的数据
http_request_total{}[5m] # 区间向量表达式，选择以当前时间为基准，5分钟内的数据
```

而如果我们想查询，5分钟前的瞬时样本数据，或昨天一天的区间内的样本数据呢？

这个时候我们就可以使用位移操作，位移操作的关键字为**offset**。

可以使用offset时间位移操作：

```
http_request_total{} offset 5m
http_request_total{}[1d] offset 1d
```

使用聚合操作

一般来说，如果描述样本特征的标签(label)在并非唯一的情况下，

通过PromQL查询数据，会返回多条满足这些特征维度的时间序列。

而PromQL提供的聚合操作可以用来对这些时间序列进行处理，形成一条新的时间序列：

```
# 查询系统所有http请求的总量
sum(http_request_total)

# 按照mode计算主机CPU的平均使用时间
avg(node_cpu) by (mode)

# 按照主机查询各个主机的CPU使用率
sum(sum(irate(node_cpu{mode!='idle'}[5m])) / sum(irate(node_cpu[5m]))) by
(instance)
```

PromQL聚合操作

Prometheus还提供了下列内置的聚合操作符，这些操作符作用域瞬时向量。

可以将瞬时表达式返回的样本数据进行聚合，形成一个新的时间序列。

- `sum` (求和)
- `min` (最小值)
- `max` (最大值)

- avg (平均值)
- stddev (标准差)
- stdvar (标准方差)
- count (计数)
- count_values (对value进行计数)
- bottomk (后n条时序)
- topk (前n条时序)
- quantile (分位数)

使用聚合操作的语法如下：

```
<aggr-op>([parameter,] <vector expression>) [without|by (<label list>)]
```

其中只有 count_values, quantile, topk, bottomk 支持参数(parameter)。

without用于从计算结果中移除列举的标签，而保留其它标签。by则正好相反，结果向量中只保留列出的标签，其余标签则移除。通过without和by可以按照样本的问题对数据进行聚合。

例如：

```
sum(http_requests_total) without (instance)
```

等价于

```
sum(http_requests_total) by (code,handler,job,method)
```

如果只需要计算整个应用的HTTP请求总量，可以直接使用表达式：

```
sum(http_requests_total)
```

count_values用于时间序列中每一个样本值出现的次数。count_values会为每一个唯一的样本值输出一个时间序列，并且每一个时间序列包含一个额外的标签。

```
count_values("count", http_requests_total)
```

topk和bottomk则用于对样本值进行排序，返回当前样本值前n位，或者后n位的时间序列。

获取HTTP请求数前5位的时序样本数据，可以使用表达式：

```
topk(5, http_requests_total)
```

quantile用于计算当前样本数据值的分布情况quantile(φ , express)其中 $0 \leq \varphi \leq 1$ 。

例如，当 φ 为0.5时，即表示找到当前样本数据中的中位数：

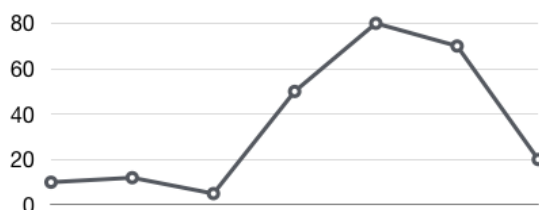
```
quantile(0.5, http_requests_total)
```

计算Counter指标增长率

我们知道Counter类型的监控指标其特点是只增不减，在没有发生重置（如服务器重启，应用重启）的情况下其样本值应该是不断增大的。

为了能够更直观地表示样本数据的变化剧烈情况，需要计算样本的增长速率。

如下图所示，样本增长率反映出了样本变化的剧烈程度：



increase(v range-vector)函数是PromQL中提供的众多内置函数之一。

其中参数v是一个区间向量，increase函数获取区间向量中的第一个后最后一个样本并返回其增长量。因此，可以通过以下表达式Counter类型指标的增长率：

```
increase(node_cpu[2m]) / 120
```

这里通过node_cpu[2m]获取时间序列最近两分钟的所有样本，increase计算出最近两分钟的增长量，最后除以时间120秒得到node_cpu样本在最近两分钟的平均增长率。

并且这个值也近似于主机节点最近两分钟内的平均CPU使用率。

rate函数

除了使用increase函数以外，PromQL中还直接内置了rate(v range-vector)函数，rate函数可以直接计算区间向量v在时间窗口内平均增长速率。

因此，通过以下表达式可以得到与increase函数相同的结果：

```
rate(node_cpu[2m])
```

需要注意的是使用rate或者increase函数去计算样本的平均增长速率，容易陷入“长尾问题”当中，其无法反应在时间窗口内样本数据的突发变化。

例如，对于主机而言在2分钟的时间窗口内，可能在某一个由于访问量或者其它问题导致CPU占用100%的情况，但是通过计算在时间窗口内的平均增长率却无法反应出该问题。

为了解决该问题，PromQL提供了另外一个灵敏度更高的函数irate(v range-vector)。

irate同样用于计算区间向量的计算率，但是其反应出的是瞬时增长率。irate函数是通过区间向量中最后两个样本数据来计算区间向量的增长速率。

这种方式可以避免在时间窗口范围内的“长尾问题”，并且体现出更好的灵敏度，通过irate函数绘制的图标能够更好的反应样本数据的瞬时变化状态。

```
irate(node_cpu[2m])
```

irate函数相比于rate函数提供了更高的灵敏度，不过当需要分析长期趋势或者在告警规则中，irate的这种灵敏度反而容易造成干扰。

因此在长期趋势分析或者告警中更推荐使用rate函数。

预测Gauge指标变化趋势

在一般情况下，系统管理员为了确保业务的持续可用运行，会针对服务器的资源设置相应的告警阈值。例如，当磁盘空间只剩512MB时向相关人员发送告警通知。

这种基于阈值的告警模式对于当资源用量是平滑增长的情况下是能够有效的工作的。但是如果资源不是平滑变化的呢？比如有些某些业务增长，存储空间的增长速率提升了高几倍。

这时，如果基于原有阈值去触发告警，当系统管理员接收到告警以后可能还没来得及去处理问题，系统就已经不可用了。

因此阈值通常来说不是固定的，需要定期进行调整才能保证该告警阈值能够发挥去作用。那么还有没有更好的方法吗？

PromQL中内置的predict_linear(v range-vector, t scalar) 函数可以帮助系统管理员更好的处理此类情况，predict_linear函数可以预测时间序列v在t秒后的值。它基于简单线性回归的方式，对时间窗口内的样本数据进行统计，从而可以对时间序列的变化趋势做出预测。

例如，基于2小时的样本数据，来预测主机可用磁盘空间的是否在4个小时被占满，可以使用如下表达式：

```
predict_linear(node_filesystem_free{job="node"}[2h], 4 * 3600) < 0
```

统计Histogram指标的分位数

在上方，我们介绍了Prometheus的四种监控指标类型，其中Histogram和Summary都可以用于统计和分析数据的分布情况。

区别在于Summary是直接在客户端计算了数据分布的分位数情况。而Histogram的分位数计算需要通过histogram_quantile(ϕ float, b instant-vector)函数进行计算。

其中 ϕ ($0 < \phi < 1$) 表示需要计算的分位数，如果需要计算中位数 ϕ 取值为0.5，以此类推即可。

以指标http_request_duration_seconds_bucket为例：

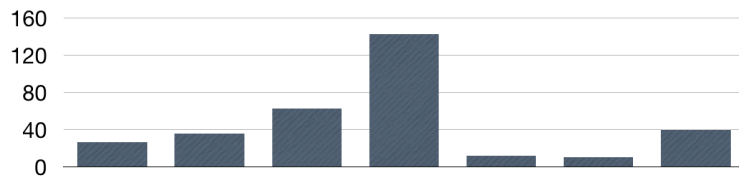
```
# HELP http_request_duration_seconds request duration histogram
# TYPE http_request_duration_seconds histogram
http_request_duration_seconds_bucket{le="0.5"} 0
http_request_duration_seconds_bucket{le="1"} 1
http_request_duration_seconds_bucket{le="2"} 2
http_request_duration_seconds_bucket{le="3"} 3
http_request_duration_seconds_bucket{le="5"} 3
http_request_duration_seconds_bucket{le="+Inf"} 3
http_request_duration_seconds_sum 6
http_request_duration_seconds_count 3
```

当计算9分位数时，使用如下表达式：

```
histogram_quantile(0.5, http_request_duration_seconds_bucket)
```

通过对Histogram类型的监控指标，用户可以轻松获取样本数据的分布情况。

同时分位数的计算，也可以非常方便的用于评判当前监控指标的服务水平。



需要注意的是通过histogram_quantile计算的分位数，并非为精确值，而是通过http_request_duration_seconds_bucket和http_request_duration_seconds_sum近似计算的结果。

HTTP API

我们不仅仅可以在 Prometheus 的 Graph 页面查询 PromQL，Prometheus 还提供了一种 HTTP API 的方式，可以更灵活的将 PromQL 整合到其他系统中使用，譬如下面要介绍的 Grafana，就是通过 Prometheus 的 HTTP API 来查询指标数据的。

实际上，我们在 Prometheus 的 Graph 页面查询也是使用了 HTTP API。

我们看下 [Prometheus 的 HTTP API 官方文档](#)，它提供了下面这些接口：

- GET /api/v1/query
- GET /api/v1/query_range
- GET /api/v1/series
- GET /api/v1/label/<label_name>/values
- GET /api/v1/targets
- GET /api/v1/rules
- GET /api/v1/alerts
- GET /api/v1/targets/metadata
- GET /api/v1/alertmanagers
- GET /api/v1/status/config
- GET /api/v1/status/flags

从 Prometheus v2.1 开始，又新增了几个用于管理 TSDB 的接口：

- POST /api/v1/admin/tsdb/snapshot
- POST /api/v1/admin/tsdb/delete_series
- POST /api/v1/admin/tsdb/clean_tombstones

告警和通知

至此，我们能收集大量的指标数据，也能通过强大而美观的面板展示出来。不过作为一个监控系统，最重要的功能，还是应该能及时发现系统问题，并及时通知给系统负责人，这就是 [Alerting \(告警\)](#)。

Prometheus 的告警功能被分成两部分：

- 一个是告警规则的配置和检测，并将告警发送给 [Alertmanager](#),

- 另一个是 Alertmanager，它负责管理这些告警，去除重复数据，分组，并路由到对应的接收方式，发出报警。

常见的接收方式有：Email、PagerDuty、HipChat、Slack、OpsGenie、WebHook 等。

配置告警规则

我们在上面介绍 Prometheus 的配置文件时了解到，

它的默认配置文件 prometheus.yml 有四大块：global、alerting、rule_files、scrape_config，其中 rule_files 块就是告警规则的配置项，alerting 块用于配置 Alertmanager，这个我们下一节再看。

现在，先让我们在 rule_files 块中添加一个告警规则文件：

```
rule_files:
  - "alert.rules"
```

然后参考 [官方文档](#)，创建一个告警规则文件 `alert.rules`：

```
groups:
- name: example
  rules:

  # Alert for any instance that is unreachable for >5 minutes.
  - alert: InstanceDown
    expr: up == 0
    for: 5m
    labels:
      severity: page
    annotations:
      summary: "Instance {{ $labels.instance }} down"
      description: "{{ $labels.instance }} of job {{ $labels.job }} has been down for more than 5 minutes."

  # Alert for any instance that has a median request latency >1s.
  - alert: APIHighRequestLatency
    expr: api_http_request_latencies_second{quantile="0.5"} > 1
    for: 10m
    annotations:
      summary: "High request latency on {{ $labels.instance }}"
      description: "{{ $labels.instance }} has a median request latency above 1s (current value: {{ $value }}s)"
```

这个规则文件里，包含了两条告警规则：`InstanceDown` 和 `APIHighRequestLatency`。

顾名思义，InstanceDown 表示当实例宕机时（up == 0）触发告警，APIHighRequestLatency 表示有一半的 API 请求延迟大于 1s 时（api_http_request_latencies_second{quantile="0.5"} > 1）触发告警。

配置好后，需要重启下 Prometheus server，然后访问 `http://localhost:9090/rules` 可以看到刚刚配置的规则：

Rules			
example			1.303ms
Rule	State	Error	Evaluation Time
<pre> alert: InstanceDown expr: up == 0 for: 5m labels: severity: page annotations: description: '{{ \$labels.instance }} of job {{ \$labels.job }} has been down for more than 5 minutes.' summary: Instance {{ \$labels.instance }} down </pre>	OK		1.059ms
<pre> alert: APIMHighRequestLatency expr: api_http_request_latencies_second(quantile="0.5") > 1 for: 10m annotations: description: '{{ \$labels.instance }} has a median request latency above is (current value: {{ \$value }}s)' summary: High request latency on {{ \$labels.instance }} </pre>	OK		223.8us

CSDN @40岁资深老架构师尼恩

访问 `http://localhost:9090/alerts` 可以看到根据配置的规则生成的告警：

Alerts			
Show annotations			
InstanceDown (1 active)			
<pre> alert: InstanceDown expr: up == 0 for: 5m labels: severity: page annotations: description: '{{ \$labels.instance }} of job {{ \$labels.job }} has been down for more than 5 minutes.' summary: Instance {{ \$labels.instance }} down </pre>			
Labels	State	Active Since	Value
alertname="InstanceDown" instance="192.168.0.107:9104" job="mysql" severity="page"	PENDING	2018-10-26 06:04:14.671040883 +0000 UTC	0
HighErrorRate (0 active)			

CSDN @40岁资深老架构师尼恩

这里我们将一个实例停掉，可以看到有一条 alert 的状态是 `PENDING`，这表示已经触发了告警规则，但还没有达到告警条件。

这是因为这里配置的 `for` 参数是 5m，也就是 5 分钟后才会触发告警，我们等 5 分钟，可以看到这条 alert 的状态变成了 `FIRING`。

使用 Alertmanager 发送告警通知

虽然 Prometheus 的 `/alerts` 页面可以看到所有的告警，但是还差最后一步：触发告警时自动发送通知。这是由 Alertmanager 来完成的，我们首先 [下载并安装 Alertmanager](#)，和其他 Prometheus 的组件一样，Alertmanager 也是开箱即用的：

```

$ wget
https://github.com/prometheus/alertmanager/releases/download/v0.15.2/alertmanager-0.15.2.linux-amd64.tar.gz
$ tar xvfz alertmanager-0.15.2.linux-amd64.tar.gz
$ cd alertmanager-0.15.2.linux-amd64
$ ./alertmanager

```

Alertmanager 启动后默认可以通过 `http://localhost:9093/` 来访问，但是现在还看不到告警，因为我们还没有把 Alertmanager 配置到 Prometheus 中，我们回到 Prometheus 的配置文件 `prometheus.yml`，添加下面几行：

```

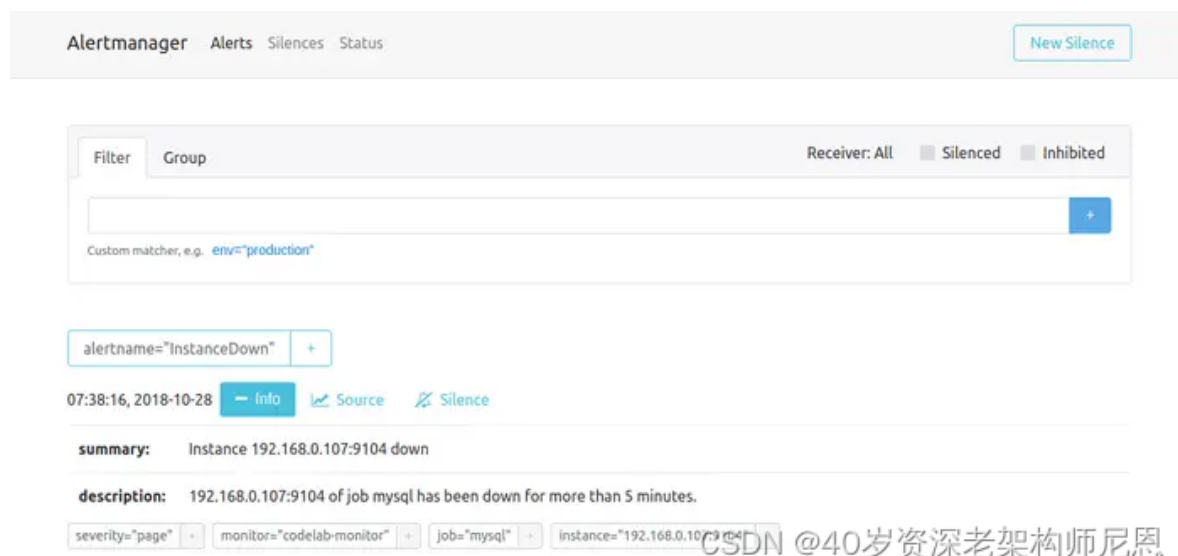
alerting:
  alertmanagers:
    - scheme: http
      static_configs:
        - targets:
          - "192.168.0.107:9093"

```

这个配置告诉 Prometheus，当发生告警时，将告警信息发送到 Alertmanager，Alertmanager 的地址为 `http://192.168.0.107:9093`。也可以使用命名行的方式指定 Alertmanager：

```
$ ./prometheus -alertmanager.url=http://192.168.0.107:9093
```

这个时候再访问 Alertmanager，可以看到 Alertmanager 已经接收到告警了：



下面的问题就是如何让 Alertmanager 将告警信息发送给我们了，我们打开默认的配置文件 `alertmanager.yml`：

```
global:
  resolve_timeout: 5m

route:
  group_by: ['alertname']
  group_wait: 10s
  group_interval: 10s
  repeat_interval: 1h
  receiver: 'web.hook'
receivers:
- name: 'web.hook'
  webhook_configs:
  - url: 'http://127.0.0.1:5001/'
inhibit_rules:
- source_match:
  severity: 'critical'
  target_match:
  severity: 'warning'
  equal: ['alertname', 'dev', 'instance']
```

参考 [官方的配置手册](#) 了解各个配置项的功能，其中 `global` 块表示一些全局配置；

`route` 块表示通知路由，可以根据不同的标签将告警通知发送给不同的 receiver，这里没有配置 `routes` 项，

表示所有的告警都发送给下面定义的 `web.hook` 这个 receiver；如果要配置多个路由，可以参考 [这个例子](#)：

```

routes:
- receiver: 'database-pager'
  group_wait: 10s
  match_re:
    service: mysql|cassandra

- receiver: 'frontend-pager'
  group_by: [product, environment]
  match:
    team: frontend
routes: ``- receiver: \'database-pager\'`` ``group_wait:
10s`` ``match_re: `` ``service: mysql|cassandra`` - receiver: \'frontend-
pager\'`` ``group_by: [product, environment]`` ``match:`` ``team: frontend

```

紧接着，receivers 块表示告警通知的接收方式，每个 receiver 包含一个 name 和一个 xxx_configs，不同的配置代表了不同的接收方式，Alertmanager 内置了下面这些接收方式：

- email_config
- hipchat_config
- pagerduty_config
- pushover_config
- slack_config
- opsgenie_config
- victorops_config
- wechat_configs
- webhook_config

虽然接收方式很丰富，但是在国内，其中大多数接收方式都很少使用。

最常用到的，莫属 email_config 和 webhook_config，另外 wechat_configs 可以支持使用微信来告警，也是相当符合国情的了。

其实告警的通知方式是很难做到面面俱到的，因为消息软件各种各样，每个国家还可能不同，不可能完全覆盖到，所以 Alertmanager 已经决定不再添加新的 receiver 了，而是推荐使用 webhook 来集成自定义的接收方式。

可以参考 [这些集成的例子](#)，譬如 [将钉钉接入 Prometheus AlertManager WebHook](#)。

动态服务发现

由于 Prometheus 是通过 Pull 的方式主动获取监控数据，所以需要手工指定监控节点的列表，当监控的节点增多之后，每次增加节点都需要更改配置文件，非常麻烦，

实际项目中，我们不可能一个一个手动的配置要抓取的目标，每次都去修改prometheus.yml文件，然后再重启服务，想都不要想，不可能这么做。

为此，我们需要动态发现目标。

这个时候就需要通过服务发现（service discovery，SD）机制去解决。

Prometheus 支持多种服务发现机制，可以自动获取要收集的 targets，可以参考 [这里](#)，

Prometheus支持很多的服务发现配置，比如：zookeeper、eureka、kubernetes等等

更多包含的服务发现机制包括：azure、consul、dns、ec2、openstack、file、gce、kubernetes、marathon、triton、zookeeper（nerve、serverset），

配置方法可以参考手册的 [Configuration](#) 页面。

可以说 SD 机制是非常丰富的，但目前由于开发资源有限，已经不再开发新的 SD 机制，只对基于文件的 SD 机制进行维护。

关于服务发现网上有很多教程

譬如 Prometheus 官方博客中这篇文章 [Advanced Service Discovery in Prometheus 0.14.0](#) 对此有一个比较系统的介绍，全面的讲解了 [relabeling 配置](#)，

以及如何使用 DNS-SRV、Consul 和文件来做服务发现。

另外，官网还提供了 [一个基于文件的服务发现的入门例子](#)，Julius Volz 写的 Prometheus workshop 入门教程中也 [使用了 DNS-SRV 来当服务发现](#)。

为什么需要服务发现

Prometheus Server 的数据抓取工作基于 Pull 模型，因而，它必须要事先知道各 target 的位置，然后才能从相应的 Exporter 或 Instrumentation 中抓取数据。

对于小型的系统环境，使用 static_configs 指定各 target 即可解决问题，但是对于较大的集群不适用，尤其不适用于使用容器和基于云的实例的动态集群，因为这些实例会经常出现变化、创建、或销毁的情况。Prometheus 为此专门设计了一组服务发现机制，以便于能够基于服务注册中心自动发现、检测、分类可被监控的各 target，以及更新发生了变动的 target。

Prometheus 可以集成到多种不同的开源服务发现工具上，以便动态发现需要监控的目标。

Prometheus 可以很好的集成到 Kubernetes 平台上，通过其 API Server 动态发现各类被监控的 Pod、Service、Endpoint、Ingress 及 Node 对象，还支持基于文件实现的动态发现。

prometheus目前支持的服务发现类型

prometheus目前支持的服务发现类型主要有如下几种：

- 1、基于文件的服务发现
- 2、基于consul 的服务发现
- 3、基于k8s API的服务发现
- 4、基于eureka的服务发现
- 5、基于nacos的服务发现
- 6、基于DNS的服务发现

```
# List of Azure service discovery configurations.
azure_sd_configs:
  [ - <azure_sd_config> ... ]

# List of Consul service discovery configurations.
consul_sd_configs:
  [ - <consul_sd_config> ... ]

# List of DNS service discovery configurations.
dns_sd_configs:
  [ - <dns_sd_config> ... ]

# List of EC2 service discovery configurations.
ec2_sd_configs:
  [ - <ec2_sd_config> ... ]
```

```

# List of OpenStack service discovery configurations.
openstack_sd_configs:
  [ - <openstack_sd_config> ... ]

# List of file service discovery configurations.
file_sd_configs:
  [ - <file_sd_config> ... ]

# List of GCE service discovery configurations.
gce_sd_configs:
  [ - <gce_sd_config> ... ]

# List of Kubernetes service discovery configurations.
kubernetes_sd_configs:
  [ - <kubernetes_sd_config> ... ]

# List of Marathon service discovery configurations.
marathon_sd_configs:
  [ - <marathon_sd_config> ... ]

# List of AirBnB's Nerve service discovery configurations.
nerve_sd_configs:
  [ - <nerve_sd_config> ... ]

# List of Zookeeper Serverset service discovery configurations.
serverset_sd_configs:
  [ - <serverset_sd_config> ... ]

# List of Triton service discovery configurations.
triton_sd_configs:
  [ - <triton_sd_config> ... ]

```

动态服务发现1: 基于文件的服务发现方式

file_sd_configs

```

[root@node01 prometheus]# vi prometheus.yml

- job_name: 'node-exporter'
  file_sd_configs:
    # 基于文件的服务发现方式
    - files:
      # 指定json 或则 yml文件位置
      - /opt/prometheus/node-exporter.json

```

```

[root@node01 prometheus]# vi node-exporter.json

[
  {
    "targets": [

```

```
    "172.16.20.00:9100"
  ],
  "labels": {
    "group": "aa",
    "app": "web",
    "hostname": "node01"
  }
},
{
  "targets": [
    "172.16.20.00:9100"
  ],
  "labels": {
    "group": "bb",
    "app": "devops",
    "hostname": "node02"
  }
}
]
```

通过这种方式，Prometheus会自动的周期性读取文件中的内容。

当文件中定义的内容发生变化时，不需要对Prometheus进行任何的重启操作。

动态服务发现2: 基于consul 的服务发现

什么是基于consul的服务发现

我们大家都知道，Consul是一个服务配置、发现的中间件，常用来作为服务注册中心使用。

在基于文件的服务发现中提到，target的更新需要修改target.json，本质上没有解决运维操作的步骤。

而基于Consul的服务发现，则是通过节点主动注册信息到Consul，prometheus通过和Consul集成，从Consul中以http协议获取target的信息。

Consul 是HashiCorp 公司推出的开源工具，产品基于GO 语言开发，主要面向分布式、服务化的系统提供服务注册、服务发现和配置管理的功能。

consul产品具有以下特点:

1. 服务发现

Consul 的客户端可以注册一个服务，例如 api 或 mysql，其他客户端可以使用 Consul 来发现给定服务的提供者。

2. 健康检查

Consul 可以根据给定的信息，对服务的状态进行检查，并获取服务的健康状态。

3. Key/Value存储

通过HTTP API的方式实现Key/Value存储，可用于动态配置、功能标记、协商等多种场景。

4. 多数据中心支持

支持多数据中心的分布式架构。

Prometheus配置

在Prometheus配置Job，这里使用Consul的服务发现方式，并配置好Consul接口地址，用于发现Consul中的node_exporter节点。

```
- job_name: 'consul-prom'
  consul_sd_configs:
    - server: '<consul_ip>:8500'
      services: ['node_exporter']
```

注释： services 用于过滤Consul服务，如果为空，则会获取全部服务信息。

动态服务发现3: 基于eureka的服务发现

如果Spring Cloud使用的是 Eureka 作为注册中心，使用到 Prometheus 做服务监控，

可以基于eureka的服务发现，Eureka服务发现协议允许使用 Eureka Rest API 检索出Prometheus需要监控的targets，

Prometheus会定时周期性的从Eureka调用 Eureka Rest API，并将每个应用实例创建出一个target。

eureka 客户端暴露出 prometheus 端口

修改 application.yml 文件

```
spring:
  application:
    name: order-provider-10004
server:
  port: 10004

management:
  endpoints:
    web:
      exposure:
        include: 'prometheus,metrics,info,health' # 暴露出 prometheus 端口
  metrics:
    tags:
      application: ${spring.application.name} # 增加每个指标的全局的tag，及给每个指标
      一个 application的 tag,值是 spring.application.name的值
  server:
    port: 10005

# 注册到 eureka 上
eureka:
  client:
    service-url:
```

```

defaultZone: http://localhost:10003/eureka/    #连接到服务注册中心的地址，如果服
务注册中心开启了权限需要设置 http://username:password@ip:port/eureka/格式
instance:
  prefer-ip-address: true
  metadata-map:
    # 集成到prometheus,以下的这些数据都会加入到prometheus的重新标记之前的标签中，比如
    sys.module会变成
    "prometheus.scrape": "true"
    "prometheus.path": "/actuator/prometheus"
    "prometheus.port": "${management.server.port}"
    "sys.module": "order"

```

主要是上方 `metadata-map` 中的配置。

prometheus配置文件

简单的集成，修改 prometheus.yml 文件

```

scrape_configs:
  # The job name is added as a label `job=<job_name>` to any timeseries scraped
  from this config.
  - job_name: 'prometheus'

    # metrics_path defaults to '/metrics'
    # scheme defaults to 'http'.

    static_configs:
      - targets: ['localhost:9090']

#原先静态配置的targets，需要重启和改配置文件
# - job_name: 'myapp'
#   metrics_path: '/actuator/prometheus'
#   scrape_interval: 5s
#   static_configs:
#     - targets: ['myapp1:8157', 'myapp2:8157']

- job_name: 'eureka_sd'
  eureka_sd_configs:
    - server: 'http://my_eureka.server:port/eureka'    #注意这里地址后面是eureka
  basic_auth:
    username: 'myuser'
    password: 'mypass'    # 应用对/actuator添加了spring security http basic验证
  relabel_configs:
    - source_labels: [__address__],
      __meta_eureka_app_instance_metadata_prometheus_port]
      action: replace
      regex: ([^:]+)(?::\d+)?;(\d+)
      replacement: $1:$2
      target_label: __address__
  metrics_path: '/actuator/prometheus' #采集地址

```

更多的 relabel_configs 参考案例，如下：

```
- job_name: 'eureka'
  eureka_sd_configs:
    # 指定 eureka 的服务发现地址
    - server: 'http://localhost:10003/eureka'
  relabel_configs:
    # 重写 metrics 的路径
    - source_labels: ["__meta_eureka_app_instance_metadata_prometheus_path"]
      action: replace
      target_label: __metrics_path__
      regex: (.+)
      # 增加一个自定义label sys_model 它的值从配置eureka中获取
    - source_labels: ["__meta_eureka_app_instance_metadata_sys_module"]
      action: replace
      target_label: sys_module
      regex: (.+)
      # 重写管理端口
    - source_labels: [__address__,
      __meta_eureka_app_instance_metadata_prometheus_port]
      action: replace
      regex: ([^:]+)(?::\d+)?;(\d+)
      replacement: $1:$2
      target_label: __address__
```

- 1、重写 metrics 的 path。
- 2、重写 管理端口。
- 3、增加一个自定义的标签。

实现效果查看下方的效果图。

最终实现的抓取端口的路径 `http://10.1.129.254:10005/actuator/prometheus`

动态服务发现4:基于nacos的服务发现

实现方案

- 1、使用某种软件生成配置文件，然后prometheus读取
- 2、使用某种提供注册服务的中间件，然后prometheus访问

第一种，需要docker启动一个容器，定时生成文件。

第二种，需要维护一套中间件，可行。

这里介绍第一种方案。

docker 编排文件

```
prometheus-nacos-sd:
  image: nien/prometheus-nacos-sd:1.0.0
  container_name: prometheus-nacos-sd
  volumes:
    - /tmp:/tmp
  command: "--nacos.address=192.168.56.121:8848 --nacos.namespace=public --output.file=/tmp/nacos_sd_public.json --refresh.interval=600"
  networks:
    base-env-network:
      aliases:
        - prometheus-nacos-sd
```

600的单位是秒

需要给tmp文件夹下的json文件授予其他人读写权限。

源码来自于 [prometheus-sd-nacos: 使用nacos的服务发现功能, 自动获取实例配置监控 \(gitee.com\)](https://gitee.com/nien/prometheus-sd-nacos)

生产的配置文件

```
[root@cdh1 tmp]# ll
total 8
drwxr-xr-x  2 root  root    51 Oct  7 10:17 hsperfdata_root
drwx----- 12 root  root 4096 Sep 28 09:32 _MEICQuN8i
-rw-----  1 65534 65534 1671 Oct  7 12:31 nacos_sd_public.json
drwxr-xr-x  4 65534 65534  28 Oct  7 12:26 prometheus-nacos-sd
drwxr-xr-x  3 root  root   17 Sep 27 17:23 tomcat.1302545122141233788.7777
drwxr-xr-x  3 root  root   17 Oct  6 17:35 tomcat.1617686018198450491.7777
drwxr-xr-x  3 root  root   17 Sep 27 15:10 tomcat.3221381097907404877.7777
drwxr-xr-x  3 root  root   17 Sep 27 17:24 tomcat.4512476252670875333.7788
drwxr-xr-x  3 root  root   17 Oct  7 10:17 tomcat.5137307452570916842.7788
drwxr-xr-x  3 root  root   17 Oct  7 10:17 tomcat.5327890546776942829.7777
drwxr-xr-x  3 root  root   17 Oct  6 17:36 tomcat.617412571170835352.7788
drwxr-xr-x  3 root  root   17 Sep 27 15:10 tomcat.7205873862886476115.7788
drwxr-xr-x  3 root  root   17 Sep 28 08:02 tomcat.7571582908938589164.7777
drwxr-xr-x  3 root  root   17 Sep 28 08:02 tomcat.8805560195831088805.7788
drwxr-xr-x  2 root  root    6 Oct  6 17:35 tomcat-
docbase.2689477778091859199.8848
drwxr-xr-x  2 root  root    6 Oct  6 17:36 tomcat-
docbase.2805418669057586567.7788
drwxr-xr-x  2 root  root    6 Oct  7 10:17 tomcat-
docbase.2883817303932250892.7788
drwxr-xr-x  2 root  root    6 Sep 28 08:02 tomcat-
docbase.3930283248758999137.7777
drwxr-xr-x  2 root  root    6 Sep 28 08:02 tomcat-
docbase.4279845123359990434.7788
```

```
drwxr-xr-x 2 root root 6 Sep 27 17:23 tomcat-  
docbase.4438109191665757249.7777  
drwxr-xr-x 2 root root 6 Sep 27 17:24 tomcat-  
docbase.5440070348330283179.7788  
drwxr-xr-x 2 root root 6 Sep 28 08:02 tomcat-  
docbase.7378412156113599520.8848  
drwxr-xr-x 2 root root 6 Oct 7 10:17 tomcat-  
docbase.7568598900300088529.8848  
drwxr-xr-x 2 root root 6 Oct 7 10:17 tomcat-  
docbase.8710493257860599713.7777  
drwxr-xr-x 2 root root 6 Sep 27 17:23 tomcat-  
docbase.8750551899014460916.8848  
drwxr-xr-x 2 root root 6 Oct 6 17:35 tomcat-  
docbase.9006194788768165054.7777
```

```
[root@cdh1 tmp]# cat nacos_sd_public.json
```

```
[  
  {  
    "targets": [  
      "192.168.56.1:9999"  
    ],  
    "labels": {  
      "__meta_nacos_group": "DEFAULT_GROUP",  
      "__meta_nacos_namespace": "public",  
      "__meta_preserved_register_source": "SPRING_CLOUD",  
      "__metrics_path__": "/actuator/prometheus",  
      "job": "springcloud-gateway"  
    }  
  },  
  {  
    "targets": [  
      "192.168.56.1:18081"  
    ],  
    "labels": {  
      "__meta_management_context_path": "/consumer/actuator",  
      "__meta_management_endpoints_web_base_path": "/actuator",  
      "__meta_nacos_group": "DEFAULT_GROUP",  
      "__meta_nacos_namespace": "public",  
      "__meta_preserved_register_source": "SPRING_CLOUD",  
      "__meta_servlet_context_path": "/consumer",  
      "__meta_user_name": "admin",  
      "__meta_user_password": "admin",  
      "__metrics_path__": "/actuator/prometheus",  
      "job": "service-consumer-demo"  
    }  
  },  
  {  
    "targets": [  
      "192.168.56.1:28088"  
    ],  
    "labels": {  
      "__meta_management_context_path": "/seata-seckill/actuator",  
      "__meta_management_endpoints_web_base_path": "/actuator",  
      "__meta_nacos_group": "DEFAULT_GROUP",  
      "__meta_nacos_namespace": "public",  
      "__meta_preserved_register_source": "SPRING_CLOUD",  
      "__meta_servlet_context_path": "/seata-seckill",  
      "job": "seata-seckill"  
    }  
  }  
]
```

```

        "__meta_user_name": "admin",
        "__meta_user_password": "admin",
        "__metrics_path__": "/actuator/prometheus",
        "job": "seata-seckill"
    }
}
][root@cdh1 tmp]#

```

修改prometheus配置文件

增加目录映射: /tmp/:/tmp/

增加外部文件配置:

```

- job_name: 'public_nacos-discovery'
  file_sd_configs:
  - files: ['/tmp/nacos_sd_public.json']
    refresh_interval: 3m
  relabel_configs:
  - source_labels: ["job"]
    regex: "DEFAULT_GROUP@@trade-chat-netty"
    action: drop

```

修改springboot项目配置文件

springboot 引入 prometheus相关jar包

```

<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>

```

增加一个配置

```

spring.cloud.nacos.discovery.metadata.context_path=${server.servlet.context-path}

```

暴露prometheus的接口; 暴露metrics.tags, 和spring.application.name一致。

```
server:
  port: 8081
spring:
  application:
    name: my-prometheus
management:
  endpoints:
    web:
      exposure:
        include: 'prometheus'
metrics:
  tags:
    application: ${spring.application.name}
```

四、遇到的问题

- 1、生成的json文件访问时，权限不足方法：用root用户授权777 给json文件
- 2、json文件生成过快，prometheus读取配置时，发现机器下线，但是没有预警方法：设置prometheus读取配置时间为5分钟，json文件生成时间为1小时。
- 3、prometheus经常内存占用高，搞挂机器方法：

目前在prometheus创建容器的时候设置存储时间和文件压缩

```
--storage.tsdb.wal-compression --storage.tsdb.retention.time=7d
```

时序数据存储篇：自定义Prometheus存储

Prometheus自带一个本地磁盘时间序列数据库，但也可以选择与远程存储系统集成。

本地存储

Prometheus的本地时间序列数据库在本地存储上以定制的、高效的格式存储数据。

注意，本地存储的一个限制是它没有集群或副本。

因此，在驱动器或节点中断时，它不是任意可伸缩或持久的，应该像任何其他单节点数据库一样进行管理。

建议使用RAID来提高存储可用性，建议使用快照作为备份。

使用适当的架构，可以在本地存储中保留多年的数据。也可以采用外部存储。

TSDB（时间序列数据库，简称时序数据库）

Prometheus具有几个用于配置本地存储的参数。最重要的是：

- --storage.tsdb.path: Prometheus写入数据库的位置，默认是data/
- --storage.tsdb.retention.time: 什么时候删除旧数据，默认是15天
- --storage.tsdb.retention.size: 要保留的最大存储块字节数。最旧的数据将首先被删除。默认为0或禁用。这个标志是实验性的，在未来的版本中可能会改变。支持的单位:B、KB、MB、GB、TB、PB、EB。例如:"512 mb"

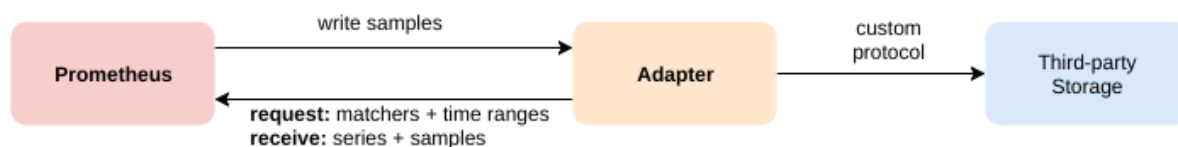
Prometheus平均每个样本仅存储1~2个字节.因此，要规划Prometheus服务器的容量，可以使用以下公式粗略计算：

```
needed_disk_space = retention_time_seconds * ingested_samples_per_second *
bytes_per_sample
```

远程存储系统集成

Prometheus通过以下三种方式与远程存储系统集成：

- Prometheus可以将其提取的样本以标准格式写入远程URL
- Prometheus可以以标准格式从其他Prometheus服务器接收样本
- Prometheus可以以标准格式从远程URL读取样本数据



自定义监控指标：基础知识

自定义监控组件涉及的基础类

Meter

Meter是收集关于你的应用的一系列指标的接口。Meter也是所有度量的抽象，

Meter是由MeterRegistry创建的。

每个支持的监控系统都必须实现MeterRegistry。

Micrometer提供了一组原生Meter类型，包括

- timer（计时器），
- gauge（量规），
- counter（计数器），
- distribution summaries（分布式摘要），
- long task timers（长任务定时器）。

Meter由name和dimension来唯一标识，

在Micrometer中dimension又等价于tag。

name是监控项的名称，tag可以作为监控的指标内容，

比如name：接口请求量；tag：接口url。这样便于分组利于统计分类。

Tag

Tag (标签)是Micrometer的一个重要的功能，

严格来说，一个度量框架只有实现了标签的功能，才能真正地多维度进行度量数据收集。

Tag的命名一般需要有意义的，所谓有意义就是可以根据Tag的命名可以推断出它指向的数据到底代表什么维度或者什么类型的度量指标。

如这样声明一个 Counter

```
registry.counter("test", "test_tag", "tag_info");
```

最后显示出的指标就是

```
test_tag_total{tag_name="tag_info"}
```

Registry

Registry是Meter的注册中心，由Registry创建Meter并统一管理Meter，每个监控系统都有对应的MeterRegistry实现。

几种内置的Registry如下：

SimpleMeterRegistry：极简实现，数据存取在内存中，每项数据只保留最新一次的数值。默认SpringBoot会帮你autowire一个。

CompositeMeterRegistry：用组合模式帮你将多个注册表串联成一个对外接口。全局的Metrics.globalRegistry就是这么个组合模式接口

Micrometer默认提供了基于内存、便于测试的SimpleRegistry，和一个静态全局注册表（Metrics.globalRegistry）。

全局注册表是一个组合注册表（CompositeMeterRegistry），当需要对接多个监控系统时，可以使用组合注册表并向其中添加多个Registry。

Micrometer中包含一个SimpleMeterRegistry，它在内存中维护每个meter的最新值，并且不将数据导出到任何地方。

如果你还没有一个首选的监测系统，你可以先用SimpleMeterRegistry：

```
MeterRegistry registry = new SimpleMeterRegistry();
```

SimpleMeterRegistry在Spring中是自动注入的

Meter Filter

每个注册表都可以配置Meter过滤器，

在工作项目中引入不同的依赖，或者配合不同的工具往往需要传递MeterRegistry或者直接使用全局静态Registry，为了方便开发者收集指定的数据，或者对注册的Meter进行转换修改，

Micrometer提供了Meter过滤器用于编写规则拒绝、转换或配置注册的Meter。

常用Meter类型指标

Counter (计数器)

`Counter` 是一种比较简单的 `Meter`，它是一种单值的度量类型，或者说是一个单值计数器。

`Counter`接口允许使用者使用一个固定值(必须为正数)进行计数。

准确来说：`Counter` 就是一个增量为正数的单值计数器。这里举个很简单的使用例子：

1、使用 `meterRegistry` 创建指标

```
Counter counter = meterRegistry.counter("http.request", "createOrder",
    "/order/create");

counter.increment();
```

2、使用 `Counter` 内部建造器类 `Counter.Builder` 去实例化 `Counter`。

```
Counter counter = Counter.builder("name") //名称
    .baseUnit("unit") //基础单位
    .description("desc") //描述
    .tag("tagKey", "tagValue") //标签
    .register(meterRegistry); //绑定的MeterRegistry

counter.increment();
```

3、`FunctionCounter` 是 `Counter` 的特化类型，

它把计数器数值增加的动作抽象成[接口类型](#) `ToDoubleFunction`，这个接口JDK1.8中对于 `Function` 的特化类型接口。`FunctionCounter` 的使用场景和 `Counter` 是一致的。

`FunctionCounter` 使用的一个明显的好处是，我们不需要感知 `FunctionCounter` 实例的存在，

实际上我们只需要操作作为 `FunctionCounter` 实例构建元素的 `AtomicInteger` 实例即可，

这种接口的设计方式在很多框架里面都可以看到。

```
AtomicInteger n = new AtomicInteger(0);
//这里ToDoubleFunction匿名实现其实可以使用Lambda表达式简化为AtomicInteger::get
FunctionCounter.builder("functionCounter", n, AtomicInteger::get)
    .baseUnit("unit") //基础单位
    .description("desc") //描述
    .tag("tagKey", "tagValue") //标签
    .register(meterRegistry);

n.incrementAndGet(); // 仅对自增api有效
```

使用场景：

`Counter` 的作用是记录XXX的总量或者计数值，适用于一些增长类型的统计，例如下单、支付次数、Http请求总量记录等等，

通过Tag可以区分不同的场景，对于下单，可以使用不同的Tag标记不同的业务来源或者是按日期划分，对于HttpRequest总量记录，可以使用Tag区分不同的URL。

gauge（量规）

Gauge 是获取当前度量记录值的句柄，也就是它表示一个可以任意上下浮动的单数值度量Meter。

Gauge 通常用于变动的测量值，测量值用 ToDoubleFunction 参数的返回值设置，如当前的内存使用情况，同时也可以测量上下移动的“计数”，比如队列中的消息数量。

Gauge 的典型使用场景是用于测量集合或映射的大小或运行状态中的线程数。

Gauge 一般用于监测有自然上界的事件或者任务，而 Counter 一般使用于无自然上界的事件或者任务的监测，

所以像HttpRequest总量计数应该使用 Counter 而非 Gauge。

MeterRegistry 中提供了一些便于构建用于观察数值、函数、集合和映射的 Gauge 相关的方法：

1、使用 meterRegistry 创建指标

```
List<String> list = registry.gauge("listGauge", Collections.emptyList(), new
ArrayList<>(), List::size);

List<String> list2 = registry.gaugeCollectionSize("listSize2", Tags.empty(), new
ArrayList<>());

Map<String, Integer> map = registry.gaugeMapSize("mapGauge", Tags.empty(), new
HashMap<>());
```

上面的三个方法通过 MeterRegistry 构建 Gauge 并且返回了集合或者映射实例，

使用这些集合或者映射实例就能在其 size 变化过程中记录这个变更值。

更重要的优点是，我们不需要感知 Gauge 接口的存在，只需要像平时一样使用集合或者映射实例就可以了。

此外，Gauge 还支持java.lang.Number的子类，

java.util.concurrent.atomic包中的AtomicInteger和AtomicLong，还有Guava提供的AtomicDouble：

```
AtomicInteger n = registry.gauge("numberGauge", new AtomicInteger(0));
n.set(1);
n.set(2);
```

2、建造器流式创建：

```
List<Integer> list = new ArrayList<>();

//一般我们不需要操作Gauge实例
Gauge gauge = Gauge.builder("gauge", list, List::size)
    .baseUnit("unit") //基础单位
    .description("desc") //描述
    .tag("tagKey", "tagValue") //标签
    .register(meterRegistry);
```

使用场景：

- 1、有自然(物理)上界的浮动值的监测，例如物理内存、集合、映射、数值等。
- 2、有逻辑上界的浮动值的监测，例如积压的消息、(线程池中)积压的任务等，其实本质也是集合或者映射的监测。

timers (计时器)

Time 适用于记录耗时比较短的事件的执行时间，通过时间分布展示事件的序列和发生频率。

所有的 **Timer** 的实现至少记录了发生事件的数量和这些事件的总耗时，从而生成一个时间序列。

Timer 的基本单位基于服务端的指标而定，但是实际上我们不需要过于关注 **Timer** 的基本单位，

因为 **Micrometer** 在存储生成的时间序列的时候会自动选择适当的基本单位。

Timer 接口提供的常用方法如下：

- 1、使用 **meterRegistry** 创建指标

```
Timer print1000 = meterRegistry.timer("print1000");
print1000.record()->{
    // 传入任务，并自行执行
    for (int i = 0; i < 1000; i++) {
        System.out.println(i);
    }
};
```

```
Timer print1000 = meterRegistry.timer("print1000");
// 只定义了指标以及任务，不自行执行
Runnable wrap = print1000.wrap() -> {
    for (int i = 0; i < 1000; i++) {
        System.out.println(i);
    }
};
// 执行后指标才有值
wrap.run();
```

- 2、建造器流式创建：

```

Timer print1000 = Timer.builder("print1000")
    .description("desc") //描述
    .tag("tagKey", "tagValue") //标签
    .register(meterRegistry);

Runnable wrap = print1000.wrap(() -> {
    for (int i = 0; i < 1000; i++) {
        System.out.println(i);
    }
});
wrap.run();

```

3、`Timer` 的使用还可以基于它的内部类 `Timer.Sample`

通过 `start` 和 `stop` 两个方法记录两者之间的逻辑的执行耗时。

```

Timer.Sample sample = Timer.start(meterRegistry);
for (int i = 0; i < 1000; i++) {
    System.out.println(i);
}
sample.stop(meterRegistry.timer("print1000"));

```

4、`FunctionTimer` 是 `Timer` 的特化类型，它主要提供两个单调递增的函数(其实并不是单调递增，只是在使用中一般需要随着时间最少保持不变或者说不减少，一个用于计数的函数和一个用于记录总调用耗时的函数。

官方文档中的例子如下：

```

IMap<?, ?> cache = ...; // 假设使用了Hazelcast缓存
registry.more().timer("cache.gets.latency", Tags.of("name", cache.getName()),
    cache,
    c -> c.getLocalMapStats().getGetOperationCount(), //实际上就是cache的一个方法，
    记录缓存生命周期初始化的增量(个数)
    c -> c.getLocalMapStats().getTotalGetLatency(), // Get操作的延迟时间总量，可以
    理解为耗时
    TimeUnit.NANOSECONDS
);

```

`countFunction` 用于统计事件个数，

`totalTimeFunction` 用于记录执行总时间，

实际上两个函数都只是 `Function` 函数的变体，

还有一个比较重要的是总时间的单位 `totalTimeFunctionUnit`。

简单的使用方式如下：

```

//这个是为了满足参数，暂时不需要理会
Object holder = new Object();
AtomicLong totalTimeNanos = new AtomicLong(0);
AtomicLong totalCount = new AtomicLong(0);

```

```

FunctionTimer.builder("functionTimer",
    holder,
    p -> totalCount.get(), // 统计事件个数
    p -> totalTimeNanos.get(), // 记录执行总时间
    TimeUnit.NANOSECONDS) // 总时间的单位
    .description("desc") //描述
    .tag("tagKey", "tagValue") //标签
    .register(meterRegistry);
totalTimeNanos.addAndGet(10000000);
totalCount.incrementAndGet();

```

使用场景：

- 1、记录指定方法的执行时间用于展示。
- 2、记录一些任务的执行时间，从而确定某些数据来源的速率，例如消息队列消息的消费速率等。

long task timer（长任务定时器）

`LongTaskTimer` 也是一种 `Timer` 的特化类型，

主要用于记录长时间执行的任务的持续时间，在任务完成之前，被监测的事件或者任务仍然处于运行状态，任务完成的时候，任务执行的总耗时才会被记录下来，而无返回的`Timer`记录的是任务启动间隔时间。

在Spring应用中，可以简单地使用 `@Scheduled` 和 `@Timed` 注解，基于spring-aop完成定时调度任务的总耗时记录：

- 1、`@Timed`注解使用

```

@Timed(value = "aws.scrape", longTask = true)
@Scheduled(fixedDelay = 360000)
void scrapeResources() {
    //这里做相对耗时的业务逻辑
}

```

- 2、使用 `meterRegistry` 创建指标

```

LongTaskTimer print1000 = meterRegistry.more().longTaskTimer("print1000");
print1000.record()->{
    // 传入任务，并自行执行
    for (int i = 0; i < 1000; i++) {
        System.out.println(i);
    }
});

```

- 3、建造器流式创建：

```

LongTaskTimer print1000 = LongTaskTimer.builder("print1000")
    .description("desc") //描述
    .tag("tagKey", "tagValue") //标签
    .register(meterRegistry);
print1000.record()->{
    // 传入任务，并自行执行
    for (int i = 0; i < 1000; i++) {
        System.out.println(i);
    }
};

```

使用场景：

`LongTaskTimer` 适合用于长时间持续运行的事件耗时的记录，例如相对耗时的定时任务。

distribution summary（分布式摘要）

`Summary` 主要用于跟踪事件的分布，

在 `Micrometer` 中，对应的类是 `Distribution Summary`（分发摘要）。

它的使用方式和 `Timer` 十分相似，但是它的记录值并不依赖于时间单位。

常见的使用场景：使用 `Distribution Summary` 测量命中服务器的请求的有效负载大小。

1、使用 `meterRegistry` 创建指标

```

DistributionSummary summer = meterRegistry.summary("summer");
summer.record(0.25);

```

2、建造器流式创建：

```

DistributionSummary register = DistributionSummary.builder("summer")
    .description("desc") //描述
    .tag("tagKey", "tagValue") //标签
    .register(meterRegistry);
register.record(0.25);

```

使用场景

不依赖于时间单位的记录值的测量，例如服务器有效负载值，缓存的命中率等。

常用标签

常用标签可以对所有指标进行添加通用标签

```

management.metrics.tags.application=${spring.application.name}
management.metrics.tags.tagKey=tagValue

```

禁用指标

下面以http.server.requests示例禁用

```
management.metrics.enable.http.server.requests=false
```

自定义监控指标：实操开发

需求

现在需求是这样的，将时间轴等距离均分，如间隔是一分钟，然后去监控每一分钟内 缓存命中率。

首先counter是不能使用的，因为counter的设定是只增不减，

然后summary和histogram在我的理解中是对分布的统计和监控，所以也不能使用，

因此只好使用了gauge（投机取巧说的是：gauge的意义更在于是统计一个实时的值，反应一种动态变化）。

gauge的定义包括名字以及一个Number（当然还有其他的注册方式，如标签等，需要可以自行研究）

加入maven依赖

```
<!--监控加入 prometheus 收集数据-->

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-actuator</artifactId>
</dependency>
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

application.yml

```
#### 暴露端点
management:
  endpoints:
    web:
      base-path: "/actuator" # 配置 Endpoint 的基础路径
      exposure:
        include: '*' #在yaml 文件属于关键字，所以需要加引号
  health:
```

```

defaults:
  enabled: true
db:
  enabled: false
  show-details: always
metrics:
  tags:
    application: ${spring.application.name}
# boot:
#   admin:
#     context-path: consumer

```

配置文件，这里特别说明，要去掉db健康状态检测，感觉是版本不对，导致包依赖冲突，所以要去掉db,就正常，还有其它禁用的方式，如果不需要都可以 禁掉

```

management.health.mail.enabled
management.health.mongo.enabled
management.health.redis.enabled
management.health.solr.enabled
management.health.elasticsearch.enabled

```

这里基于安全考虑，可以对actuator添加了额外的端口，和服务端口不一样。

```

management:
  endpoints:
    enabled-by-default: true
  web:
    exposure:
      include: ['pause','prometheus']
  endpoint:
    pause:
      enabled: true
    prometheus:
      enabled: true
  server:
    port: 8293
  servlet:
    context-path: /api    # 给actuator也设置context-path 这个配置在springboot 2.4
0 之后，改为base-path， 相应eureka配置也要修改

```

因此，在本地eureka的配置中添加一个 `metadataMap.prometheus.port` 项。

```
eureka:
  instance:
    prefer-ip-address: true
    lease-renewal-interval-in-seconds: 10
    metadataMap.prometheus.port: 8293
```

这个会保存到eureka的metadata，然后被prometheus eureka sd解析到。

Actuator提供的所有监控

启动项目，

访问以下地址，

<http://localhost:7711/stock-provider/actuator>

就可以看到Actuator提供的所有监控

```
{
  "_links": {
    "self": {
      "href": "http://cdh1:8080/stock-provider/actuator",
      "templated": false
    },
    "prometheus": {
      "href": "http://cdh1:8080/stock-provider/actuator/prometheus",
      "templated": false
    },
    "metrics-requiredMetricName": {
      "href": "http://cdh1:8080/stock-provider/actuator/metrics/{requiredMetricName}",
      "templated": true
    },
    "metrics": {
      "href": "http://cdh1:8080/stock-provider/actuator/metrics",
      "templated": false
    }
  }
}
```

访问以下地址，

<http://localhost:7711/stock-provider/actuator/metrics>

可以看到Actuator默认收集的监控指标，

包括JVM相关指标(内存使用，垃圾收集)，tomcat相关指标，系统相关指标，自定义的指标

```
{
  "names": [
    "jvm.memory.committed",
    "http.server.requests",
    "tomcat.sessions.expired",
    "jvm.memory.used",
    "jvm.threads.live",
    "jvm.threads.peak",
    "process.uptime",
    "logback.events",
    "tomcat.sessions.rejected",
    "tomcat.global.sent",
    "process.cpu.usage",
    "jvm.buffer.memory.used",
    "jvm.classes.loaded",
    "jvm.threads.states",
    "tomcat.global.request.max",
    "jvm.gc.pause",
    "tomcat.global.request",
    "jvm.classes.unloaded",
    "tomcat.sessions.active.current",
    "jvm.memory.max",
    "tomcat.sessions.alive.max",
    "jvm.gc.live.data.size",
    "tomcat.global.received",
    "tomcat.sessions.active.max",
    "tomcat.threads.config.max",
    "process.start.time",
    "tomcat.global.error",
    "jvm.gc.memory.promoted",
    "tomcat.threads.current",
    "tomcat.cache.hit",
    "cache_hit_sum",
    "cache_mis_sum",
    "jvm.buffer.count",
    "tomcat.cache.access",
    "jvm.gc.max.data.size",
    "jvm.buffer.total.capacity",
    "system.cpu.count",
    "tomcat.threads.busy",
    "tomcat.sessions.created",
    "jvm.threads.daemon",
    "system.cpu.usage",
    "jvm.gc.memory.allocated"
  ]
}
```

我们可以通过以下链接来查看具体某个指标

```
http://localhost:7711/stock-provider/actuator/metrics/metricName
```

其中metricName为需要查看指标的名称，例如查看jvm内存

```
http://localhost:7711/stock-provider/actuator/metrics/jvm.memory.used
```

从图中可以看到jvm.memory.used有两个tag，area和id，area指定内存位置(堆内存和非堆内存)，id指定内存分类，我们可以指定tag来查看更细致的指标

```
http://localhost:7711/stock-provider/actuator/metrics/jvm.memory.used?tag=area:heap
```

```
http://localhost:8080/actuator/metrics/jvm.memory.used?tag=area:heap&tag=id:PS%20Eden%20Space
```

```
{
  "name": "jvm.memory.used",
  "description": "The amount of used memory",
  "baseUnit": "bytes",
  "measurements": [
    {
      "statistic": "VALUE",
      "value": 3.702654E8
    }
  ],
  "availableTags": [
    {
      "tag": "application",
      "values": [
        "stock-provider"
      ]
    },
    {
      "tag": "id",
      "values": [
        "PS Eden Space",
        "PS Survivor Space",
        "PS Old Gen"
      ]
    }
  ]
}
```

指标转换成prometheus格式的

Micrometer支持Prometheus，Micrometer提供PrometheusMeterRegistry注册表，用于将指标转为Prometheus格式的指标。

```
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-prometheus</artifactId>
</dependency>
```

其次在配置文件中，配置暴露Prometheus，并允许将指标导入到Prometheus中

```
management.endpoint.prometheus.enabled=true
management.metrics.export.prometheus.enabled=true
```

yaml 格式为

```
#### 暴露端点
management:
  endpoints:
    web:
      base-path: "/actuator" # 配置 Endpoint 的基础路径
      exposure:
        include: '*' #在yaml 文件属于关键字，所以需要加引号
    health:
      defaults:
        enabled: true
      db:
        enabled: false
      show-details: always
    metrics:
      export:
        prometheus:
          enabled: true
      tags:
        application: ${spring.application.name}
    endpoint:
      prometheus:
        enabled: true
```

项目启动后,

访问以下地址,

<http://localhost:7711/stock-provider/actuator/prometheus>

可以看到指标以变成Prometheus格式的指标。

```
# HELP jvm_threads_daemon_threads The current number of live daemon threads
# TYPE jvm_threads_daemon_threads gauge
jvm_threads_daemon_threads{application="stock-provider",} 74.0
# HELP jvm_gc_max_data_size_bytes Max size of old generation memory pool
# TYPE jvm_gc_max_data_size_bytes gauge
jvm_gc_max_data_size_bytes{application="stock-provider",} 2.822766592E9
# HELP process_uptime_seconds The uptime of the Java virtual machine
# TYPE process_uptime_seconds gauge
process_uptime_seconds{application="stock-provider",} 32.147
# HELP jvm_threads_states_threads The current number of threads having NEW state
# TYPE jvm_threads_states_threads gauge
jvm_threads_states_threads{application="stock-provider",state="terminated",} 0.0
jvm_threads_states_threads{application="stock-provider",state="new",} 0.0
jvm_threads_states_threads{application="stock-provider",state="runnable",} 21.0
jvm_threads_states_threads{application="stock-provider",state="blocked",} 0.0
jvm_threads_states_threads{application="stock-provider",state="waiting",} 61.0
jvm_threads_states_threads{application="stock-provider",state="timed-waiting",}
51.0
```

```

# HELP process_cpu_usage The "recent cpu usage" for the Java Virtual Machine
process
# TYPE process_cpu_usage gauge
process_cpu_usage{application="stock-provider",} 0.06918860037550037
# HELP jvm_buffer_memory_used_bytes An estimate of the memory that the Java
virtual machine is using for this buffer pool
# TYPE jvm_buffer_memory_used_bytes gauge
jvm_buffer_memory_used_bytes{application="stock-provider",id="direct",} 16470.0
jvm_buffer_memory_used_bytes{application="stock-provider",id="mapped",} 0.0
# HELP cache_mis_Sum
# TYPE cache_mis_Sum summary
cache_mis_Sum_count{application="stock-provider",mis="cache",} 0.0
cache_mis_Sum_sum{application="stock-provider",mis="cache",} 0.0
# HELP cache_mis_Sum_max
# TYPE cache_mis_Sum_max gauge
cache_mis_Sum_max{application="stock-provider",mis="cache",} 0.0
# HELP tomcat_sessions_active_current_sessions
# TYPE tomcat_sessions_active_current_sessions gauge
tomcat_sessions_active_current_sessions{application="stock-provider",} 0.0
# HELP tomcat_sessions_expired_sessions_total
# TYPE tomcat_sessions_expired_sessions_total counter
tomcat_sessions_expired_sessions_total{application="stock-provider",} 0.0
# HELP jvm_memory_committed_bytes The amount of memory in bytes that is
committed for the Java virtual machine to use
# TYPE jvm_memory_committed_bytes gauge
jvm_memory_committed_bytes{application="stock-
provider",area="nonheap",id="Compressed Class Space",} 1.441792E7
jvm_memory_committed_bytes{application="stock-provider",area="heap",id="PS Old
Gen",} 2.57949696E8
jvm_memory_committed_bytes{application="stock-provider",area="heap",id="PS
Survivor Space",} 2.9360128E7
jvm_memory_committed_bytes{application="stock-
provider",area="nonheap",id="Metaspace",} 1.05906176E8
jvm_memory_committed_bytes{application="stock-provider",area="heap",id="PS Eden
Space",} 4.63470592E8
jvm_memory_committed_bytes{application="stock-provider",area="nonheap",id="Code
Cache",} 1.9595264E7
# HELP cache_hit_Sum
# TYPE cache_hit_Sum summary
cache_hit_Sum_count{application="stock-provider",hit="cache",} 0.0
cache_hit_Sum_sum{application="stock-provider",hit="cache",} 0.0
# HELP cache_hit_Sum_max
# TYPE cache_hit_Sum_max gauge
cache_hit_Sum_max{application="stock-provider",hit="cache",} 0.0
# HELP tomcat_cache_hit_total
# TYPE tomcat_cache_hit_total counter
tomcat_cache_hit_total{application="stock-provider",} 0.0
# HELP tomcat_sessions_rejected_sessions_total
# TYPE tomcat_sessions_rejected_sessions_total counter
tomcat_sessions_rejected_sessions_total{application="stock-provider",} 0.0
# HELP process_start_time_seconds Start time of the process since unix epoch.
# TYPE process_start_time_seconds gauge
process_start_time_seconds{application="stock-provider",} 1.674565556152E9
# HELP tomcat_sessions_created_sessions_total
# TYPE tomcat_sessions_created_sessions_total counter
tomcat_sessions_created_sessions_total{application="stock-provider",} 0.0
# HELP jvm_gc_memory_promoted_bytes_total Count of positive increases in the
size of the old generation memory pool before GC to after GC

```

```

# TYPE jvm_gc_memory_promoted_bytes_total counter
jvm_gc_memory_promoted_bytes_total{application="stock-provider",} 3.265172E7
# HELP jvm_gc_memory_allocated_bytes_total Incremented for an increase in the
size of the young generation memory pool after one GC to before the next
# TYPE jvm_gc_memory_allocated_bytes_total counter
jvm_gc_memory_allocated_bytes_total{application="stock-provider",} 1.199500592E9
# HELP tomcat_threads_config_max_threads
# TYPE tomcat_threads_config_max_threads gauge
tomcat_threads_config_max_threads{application="stock-provider",name="http-nio-
7711",} 200.0
# HELP tomcat_global_received_bytes_total
# TYPE tomcat_global_received_bytes_total counter
tomcat_global_received_bytes_total{application="stock-provider",name="http-nio-
7711",} 0.0
# HELP tomcat_global_error_total
# TYPE tomcat_global_error_total counter
tomcat_global_error_total{application="stock-provider",name="http-nio-7711",}
0.0
# HELP system_cpu_count The number of processors available to the Java virtual
machine
# TYPE system_cpu_count gauge
system_cpu_count{application="stock-provider",} 16.0
# HELP tomcat_sessions_alive_max_seconds
# TYPE tomcat_sessions_alive_max_seconds gauge
tomcat_sessions_alive_max_seconds{application="stock-provider",} 0.0
# HELP jvm_memory_used_bytes The amount of used memory
# TYPE jvm_memory_used_bytes gauge
jvm_memory_used_bytes{application="stock-provider",area="nonheap",id="Compressed
Class Space",} 1.359048E7
jvm_memory_used_bytes{application="stock-provider",area="heap",id="PS Old Gen",}
5.9266312E7
jvm_memory_used_bytes{application="stock-provider",area="heap",id="PS Survivor
Space",} 0.0
jvm_memory_used_bytes{application="stock-
provider",area="nonheap",id="Metaspace",} 1.0131304E8
jvm_memory_used_bytes{application="stock-provider",area="heap",id="PS Eden
Space",} 2.38618104E8
jvm_memory_used_bytes{application="stock-provider",area="nonheap",id="Code
Cache",} 1.955936E7
# HELP tomcat_threads_busy_threads
# TYPE tomcat_threads_busy_threads gauge
tomcat_threads_busy_threads{application="stock-provider",name="http-nio-7711",}
1.0
# HELP tomcat_global_request_seconds
# TYPE tomcat_global_request_seconds summary
tomcat_global_request_seconds_count{application="stock-provider",name="http-nio-
7711",} 0.0
tomcat_global_request_seconds_sum{application="stock-provider",name="http-nio-
7711",} 0.0
# HELP jvm_threads_peak_threads The peak live thread count since the Java
virtual machine started or peak was reset
# TYPE jvm_threads_peak_threads gauge
jvm_threads_peak_threads{application="stock-provider",} 133.0
# HELP system_cpu_usage The "recent cpu usage" for the whole system
# TYPE system_cpu_usage gauge
system_cpu_usage{application="stock-provider",} 0.0
# HELP tomcat_cache_access_total
# TYPE tomcat_cache_access_total counter

```

```

tomcat_cache_access_total{application="stock-provider",} 0.0
# HELP jvm_classes_unloaded_classes_total The total number of classes unloaded
since the Java virtual machine has started execution
# TYPE jvm_classes_unloaded_classes_total counter
jvm_classes_unloaded_classes_total{application="stock-provider",} 7.0
# HELP tomcat_global_sent_bytes_total
# TYPE tomcat_global_sent_bytes_total counter
tomcat_global_sent_bytes_total{application="stock-provider",name="http-nio-
7711",} 0.0
# HELP jvm_buffer_total_capacity_bytes An estimate of the total capacity of the
buffers in this pool
# TYPE jvm_buffer_total_capacity_bytes gauge
jvm_buffer_total_capacity_bytes{application="stock-provider",id="direct",}
16469.0
jvm_buffer_total_capacity_bytes{application="stock-provider",id="mapped",} 0.0
# HELP tomcat_sessions_active_max_sessions
# TYPE tomcat_sessions_active_max_sessions gauge
tomcat_sessions_active_max_sessions{application="stock-provider",} 0.0
# HELP jvm_memory_max_bytes The maximum amount of memory in bytes that can be
used for memory management
# TYPE jvm_memory_max_bytes gauge
jvm_memory_max_bytes{application="stock-provider",area="nonheap",id="Compressed
Class Space",} 1.073741824E9
jvm_memory_max_bytes{application="stock-provider",area="heap",id="PS Old Gen",}
2.822766592E9
jvm_memory_max_bytes{application="stock-provider",area="heap",id="PS Survivor
Space",} 2.9360128E7
jvm_memory_max_bytes{application="stock-
provider",area="nonheap",id="Metaspace",} -1.0
jvm_memory_max_bytes{application="stock-provider",area="heap",id="PS Eden
Space",} 1.353711616E9
jvm_memory_max_bytes{application="stock-provider",area="nonheap",id="Code
Cache",} 2.5165824E8
# HELP jvm_classes_loaded_classes The number of classes that are currently
loaded in the Java virtual machine
# TYPE jvm_classes_loaded_classes gauge
jvm_classes_loaded_classes{application="stock-provider",} 19777.0
# HELP tomcat_threads_current_threads
# TYPE tomcat_threads_current_threads gauge
tomcat_threads_current_threads{application="stock-provider",name="http-nio-
7711",} 20.0
# HELP jvm_gc_live_data_size_bytes Size of old generation memory pool after a
full GC
# TYPE jvm_gc_live_data_size_bytes gauge
jvm_gc_live_data_size_bytes{application="stock-provider",} 5.9266312E7
# HELP jvm_threads_live_threads The current number of live threads including
both daemon and non-daemon threads
# TYPE jvm_threads_live_threads gauge
jvm_threads_live_threads{application="stock-provider",} 133.0
# HELP logback_events_total Number of error level events that made it to the
logs
# TYPE logback_events_total counter
logback_events_total{application="stock-provider",level="debug",} 0.0
logback_events_total{application="stock-provider",level="error",} 1.0
logback_events_total{application="stock-provider",level="info",} 90.0
logback_events_total{application="stock-provider",level="warn",} 2.0
logback_events_total{application="stock-provider",level="trace",} 0.0
# HELP jvm_gc_pause_seconds Time spent in GC pause

```

```
# TYPE jvm_gc_pause_seconds summary
jvm_gc_pause_seconds_count{action="end of minor GC",application="stock-provider",cause="Allocation Failure",} 2.0
jvm_gc_pause_seconds_sum{action="end of minor GC",application="stock-provider",cause="Allocation Failure",} 0.033
jvm_gc_pause_seconds_count{action="end of major GC",application="stock-provider",cause="Metadata GC Threshold",} 2.0
jvm_gc_pause_seconds_sum{action="end of major GC",application="stock-provider",cause="Metadata GC Threshold",} 0.232
jvm_gc_pause_seconds_count{action="end of minor GC",application="stock-provider",cause="Metadata GC Threshold",} 2.0
jvm_gc_pause_seconds_sum{action="end of minor GC",application="stock-provider",cause="Metadata GC Threshold",} 0.022
# HELP jvm_gc_pause_seconds_max Time spent in GC pause
# TYPE jvm_gc_pause_seconds_max gauge
jvm_gc_pause_seconds_max{action="end of minor GC",application="stock-provider",cause="Allocation Failure",} 0.018
jvm_gc_pause_seconds_max{action="end of major GC",application="stock-provider",cause="Metadata GC Threshold",} 0.123
jvm_gc_pause_seconds_max{action="end of minor GC",application="stock-provider",cause="Metadata GC Threshold",} 0.012
# HELP tomcat_global_request_max_seconds
# TYPE tomcat_global_request_max_seconds gauge
tomcat_global_request_max_seconds{application="stock-provider",name="http-nio-7711",} 0.0
# HELP jvm_buffer_count_buffers An estimate of the number of buffers in the pool
# TYPE jvm_buffer_count_buffers gauge
jvm_buffer_count_buffers{application="stock-provider",id="direct",} 7.0
jvm_buffer_count_buffers{application="stock-provider",id="mapped",} 0.0
```

prometheus.yml

单独配置 springboot 应用

```
scrape_configs:
  - job_name: 'springboot-prometheus'
    metrics_path: '/actuator/prometheus'
    static_configs:
      - targets: ['192.168.100.93:8080','192.168.100.16:8080']
```

集成 eureka 配置所有的 springboot 应用

```
- job_name: 'eureka_sd'
  eureka_sd_configs:
    - server: 'http://192.168.56.121:7777/eureka' #注意这里地址后面是eureka
  # basic_auth:
  #   username: 'myuser'
  #   password: 'mypass' # 应用对/actuator添加了spring security http basic验证
  relabel_configs:
    - source_labels: [__address__],
      __meta_eureka_app_instance_metadata_prometheus_port
      action: replace
      regex: ([^:]+)(?::\d+)?;(\d+)
      replacement: $1:$2
```

```
target_label: __address__
metrics_path: '/actuator/prometheus' #采集地址
```

relabel_configs

这里不得不提的是relabel_configs

Relabeling (重新标记) 是一种强大的工具, 可以在抓取目标之前动态重写目标的标签集。

每个抓取配置可以配置多个重新标记步骤。它们按照在配置文件中出现的顺序应用于每个目标的标签集。

Relabeling是在抓取(scraping)前修改target和它的labels

<eureka_sd_config>

Eureka SD configurations allow retrieving scrape targets using the [Eureka](#) REST API. Prometheus will periodically check the REST endpoint and create a target for every app instance.

The following meta labels are available on targets during [relabeling](#):

- `__meta_eureka_app_name`: the name of the app
- `__meta_eureka_app_instance_id`: the ID of the app instance
- `__meta_eureka_app_instance_hostname`: the hostname of the instance
- `__meta_eureka_app_instance_homepage_url`: the homepage url of the app instance
- `__meta_eureka_app_instance_statuspage_url`: the status page url of the app instance
- `__meta_eureka_app_instance_healthcheck_url`: the health check url of the app instance
- `__meta_eureka_app_instance_ip_addr`: the IP address of the app instance
- `__meta_eureka_app_instance_vip_address`: the VIP address of the app instance
- `__meta_eureka_app_instance_secure_vip_address`: the secure VIP address of the app instance
- `__meta_eureka_app_instance_status`: the status of the app instance
- `__meta_eureka_app_instance_port`: the port of the app instance
- `__meta_eureka_app_instance_port_enabled`: the port enabled of the app instance
- `__meta_eureka_app_instance_secure_port`: the secure port address of the app instance
- `__meta_eureka_app_instance_secure_port_enabled`: the secure port of the app instance
- `__meta_eureka_app_instance_country_id`: the country ID of the app instance
- `__meta_eureka_app_instance_metadata_<metadataname>`: app instance metadata
- `__meta_eureka_app_instance_datacenterinfo_name`: the datacenter name of the app instance
- `__meta_eureka_app_instance_datacenterinfo_<metadataname>`: the datacenter metadata

一键启动 prometheus

```
[root@cdh1 ~]# cd /home/docker-compose/prometheus
[root@cdh1 ~]# rm -rf /home/docker-compose/prometheus
[root@cdh1 ~]# cd /
[root@cdh1 ~]# cp -rf /vagrant/chapter26/prometheus /home/docker-compose
[root@cdh1 ~]#
[root@cdh1 ~]# chmod 777 -R /home/docker-compose/prometheus
[root@cdh1 ~]#
[root@cdh1 ~]# cd /home/docker-compose/prometheus
[root@cdh1 ~]#
[root@cdh1 ~]# ls -l
total 40
-rwxrwxrwx 1 root root 2293 Jan 25 09:36 docker-compose-prometheus.yml
-rwxrwxrwx 1 root root 2182 Jan 25 09:36 docker-compose.yml
-rwxrwxrwx 1 root root 2215 Jan 25 09:36 docker-compose.yml.bak
drwxrwxrwx 7 root root 79 Jan 25 09:36 grafana
-rwxrwxrwx 1 root root 867 Jan 25 09:36 prometheus.yml
-rwxrwxrwx 1 root root 1065 Jan 25 09:36 prometheus.yml.bak
-rwxrwxrwx 1 root root 815 Jan 25 09:36 prometheus - 副本 (2).yml
-rwxrwxrwx 1 root root 1065 Jan 25 09:36 prometheus - 副本 (3).yml
-rwxrwxrwx 1 root root 1065 Jan 25 09:36 prometheus - 副本.yml
-rwxrwxrwx 1 root root 2170 Jan 25 09:36 一键install.txt
-rwxrwxrwx 1 root root 2132 Jan 25 09:36 一键install.txt.bak
[root@cdh1 ~]#
[root@cdh1 ~]# docker-compose --compatibility up -d
```

访问：

<http://cdh1:9090/classic/targets>

如下图，可以看到Prometheus已经发现了服务的接口，但是没有加上 context上下文，所以 地址有误

不安全 | cdh1:9090/classic/targets#job-eureka_sd

Prometheus Alerts Graph Status Help New UI

Targets

AllUnhealthyCollapse All

eureka_sd (0/2 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://192.168.56.121:7788/actuator/prometheus	DOWN	instance="192.168.56.121:7788" job="eureka_sd"	8.693s ago	64.44ms	"INVALID" is not a valid start token
http://192.168.56.1:7711/actuator/prometheus	DOWN	instance="192.168.56.1:7711" job="eureka_sd"	5.319s ago	2.716ms	server returned HTTP status 404

prometheus (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://prometheus:9090/metrics	UP	instance="prometheus" job="prometheus"	4.863s ago	1.621ms	

CSDN @40岁资深老架构师尼恩

application.yml 有个地方要注意，原来我们没有加上下文路径（server.servlet.context-path），但是一般项目是会设置的，所以这次我们也加上。

为了能够根据各服务动态自定义指标路径（metrics_path），最最重要的是下面这三行

```
eureka:
  instance:
    metadata-map:
      "prometheus.scrape": "true"
      "prometheus.path": "${server.servlet.context-path}/actuator/prometheus"
      "prometheus.port": "${server.port}"
```

prometheus是通过eureka发现服务的，因此只有将服务的指标路径（抓取地址）写到eureka里，prometheus才能拿到
换言之，只有服务在注册的时候，将自己暴露的端点（endpoint）以元数据的方式写到eureka中prometheus才能正确的从目标抓取数据

可以正常采集7771应用数据了。其中两个down 状态的应用其他服务，不用管。

← ↻ ⚠ 不安全 | cdh1:9090/classic/targets#job-eureka_sd

Prometheus Alerts Graph Status ▾ Help New UI

Targets

All Unhealthy Collapse All

eureka_sd (1/2 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://192.168.56.121:7788/metrics	DOWN	instance="192.168.56.121:7788" job="eureka_sd"	9.029s ago	17.77ms	server returned HTTP status 404
http://192.168.56.1:7711/stock-provider/actual/prometheus	UP	instance="192.168.56.1:7711" job="eureka_sd"	3.072s ago	532.3ms	

prometheus (1/1 up) show less

Endpoint	State	Labels	Last Scrape	Scrape Duration	Error
http://prometheus:9090/metrics	UP	instance="prometheus" job="prometheus"	5.345s ago	3.547ms	

GSDN @40岁资深老架构师尼恩

Grafana Dashboard 与 Panel

Dashboard（面板）与 Panel（图表）是 Prometheus 的基本单元，

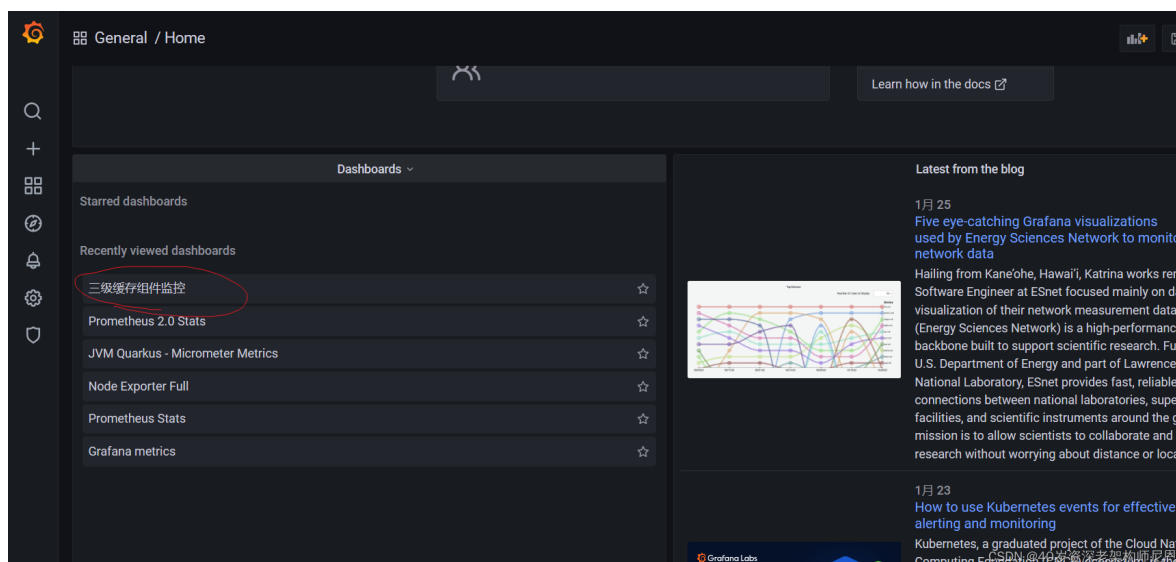
我们所有的监控数据都会通过这两个东西来呈现。

Dashboard 就是一个大的面板，里面包含了无数个图表（Panel）。

例如：我们希望 三级缓存组件监控 的各项业务指标，

那么我们可以建立一个名为「三级缓存组件监控」的面板，然后在这个面板下建立许多个图表。

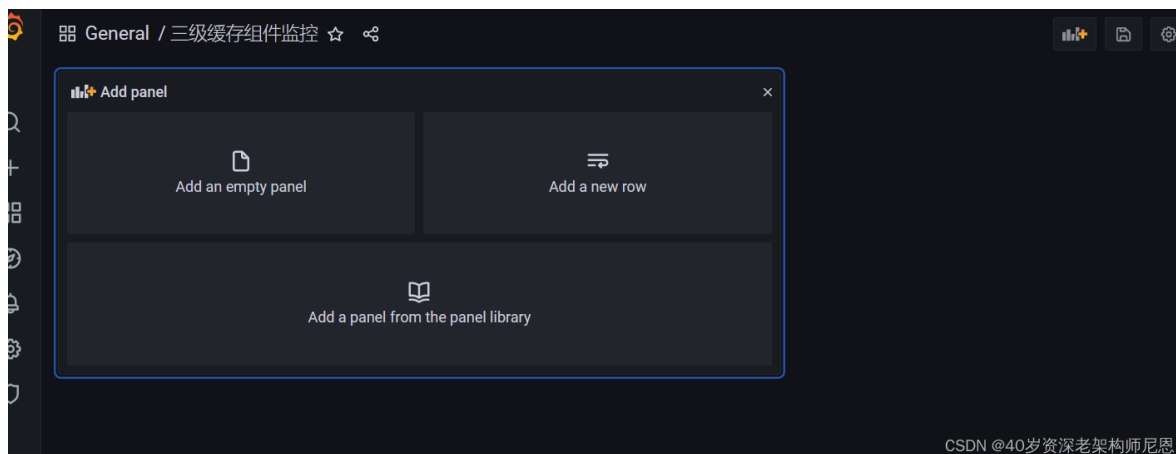
建立Dashboard之后，可以在Dashboard 清单里边，看到这个 Dashboard



Panel 图表添加

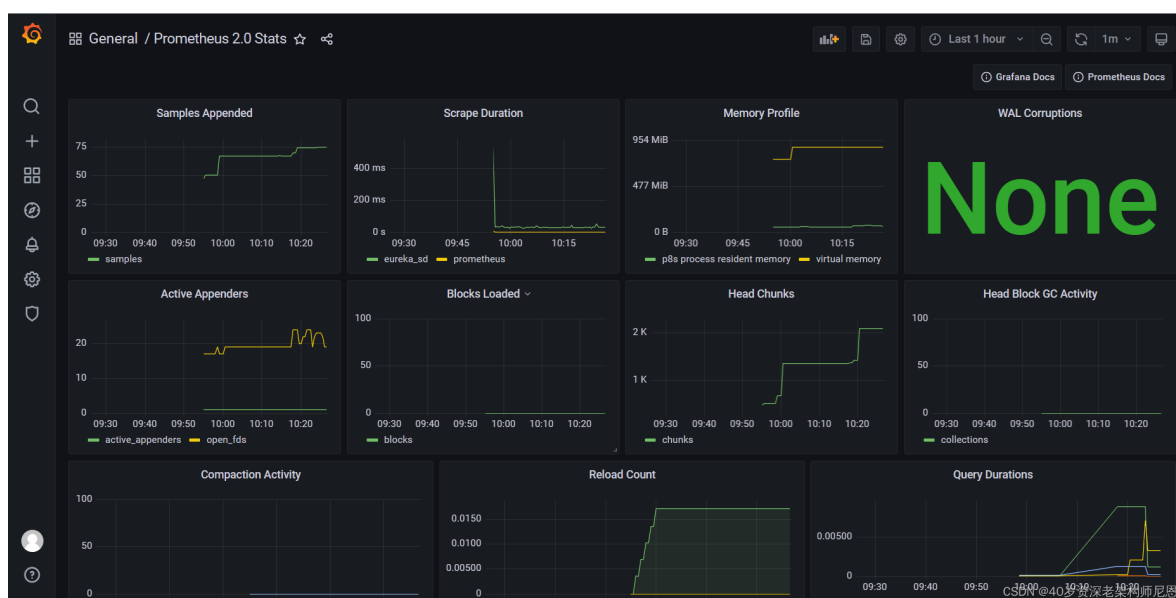
Panel（图表）是 Prometheus 中数据呈现的最小单元，我们看到的面板数据，都是由一个个图表构成的。

点击添加Panel（图表）的按钮，可以添加Panel（图表）



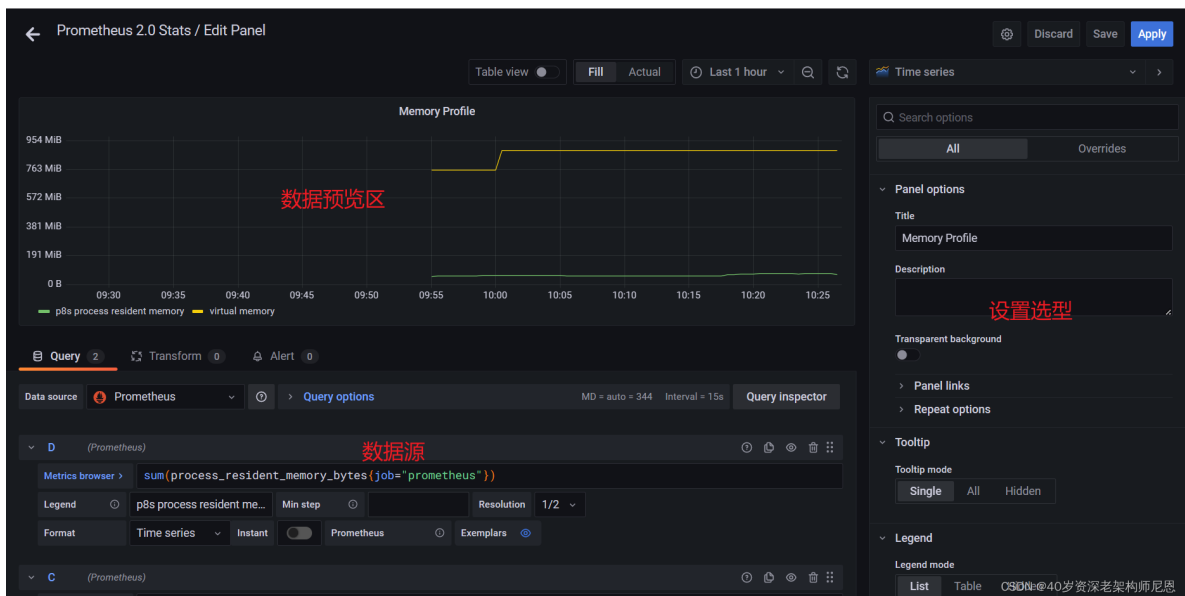
Panel 图表配置

选择一个Panel（图表），点击edit，可以进行配置

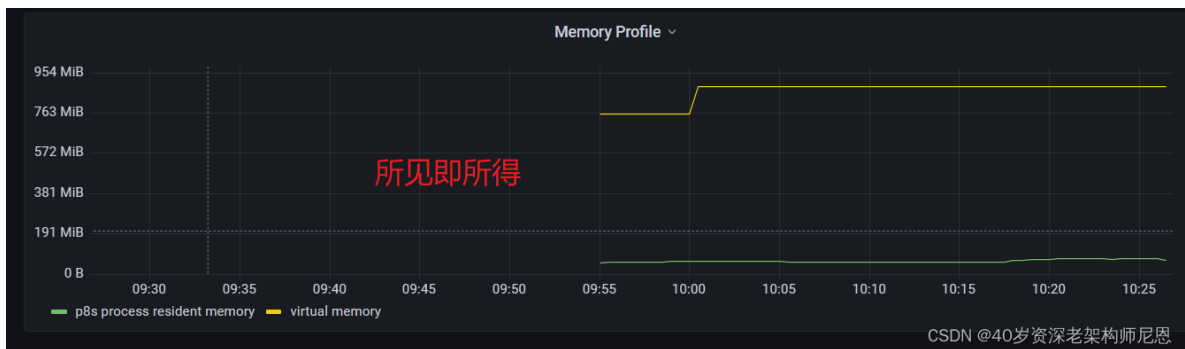


图表（Panel）的设置区域一共可以分为三部分：

- 预览区：用来预览设置的图表。
- 数据设置区：用来设置数据源、数据指标等。
- 图表设置区：用来对图表的名称等进行设置。

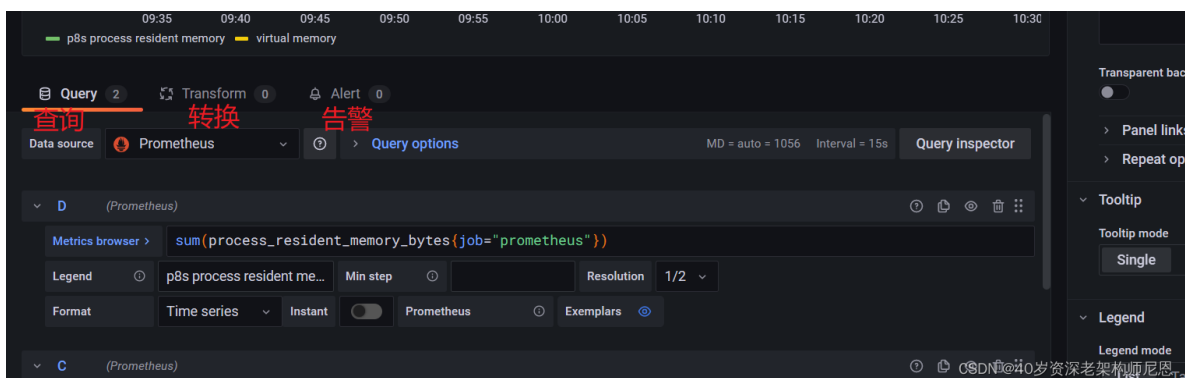


预览区非常简单，就是针对「数据设置区」和「图表设置区」的设置结果，进行模拟显示，从而让我们可以实时看到效果，方便我们进行配置。



数据设置区可以分为三个设置区域，分别是：

- 查询设置区：设置查询数据的数据源、指标等。
- 转换设置区：可以对查询出来的数据进行过滤等。
- 告警设置区：可以对设置的指标设置告警规则。



查询设置区

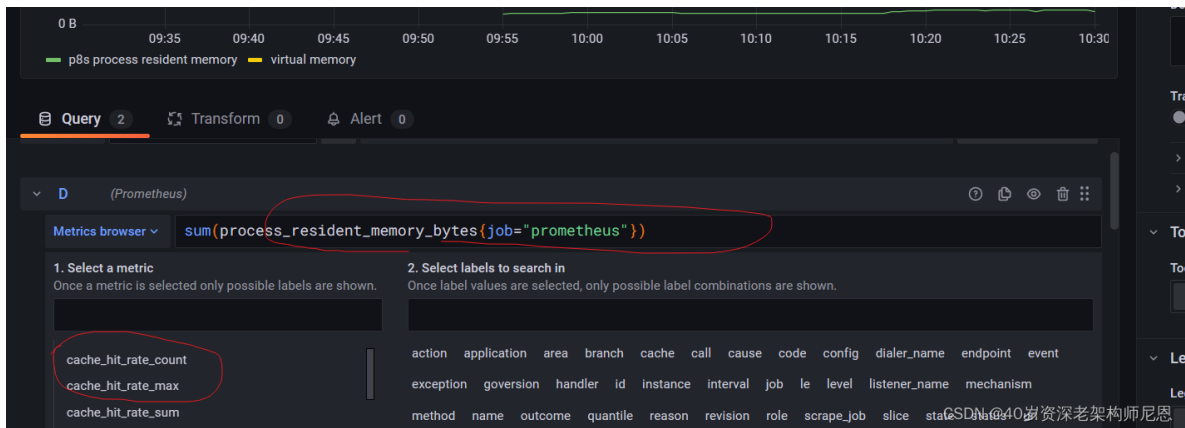
查询设置区可以分为两块，分别是：

- 数据源设置

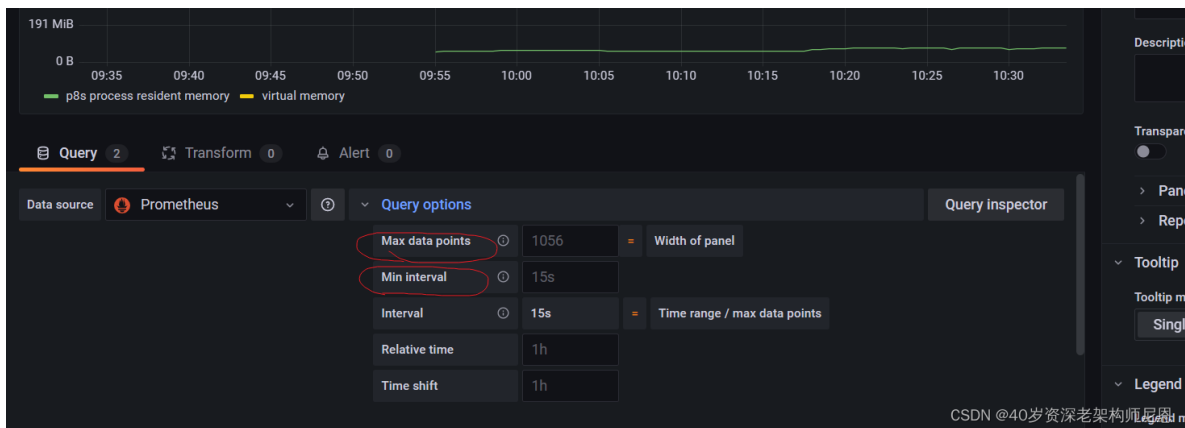
- 数据指标设置

数据源设置

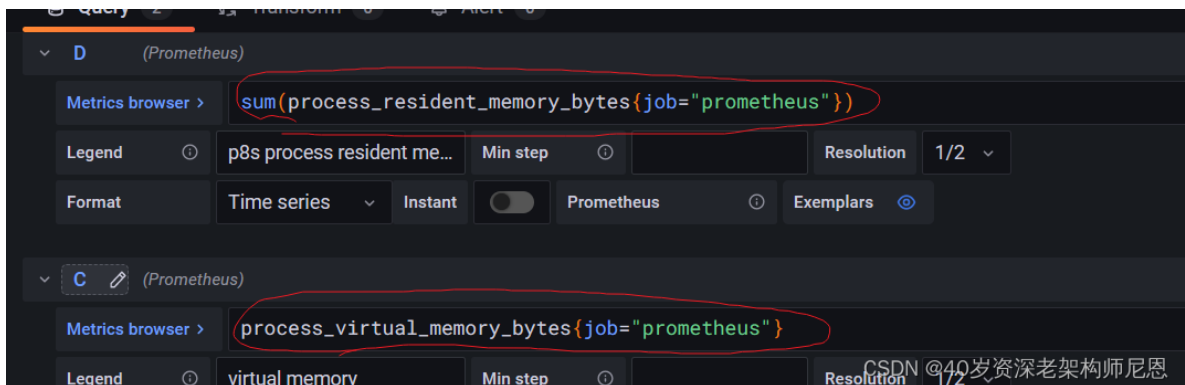
数据源设置用来设置使用哪个数据源。



此外，还可以针对选择的数据进行更精细化的设置，例如：Max data points 等设置。



这块用来设置你想显示数据源中的哪个指标、标识填什么、格式是啥。



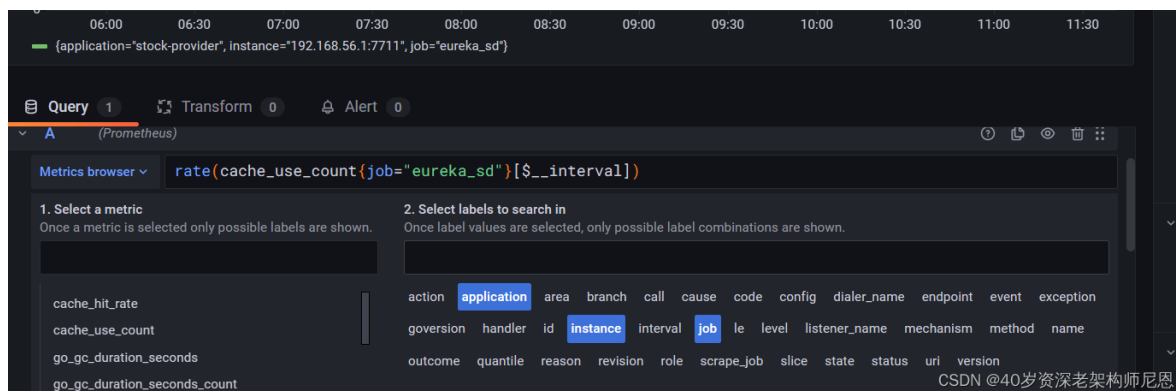
案例：展示缓存使用变化率

- Metrics 指标名

表明这个图表是用来展示数据源中的什么数据，是显示变化率，还是数值，

这里是一个表达式。

通过 面板的 操作步骤，可以 设置 表达式



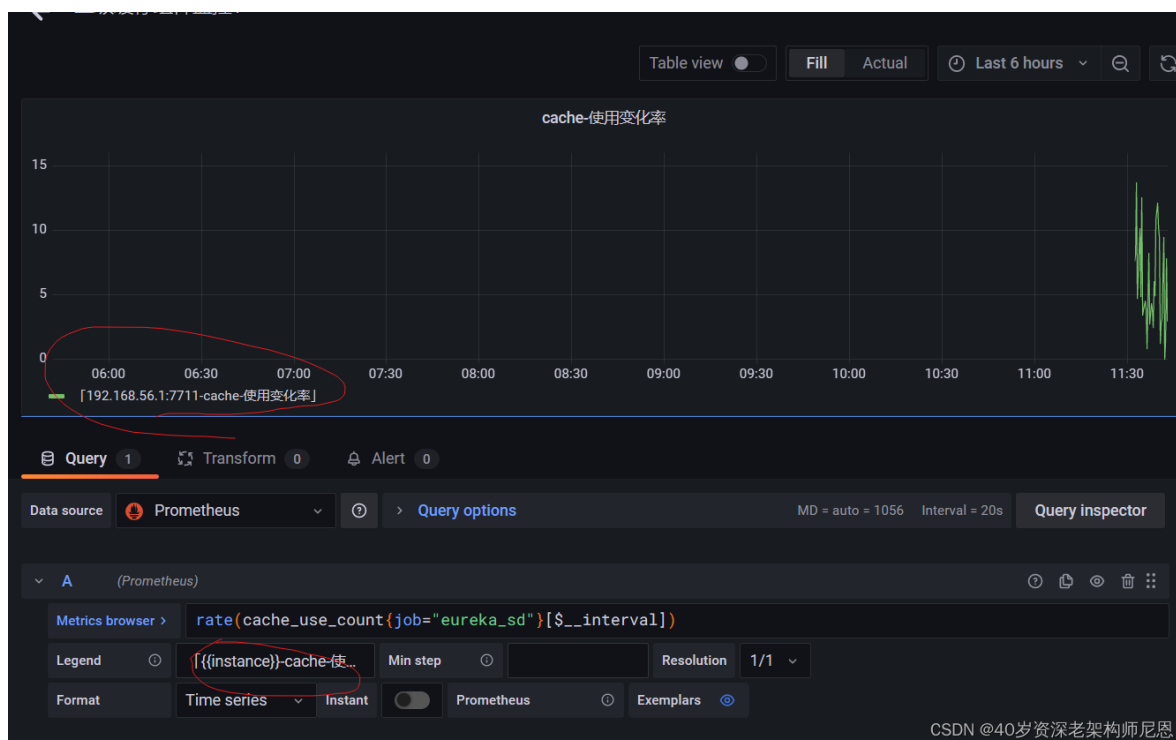
例如我这里是用来显示 缓存使用的变化率的，所以这里是：

```
rate(cache_use_count{job="eureka_sd"}[$__interval])
```

\$__interval是20s，这表示使用 20s 的数据变化率来显示 CPU 的变化情况。

- Legend 图例

图例可以设置图例的显示数值是什么，



例如这里我写了：「[(instance))-cache-使用变化率」，

这表示使用实例名为前缀，后面再加上「-1m」字符串，最终显示为：「192.168.56.1:7711-cache-使用变化率」，

其中 192.168.56.1:7711为实例名。

- Min Step 最小步长

最小步长表示图形里每两个点的最小数据间隔是多少，

例如：这里我设置了 200，那表示图形上每隔 200 个单位才会有一个点。

- Resolution数据精度

这表示其数据精度是怎样的，是 1 比 1 的精度，还是原有的 1/2，还是 1/3 等等。

如果是 1/2 的话，那么就是原本 1 个单位显示一个点，现在 2 个单位合并起来显示成一个点了，那么其精度就变低了。

- Format 格式

表示你的数据格式是什么，这里有：Time series、Table、Heap Map 三个选项。

Time series 表示是时间序列数据，即随着时间的流动有源源不断的数据。

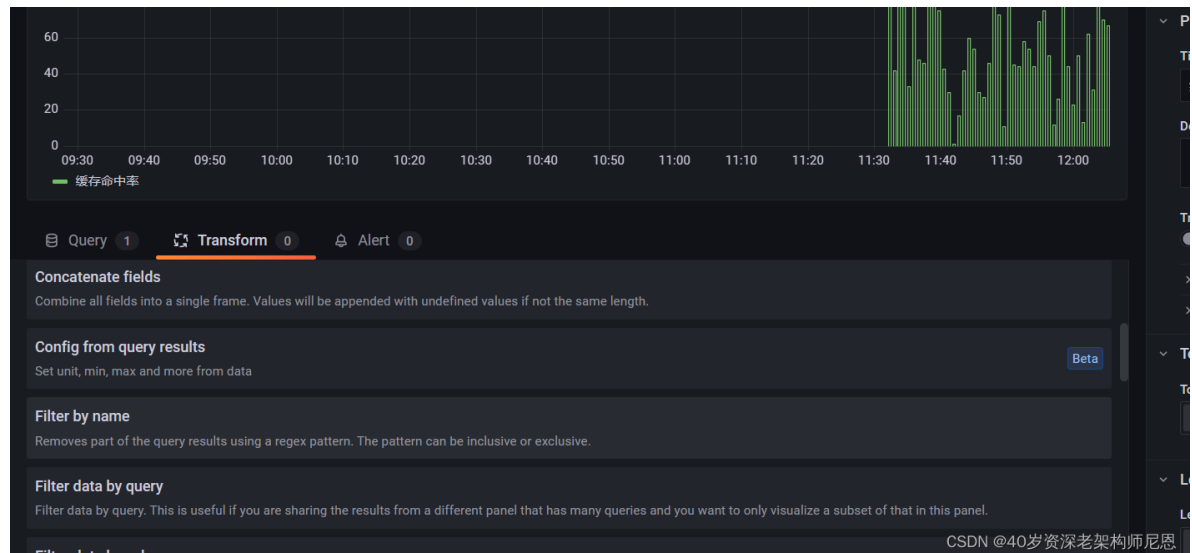
Table 表示是一个表格数据。

Heap Map 表示是热力图数据。

我们可以通过左下角的 Query 来增加显示的数据。

转换设置区

转换设置区主要是用来一些更复杂的数据处理，例如：过滤一些数据，做集合操作等。

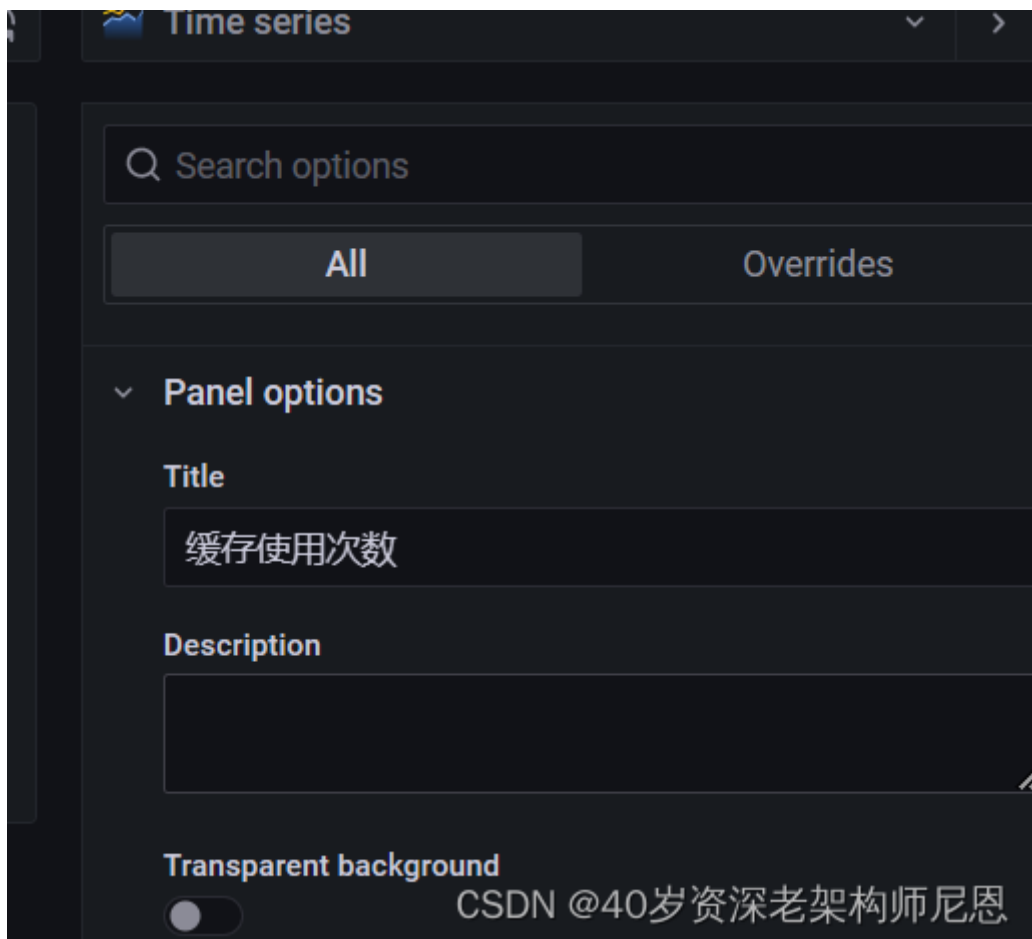


告警设置区

告警设置区用来进行指标告警，这块内容也相对简单，直接看页面就可以弄清楚了。

Setting 设置

用来设置图表名称和描述。

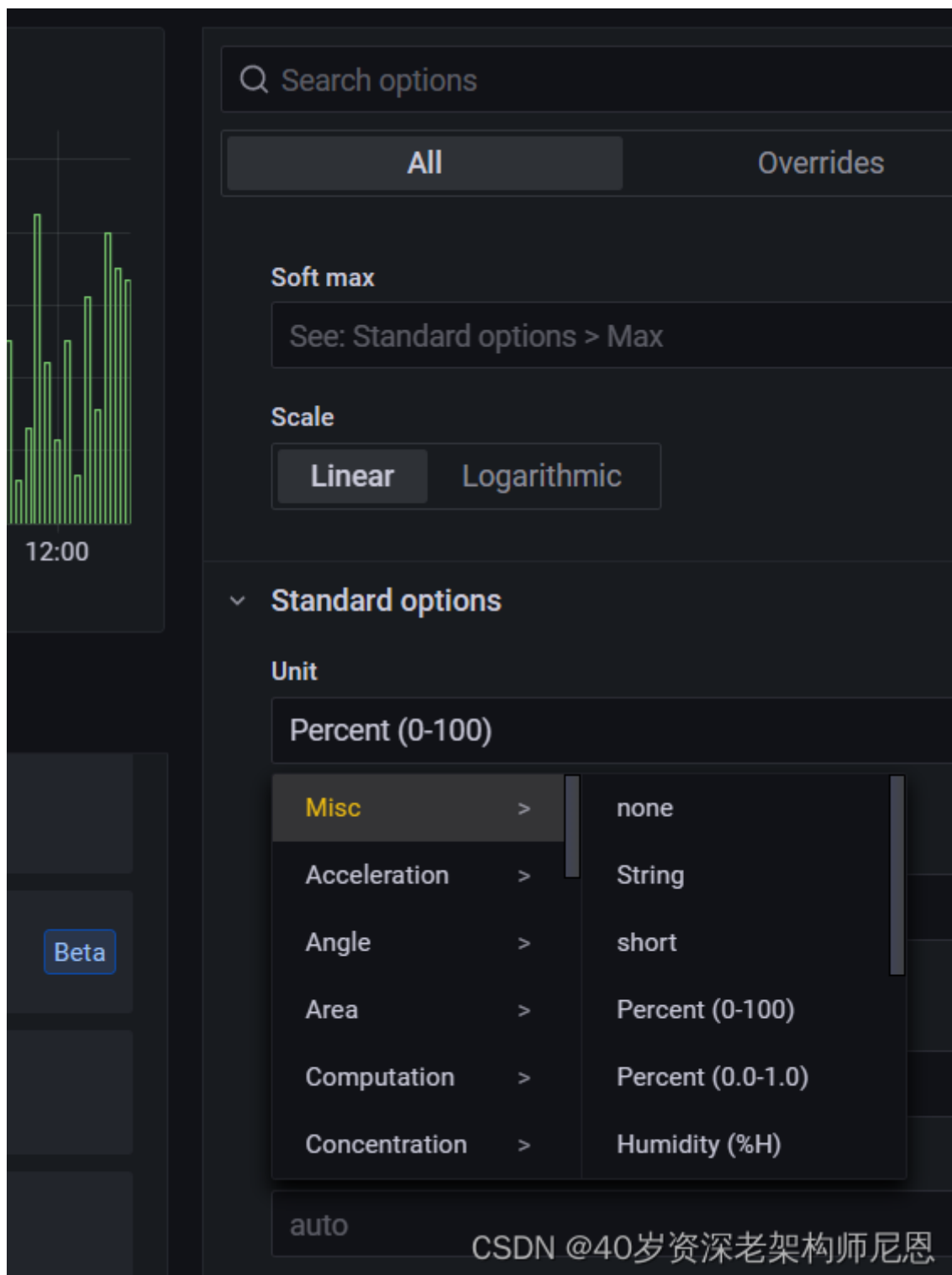


Axes 轴

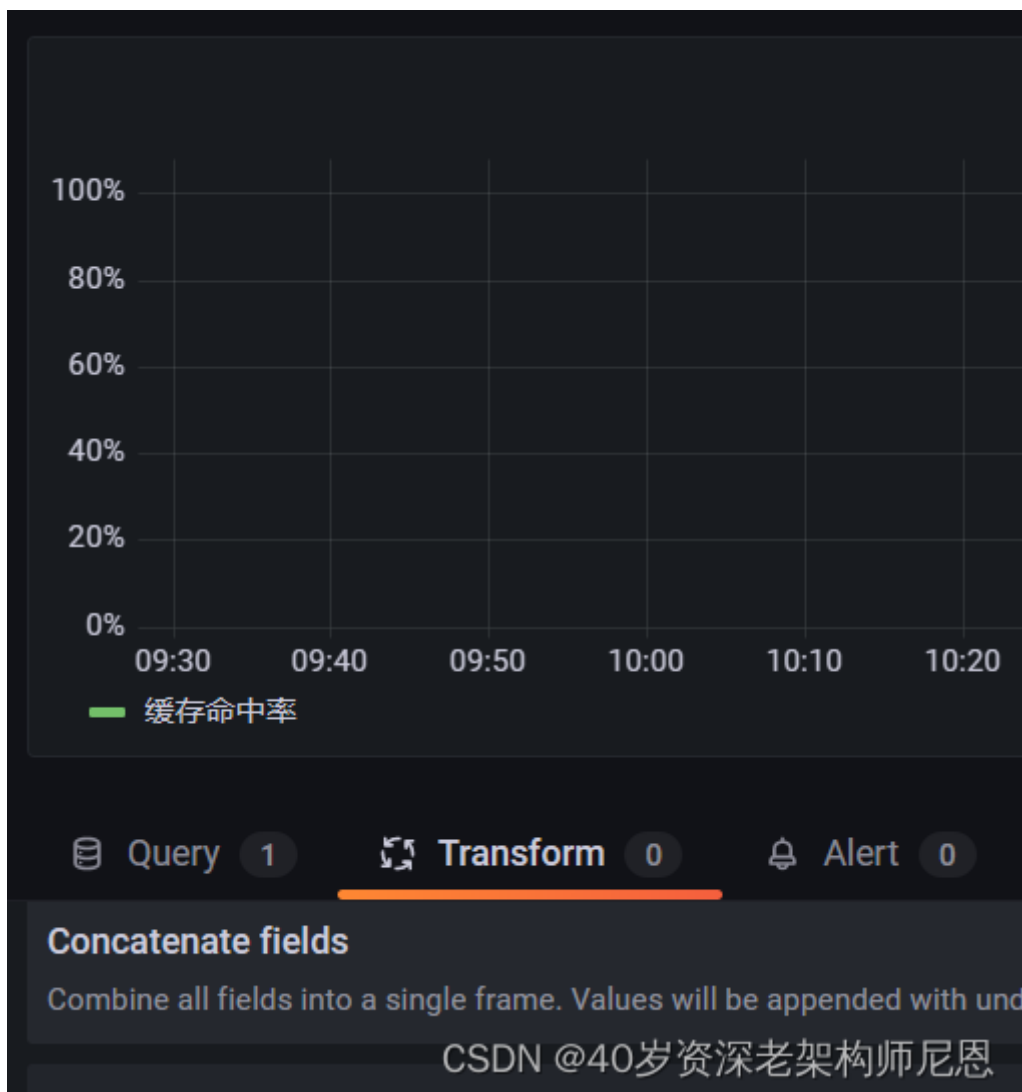
用来设置轴的相关信息，

例如：X/Y 轴的显示单位，X/Y 轴的最小最大值等。

那如果我希望它能用百分比显示，那么我可以设置 Left Y 的 Unit 参数，将其设置为：percent (0.0-1.0)。



效果



其他参数的设置都类似，这里不再赘述了。

legend 图例

这里用来设置图例信息，例如：图例的显示位置、图例显示最大值、最小值。

Thresholds 阈值

用来设置一个阈值，当超过指定的值，会有标记成背景颜色。

通常用来针对某个关键数据，当超过某个值进行高亮，方便一目了然查看数据。

总结

我们通过一个 cache组件使用率的例子，将 Grafana 图表的配置都过了一遍。

总的来说，Grafana 图表配置可分为：预览区、数据配置区、图表配置区。'

'其中涉及到了数据源的配置、数据指标的配置以及各种图表显示的配置等。

虽然我们可以直接使用 Grafana 官网 - Dashboards 模块的现成模板，但如果你想做符合自己业务情况的监控面板，掌握 Grafana 图表的配置可是必不可少哦。

访问的地址

stock-provider

<localhost:7711/stock-provider/actuator/prometheus>

<http://localhost:7711/stock-provider/actuator/prometheus>

prometheus

<http://cdh1:9090/classic/targets>

进入 grafana

grafana 默认密码 admin admin

<http://cdh1:3000/explore>

参考文献

官方资料

1. [prometheus存储](#)
2. [prometheus集成eureka配置示例](#)
3. [prometheus服务发现#eureka](#)
4. [actuator+prometheus+grafana集成1](#)
5. [actuator+prometheus+grafana集成2](#)
6. [SpringBoot Prometheus Metric](#)
7. [Eureka Prometheus PR](#)

<https://github.com/prometheus/prometheus/blob/release-2.25/documentation/examples/prometheus-eureka.yml>

<https://github.com/prometheus/prometheus/blob/main/documentation/examples/prometheus-eureka.yml>

<https://github.com/prometheus/prometheus/tree/main/documentation/examples>

<https://www.cnblogs.com/gaofeng-henu/p/12594820.html>

<https://www.jianshu.com/p/2e905ab659fb>

<https://www.jianshu.com/p/ccffd6b9e3d1>

<https://www.jianshu.com/p/e6ec6a8a6c38>

<https://www.freesion.com/article/83831098412/>

<https://blog.csdn.net/zhipepengfang/article/details/122646692>

<https://www.likecs.com/show-379874.html>

https://help.aliyun.com/document_detail/29342.html

<https://blog.csdn.net/fanliunian/article/details/112259986>

<https://www.modb.pro/db/325486>

<https://nacos.io/zh-cn/docs/monitor-guide.html>

[Nacos 监控手册](#)

<https://github.com/vovolie/lua-nginx-prometheus>

https://blog.csdn.net/weixin_44268481/article/details/122355075

https://blog.csdn.net/fu_huo_1993/article/details/114876026

<https://blog.csdn.net/u012888704/article/details/126232665>

<https://www.my607.com/jianzhanjingyan/2020-10-31/92668.html>

<https://www.jianshu.com/p/acbc04040519>

https://blog.csdn.net/qg_43437874/article/details/120348217

<https://blog.csdn.net/nandao158/article/details/120306729>

<https://blog.csdn.net/b644ROfP20z37485O35M/article/details/120124219>

[prometheus监控JAVA应用（JVM等）并自定义监控指标](#)

https://blog.51cto.com/u_13879334/5733979

[Prometheus 入门教程（七）：SpringBoot 实现自定义指标监控 \(qq.com\)](#)

<https://www.jianshu.com/p/e17bb4808bff>

技术自由圈 技术圣经系列

技术自由圈 7个 学习圣经 PDF

- 1 《SpringCloud Alibaba 学习圣经》** 377页 PDF
v1 版，发布日期：2023年03月15日
- 2 《Docker 学习圣经》** 145页 PDF
v2 版，发布日期：2023年03月12日
- 3 《Kubernetes 学习圣经》** 530页 PDF
v1 版，发布日期：2023年04月11日
- 4 《响应式圣经》** 114页 PDF
v2 版，发布日期：2023年02月14日
- 5 《缓存之王 Caffeine 红宝书》** 175页 PDF
v2 版，发布日期：2022年11月30日
- 6 《队列之王 Disruptor 红宝书》** 75页 PDF
v1 版，发布日期：2022年10月05日
- 7 《Prometheus+grafna 红宝书》** 100页 PDF
v1 版，发布日期：2022年10月08日

技术自由圈 3个 高并发圣经 PDF

- 8 《Java高并发核心编程 卷1 加强版：NIO、Netty、Redis、ZooKeeper》** 609页 PDF
加强版，发布日期：2023年01月01日
- 9 《Java高并发核心编程 卷2 加强版：多线程、锁、JMM、JUC、高并发设计模式》** 448页 PDF
加强版，发布日期：2022年11月14日
- 10 《Java高并发核心编程 卷3 加强版：亿级用户Web应用架构与实战》** 481页 PDF
加强版，发布日期：2022年11月14日

技术自由圈 面试圣经 40个 面试题 PDF

- 11 《尼恩 Java 面试宝典》40个专题** 4000页 PDF
v60 版，发布日期：2023年03月11日

领取方式：技术自由圈 公众号



硬核推荐：尼恩Java硬核架构班

详情：<https://www.cnblogs.com/crazymakercircle/p/9904544.html>



尼恩java 硬核架构班



已经发布

- ★ 《高性能RPC的基础实操之：从0到1开始IM撸一个IM》
- ★ 《分布式高性能RPC的基础实操之：千万级用户分布式IM实操- 含简历指导》
- ★ 《亿级用户超高并发秒杀实操- 含简历指导》
亮点：助力小伙伴搞定70W年薪，N个涨薪50%，**2023夏招面试涨薪神器**
- ★ 《横扫全网，工业级elasticsearch底层原理与高并发、高可用架构实操》
亮点：40岁老架构师细致解读，处处透着分布式、高性能中间件的原理和精髓
- ★ 《第1部曲：超级底层：葵花宝典（高性能秘籍）__架构师视角解读OS操作系统》
亮点：大制作解读OS操作系统，并揭秘mmap、pagecache、zerocopy等底层的底层原理
2023夏招面试涨薪大神器
- ★ 《Rocketmq视频第2部曲：横扫全网工业级 rocketmq 高可用（HA）底层原理和实操》
亮点：起底式、绞杀式解读 rocketmq如何保障消息的可靠性？
- ★ 《Rocketmq视频第3部曲：超级内功篇、横扫全网 rocketmq 源码学习以及3高架构模式解读》
亮点：大制作解读 Rocketmq源码以及3高架构模式，助力大家内力猛增
- ★ 《Rocketmq视频第4部曲：10Wqps消息推送中台架构、设计、编码、测试实操》
亮点：Netty实操、分库分表实操、Rocketmq工业级使用实操
- ★ 《架构师内功篇：横扫全网 netty 高性能、高并发架构 底层原理、源码学习》
- ★ 《架构师实操篇：redis cluster 工业级高可用实操》
- ★ 《架构师实操篇：100W级别QPS日志平台实操》
- ★ 《彻底穿透：skywalking 源码(代表链路跟踪)+Java agent+bytebuddy 探针》
- ★ 《超高并发场景100Wqps三级缓存组件原理和实操》
- ★ 《全链路异步超底层原理和实操：手写hystrix熔断+webflux+Lettuce+Dubbo》

规划中



左手大数据 (写入简历, 让简历 蓬荜生辉、金光闪闪)

HBASE + Flink + ElasticSearch 原理、架构、真刀实操



右手云原生 (写入简历, 让简历 蓬荜生辉、金光闪闪)

K8S + Devops + ServiceMesh 原理、架构、真刀实操

架构师实操篇: 基于netty 手写 rpc 框架- 参考 dubbo、seata rpc框架

架构师实操篇: go语言学习, 以及基于 go 手写 rpc 框架

架构师实操篇: 千万级任务调度平台 架构与实操- 基于尼恩17年的亿级搜索项目

架构师实操篇: 工业级 亿级文档搜索 平台 架构与实操- 基于尼恩17年的亿级搜索项目

特色

会员制

提供技术方向指导,
职业生涯指导, 少躺坑, 少弯路

简历指导

这个很重要,
对于挪窝涨薪来说

实操性

以上项目, 都是老架构师
在生产上实操过的项目

非水货

40岁老架构师, 不是水货架构师
《Java高并发三部曲》为证

架构班（社群 VIP）的起源：

最初的视频，主要是给读者加餐。很多的读者，需要一些高质量的实操、理论视频，所以，我就围绕书，和底层，做了几个实操、理论视频，然后效果还不错，后面就做成迭代模式了。

架构班（社群 VIP）的功能：

提供高质量实操项目整刀真枪的架构指导、快速提升大家的：

- 开发水平
- 设计水平
- 架构水平

弥补业务中 CRUD 开发短板，帮助大家尽早脱离具备 3 高能力，掌握：

- 高性能
- 高并发
- 高可用

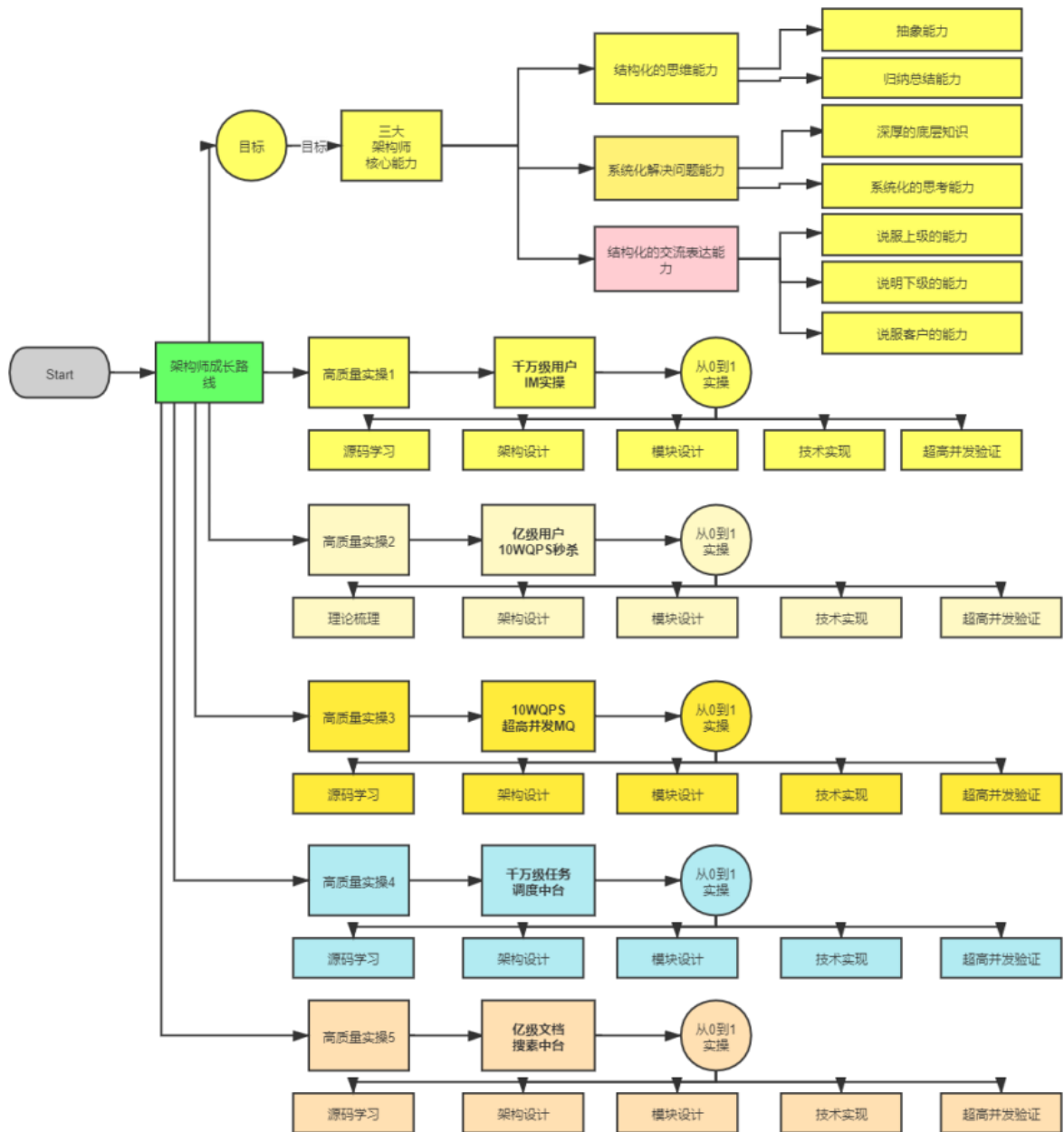
作为一个高质量的架构师成长、人脉社群，把所有的卷王聚焦起来，一起卷：

- 卷高并发实操
- 卷底层原理
- 卷架构理论、架构哲学
- 最终成为顶级架构师，实现人生理想，走向人生巅峰

架构班（社群 VIP）的目的：

- 高质量的实操，大大提升简历的含金量，吸引力，增强面试的召唤率
- 为大家提供九阳真经、葵花宝典，快速提升水平
- 进大厂、拿高薪
- 一路陪伴，提供助学视频和指导，辅导大家成为架构师
- 自学为主，和其他卷王一起，卷高并发实操，卷底层原理、卷大厂面试题，争取狠卷 3 月成高手，狠卷 3 年成为顶级架构师

N 个超高并发实操项目：简历压轴、个顶个精彩



工业级 rocketmq 高可用底层原理和搭建实操，包含：高可用集群的搭建。

- 1、技术难题：RocketMQ 如何最大限度的保证消息不丢失的呢？RocketMQ 消息如何做到高可靠投递？
- 2、技术难题：基于消息的分布式事务，核心原理不理解
- 3、选型难题：kafka or rocketmq，该娶谁？

[illegible]

成功案例：2 年翻 3 倍，35 岁卷王成功转型为架构师

详情：<http://topcoder.cloud/forum.php?mod=forumdisplay&fid=43&page=1>

最新 最后发表 热门 精华

成功案例：[1057号卷王] 3年小伙拿到外企offer，薪酬涨了200%

1 卷王1号 超级版主 前天 17:41

成功案例：[645号卷王] 4年经验卷王逆袭，被毕业后，反涨24W

1 卷王1号 超级版主 2022-9-21

成功案例：[878号卷王] 小伙8年经验，年薪60W

1 卷王1号 超级版主 2022-8-13

年薪70W案例：通过尼恩的指导，小伙伴年薪从40W涨到70W

1 卷王1号 超级版主 2022-2-11

成功案例：[493号卷王] 5年小伙拿满意offer，就业寒冬季逆涨30%

1 卷王1号 超级版主 前天 17:43

成功案例：[250号卷王] 就业极寒时代，收offer 涨25%

1 卷王1号 超级版主 前天 17:38

成功案例：[612号卷王] 就业极寒时代，从外包到自研

1 卷王1号 超级版主 前天 17:15

成功案例：[913号卷王] 热烈祝贺6年经验卷王，年薪40W

1 卷王1号 超级版主 2022-9-21

成功案例：[959号卷王] 4年经验卷王，喜获百度、Boss直聘等N个优质offer，最高涨100%

1 卷王1号 超级版主 2022-9-21

成功案例：[529号卷王] 5年经验卷王喜收2大offer，最高涨5K

1 卷王1号 超级版主 2022-9-21

成功案例：[811号卷王] 热烈祝贺7年经验卷王，薪酬涨30%

1 卷王1号 超级版主 2022-9-21

成功案例：[287号卷王] 不惧大寒潮，卷王逆市收4 offer，涨30%，可喜可贺

1 卷王1号 超级版主 2022-5-30

成功案例：[1002号卷王] 5月份“被毕业”，改简历后，斩获顶级央企Offer，涨薪7000+

1 卷王1号 超级版主 2022-7-5

成功案例: [7号卷王] 热烈祝贺小伙伴涨薪120%

1 卷王1号 超级版主 2022-8-13

成功案例: [134号卷王] 大三小伙卷1年, 斩获顶级央企Offer, 成功逆袭

1 卷王1号 超级版主 2022-7-6

成功案例: [1008号卷王] 5年经验卷王收42W offer, 月涨8000, 可喜可贺

1 卷王1号 超级版主 2022-5-30

成功案例: [453号卷王] 非全日制 6年卷王喜提3 offer, 年薪30W, 可喜可贺

1 卷王1号 超级版主 2022-5-21

成功案例: [924号卷王] 6年卷王喜提4 offer, 最高涨薪9000, 可喜可贺

1 卷王1号 超级版主 2022-5-21

成功案例: [15号卷王] 4年卷王入职 微软, 涨薪50%, 可喜可贺

1 卷王1号 超级版主 2022-5-12

成功案例: [527号卷王] 4年卷王喜提2 offer, 涨薪50%, 可喜可贺

1 卷王1号 超级版主 2022-5-13

成功案例: [788号卷王] 3年卷王喜提优质Offer, 涨薪60%

1 卷王1号 超级版主 2022-5-11

成功案例: 热烈祝贺: 非全日制卷王, 喜提2个心仪offer, 面3家过2家

1 卷王1号 超级版主 2022-4-21

成功案例: [693号卷王] 二线城市6年卷王喜提4大优质Offer, 含央企offer, 最高薪酬35W

1 卷王1号 超级版主 2022-4-16

成功案例: [85号卷王] 双非2本小伙, 春招大捷, 喜提9个offer, 最高薪酬近30万

1 卷王1号 超级版主 2022-4-14

成功案例: [741号卷王] 卷王逆袭! 6年小伙从很少面试机会到搞定35K*14薪Offer

1 卷王1号 超级版主 2022-4-12

成功案例: [642号卷王] 热烈祝贺, 6年卷王喜提优质国企offer

1 卷王1号 超级版主 2022-4-7

成功案例: [796号卷王] 热烈祝贺, 36岁卷王喜提52万优质offer

1 卷王1号 超级版主 2022-3-25

❑ 成功案例: [15号卷王] 小伙卷1年, 涨薪9K+, 喜收ebay等多个优质offer

① 卷王1号 超级版主 2022-3-24

❑ 成功案例: [821号卷王] 小伙狠卷3个月, 喜提10多个offer

① 卷王1号 超级版主 2022-3-21

❑ 成功案例: [736号卷王] 3年半经验收22k offer, 但是小伙志存高远, 冲击25k+

① 卷王1号 超级版主 2022-3-20

❑ 成功案例: 热烈祝贺1群小卷王offer拿到手软, 甚至拒了阿里offer

① 卷王1号 超级版主 2022-3-16

❑ 简历案例: 简历一改, 腾讯的邀请就来了! 热烈祝贺, 小伙收到一大堆面试邀请

① 卷王1号 超级版主 2022-3-10

❑ 成功案例: 祝贺我圈两大超级卷王, 一个过了阿里HR面, 一个过了阿里2面

① 卷王1号 超级版主 2022-3-10

❑ 成功案例: 小伙伴php转Java, 卷1.5年Java, 涨薪50%, 喜收多个优质offer

① 卷王1号 超级版主 2022-3-10

❑ 成功案例: 4年小伙狠卷半年, 拿到 移动、京东 两大顶级offer

 尼恩 超级版主 2022-3-5

❑ 成功案例: [267号卷王] 助力3年经验卷王, 拿到蜂巢的17k x 14薪的offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [143号卷王] 二本院校00后卷神, 毕业没到一年跳到字节, 年薪45W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [494号卷王] 尼恩分布式事务助力卷王拿到 中信银行offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [76号卷王] 2线城市卷王, 狠卷1.5年, 喜收22K offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [429号卷王] 小伙伴在社群卷5个月, 涨8k+

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [154号卷王] 双非学校毕业卷王, 连拿 京东到家&滴滴 两个大厂Offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [232号卷王] 涨薪10K, 继续卷向食物链顶端

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: 狠卷1年技术, 喜收 腾讯、阿里、微软三大Offer, 最高年薪56W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [449号卷王] 应届毕业卷王喜收 滴滴offer, 年薪33W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [551号卷王] 小伙伴学完后, 成功进入大厂, 并且推荐自己的朋友加VIP学习

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: [214号卷王] 助力2年经验卷王, 成功拿到17K月薪

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: [92号卷王] 课程实操助力社群小伙伴喜收 喜马拉雅Offer

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: 社群卷王小伙伴成功过了滴滴三面 获滴滴Offer

① 卷王1号 超级版主 2022-2-10

❑ [612号卷王]滴滴小伙伴, 蹲点考察半年, 觉得靠谱后加入 疯狂创客圈

① 卷王1号 超级版主 2022-2-10

❑ 成功案例: [732号卷王] 尼恩助力3年经验卷王收获 京东offer, 年薪35W

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [558号卷王] 2年经验卷王, 喜收 网易和阿里子公司两个优质offer

① 卷王1号 超级版主 2022-2-27

❑ 成功案例: [569号卷王] 双非应届生卷王, 喜收字节跳动实习offer

① 卷王1号 超级版主 2022-2-25

❑ 成功案例: [420号卷王] 狠卷1年, 卷王涨薪80%, 涨薪12000元!

① 卷王1号 超级版主 2022-2-25

❑ 成功案例: [76号卷王] 通过尼恩1年半的指导, 专科学历小伙伴从0.8K涨到22K

① 卷王1号 超级版主 2022-2-10

简历优化后的成功涨薪案例（VIP含免费简历优化）

6年专科，2年翻4倍

2年从8K涨到35K

2021年从8K涨到22K

高并发 VIP76

老师，求助。

现在有两个满意的 offer，不知道怎么抉择。

一个是吉利，17k，大数据与 ai 部门。

另一个是一个平台，从零开始用 java 重写现在的项目，分布式架构，带团队，自己招人。22k，我觉得我说少了，我自己提的，然后今天发了 offer

呵呵，你太牛了

我也不好说

工资高的是个小公司，不到 50 人

感觉好事都被你占了

这一年半，真的谢谢您。

呵呵，相互交流，相互成长。

您写的书本，解决了我项目上很多问题。您在群里不厌其烦地告诉我们学习，也是我能坚持下来的重要因素，还有每次提问您都能解答疑惑，让我始终能戒骄戒躁。恩师，

**秘诀：
简历指导+ 狠狠卷**

2022年涨到35K

VIP76

解决了，限制 ip 频率。

谢谢老师

中午12:43

调整到了 35,加上这个月加班费，38

中午12:43

老师，我隔壁来了。

晚上8:23

大大的赞

老师你这路子是对的。我就跟着你学习思路和方法，还有教程走的。

我和你一样的兴奋和喜悦

记得咱们去年改简历的时候，还是 10k

这种提升，已经太令人震撼啦

是 8K...

20 年 4 月份转行，就一路跟着你学习



1.5年小伙搞定15K offer

就业寒冬涨100%

5月7日改简历 11月21日晒offer

秘诀:
简历指导+ 狠狠卷

他那个不止那啥... 我才是两...
原先7k, 现在15k

那也很牛...
都翻倍了, 都是牛人...
正常就30%...
慢点来...
下一步可以瞄准25k...
其实我工作一年...
那你就更牛逼拉...
一年经验, 搞定15k offer, 舍你其谁...

卷王逆袭成功案例

6年小伙从很少面试机会到搞定35K*14薪

3月5日改简历 4月11日拿offer

一个月拿到了理想的offer

面试邀请少 小伙很苦恼

理想很丰满 目标35K

面试法宝 rocketmq四部曲

6年 经验小伙伴喜收25K offer

3月12日改简历 12月1日晒offer

秘诀:
简历指导+ 狠狠卷

咱们以这个项目为模板改哈

项目有多少人, 你带领多少人?

你工作几年了?

6年了

改简历脚窝, 20涨到25k, 高级开发, P7带的后端团队

任务重, 道路远, 并不是没有方向

7年经验卷王薪酬涨30%

7月11日改简历 9月1日晒offer

秘诀:
改简历+ 狠狠卷

只要本事好, 拿offer 比较容易的

入职了, 最终并没那么涨薪, 还是旧的太... 只涨了30%

恭喜一下

现在不比往年

能涨30%是大牛

拿到offer 都算大牛

现在很多人投几百发, 连个电话都没有

你已经很... 拉

4年经验卷王逆袭 被毕业后，反涨24W

7月改简历 **8月30日晒offer**

**秘诀：
改简历 + 狠狠卷**

这就是你的简历

老哥 我八月十号开始找工作，今天已经入职了

现金基本持平，股票+24W

能涨24W

股票这个吧只能到手了才算

也不错啦

很多小伙伴，面试机会都没有

感谢老哥的指导，继续跟你卷技术

继续加油吧，马上就技术自由啦

小伙5月份"被毕业"，改简历后 斩获顶级央企Offer 涨薪7000+

5月29日改简历 **7月5日晒offer**

**秘诀：
简历指导+ 狠卷3高**

快速看书，就要不求甚解，把目录和场景大概一下，然后重点的地方，用到的地方，再去回顾

我被"毕业"了

毕业了没有关系

ok，发我吧

it行业，就来跑去，太频繁啦

嗯，其实有点心理准备

拿到offer了

涨20%，2家要多，结果人家都不还价的

看起来半面不差钱呀

超过了8000没

平均算下来

7000多

好的

有面试的心得吗

可以分享给其他小伙伴的

卷王逆袭成功案例 武汉6年喜收4个优质offer 最高的年薪35W

2月9日改简历 **4月15日晒offer**

恩恩老师，新年好！

能帮忙修改下简历吗？

金三银四准备啦

可以的

java开发-6年-简历-.pdf

拜托了，恩恩。希望能拿25k回来给你报喜

好，我加一下

还有吗？

**面试法宝：
改简历 + 实操**

这么多offer，我看看哈

都是恩恩指点有方，本来还有个新能源车，35w给拒了，主要太远了

跟着恩恩卷的时间太短了，目前实力也只能到这了

这边有个大数据的，感觉也不错

卷王逆袭成功案例 6年小伙喜提4个Offer 最高涨9k，年薪35W

4月14日改简历 **5月17日晒offer**

Java开发工程师_dock

52.5 KB

微信电话本

你看到我给你改的

好的呀

麻烦大佬了

这个你自己别哈

不对的，你自己别

那我照着这个改一下库存系统呀

一个简历，哪里哪里

这么漂亮的简历，涨50%，已经没啥问题

只要准备好，不出大批量，基本没问题啦

**涨薪法宝：
改简历 + 狠狠卷**

后面继续跟恩恩卷哈，感觉卷的时间越长，涨薪越快，恩恩，恩恩，都卷透啦

加油卷哈

感觉自己学的不太透彻了

嗯嗯

跟着大佬一起

我周围好几个年薪百万的，都是这

卷王逆袭成功案例

5年经验小伙收2个offer 最高涨薪8k，年薪42W

5月9日改简历

5月30日晒offer

秘诀:
简历指导+ 狠卷3高

以此为样
大家狠狠卷
打造最卷IT社群

卷王逆袭成功案例

非全日制 6年经验卷王 喜提3个Offer，年包30W

5月9日改简历

5月18日晒offer

面试法宝:
改简历+ 狠狠卷

卷王逆袭成功案例

寒五冻六之际卷王大逆袭 收3大offer，涨30%

5月17日改简历

5月27日晒offer

秘诀:
简历指导+ 狠卷3高

卷王逆袭成功案例

4年卷王入职微软，涨50%

3月7日改简历

5月12日晒offer

涨薪法宝:
改简历+ 狠狠卷

4年小伙喜收百度、Boss直聘等N个顶级Offer 最高涨幅100%

6月27日改简历 9月19日晒offer

**秘诀：
改简历 + 狠狠卷**

有个offer选择问题

boss直聘和小满之间，boss那边给的offer，工资比小满高，但是小满那边给的offer，福利比boss高，不知道该怎么选，如果你应该怎么选呀。

还有其他很多offer的，还有一个小满的，总体上看boss和小满之间选一个

了

boss比小满年包多7w以上，我这涨幅都接近百分之百了

太牛啦

按照你那个思路结合上次指导的内容改了几个点，发出去确实大上了很多，加上一些指标的说明确实会让人眼前一亮的感觉。

明天难时候有没了我和我看下哈，一是看两个点我实在不知道该怎么改了，二是我在推荐表达这方面确实不行，还得让你帮忙把把关。

卷王逆袭成功案例 4年卷王入收2个offer，涨50%

3月23日改简历 5月12日晒offer

尼恩老师，我的简历你看了吗，有什么建议？

ok

咱们开始吧

工作经历 这里不需要写描述

下边请描述，每段经历写什么

这个不用加粗

都是一样，加粗了反而没有效果

**涨薪法宝：
改简历 + 狠狠卷**

offer决策圈

n+1 电商erp，部门成熟，人数多，加班少。

n+2 做业务中台 内部系统，新部门，人数少，加班多

又搞到个offer

能搞定两个offer，不尴尬

现在很多小伙伴面试机会都没有

涨了多少钱

地点在哪里

涨50%的样子

忘记你工作几年啦，大概工作几年啦

小伙大三暑期很焦虑 跟着尼恩卷一年 校招斩获顶级央企Offer

去年5月19日加入VIP群 今年7月5日晒offer

**秘诀：
狠狠卷书+视频**

邀请你加入群聊

尼恩老师

我校招去华润电力控股有限公司了

跟着你卷了一年 大学顺便拿了几个国奖

不错不错，这是央企

放空中心

这太牛啦

其实拿到手的也就一个a类国一

尼恩老师 大三的暑期实习找不到 现在准备秋招应该没关系吧

看身边 总是很焦虑 自己算法这一块卡住

网盘里边有算法视频

去刷一刷吧

秋招来得及

谢谢大佬 不过经过几个月练习 看您写的书比之前轻松多了

趁着还是学生这段时间 慢慢把知识吃透

嗯哪，我的书，比较深

期待大佬的下一本书，已经迫不及待去学习了

小伙高中学历 薪酬涨120%

5月6日改简历 7月22日晒offer

你这块估计要送你668

模块的开发工作，就这个三个哈

哈哈，不用了老师，您真的讲了就行 光今天晚上辅导就值几千了

老弟很感谢您

还有很多其他的模块

面试官提问

就是其他人做的

嗯哪，好

**秘诀：
改简历 + 狠狠卷**

之前你的工资是多少来的

翻了一翻

我得送你多少奖金来的

不知道，原价给老弟就行了

咱们得说这话呀

老弟拿着您的那个高并发改造亮点，所向披靡。

从从到尾给面试官讲明白

后面继续狠狠卷哈

卷王逆袭成功案例

非全日制卷王 面试3家
收2个offer 涨薪30%

4月13日改简历

4月21日晒offer

5年卷王喜收2大Offer

最高涨5K

5月19日改简历

9月13日晒offer

面试法宝:
改简历 + 面试题

卷王逆袭成功案例

3年经验卷王，涨60%

4月16日改简历

5月11日晒offer

卷王逆袭成功案例

双非二本小伙春招大翻身
喜提9大offer

2月22日改简历

4月13日晒offer

面试法宝:
改简历 + IM实操

9大offer 最高年薪30万

序号	公司	部门	岗位	薪资结构	总包
1	公司	数据数字化产品部	java后端开发	18.5k+14.5k+5k+200k/每月+500k期权	>30w
2	公司	交易研发部	java后端开发	16k+14k	22.4w
3	公司	待定	java游戏开发	15k+15k+餐补+100k/每月	>22.5w
4	公司	待定	java后端开发	11k+13k	14.2w
5	公司	待定	java后端开发	14k	>16.8w
6	公司	待定	java后端开发	9k	9k
7	公司	待定	java后端开发	9k+包房租+报销津贴	10k+餐补+房补
8	公司	待定	java后端开发	10k+餐补+房补	17w
9	公司	待定	java后端开发	10k+餐补+房补	17w

修改简历找尼恩（资深简历优化专家）

- 如果面试表达不好，尼恩会提供 简历优化指导
- 如果项目没有亮点，尼恩会提供 项目亮点指导
- 如果面试表达不好，尼恩会提供 面试表达指导

作为 40 岁老架构师，尼恩长期承担技术面试官的角色：

- 从业以来，“阅历”无数，对简历有着点石成金、改头换面、脱胎换骨的指导能力。
- 尼恩指导过刚刚就业的小白，也指导过 P8 级的老专家，都指导他们上岸。

如何联系尼恩。尼恩微信，请参考下面的地址：

语雀：<https://www.yuque.com/crazymakercircle/gkkw8s/khigna>

码云：<https://gitee.com/crazymaker/SimpleCrayIM/blob/master/疯狂创客圈总目录.md>